PMS - Under Load

A Structural Self-Critique of the Praxeological Meta-Structure

On Calibration, Coverage, Stack Drift, Publicness,

and Self-Application

0. Front Matter

0.1 Working Assumption / Entry Posture

Function

This paper begins from a controlled assumption:

> **We assume PMS works — and test where that assumption stops being informative.**

This is not a concession to PMS. It is a methodological burden placed on it.

The paper does not begin by treating PMS as a fragile framework that must first prove minimal viability. The current PMS model, in its PMS_1.3 form, is assumed to be productive enough, formally disciplined enough, and portable enough that its limits cannot be found only by looking for obvious failure. A weak model can be tested by asking where it breaks. A strong grammar must also be tested where it continues to work.

The working assumption therefore does not protect PMS. It removes the easy escape of treating failure as the only relevant test.

Entry Posture

The analysis does not ask first whether PMS can generate coherent readings. It can. The more demanding question is whether such coherence remains discriminating once PMS is operating under the very conditions that make it powerful: high portability, strong operatoric readability, cross-domain translation, governance hardening, self-application, implementation pressure, and — now documented in PMS-RUST v0.1 — bounded executable artifact-backed partial realization.

PMS Under Load therefore treats PMS success as a pressure condition.

PMS may lose discriminative force not because it becomes incoherent, but because its coherence becomes too easy to stabilize. Coverage may begin to stand in for distinction. Portability may begin to look like equivalence. Stack extensions may be mistaken for operator identity. Governance constraints may be misread as validation. Implementation pressure, including the documented PMS-RUST v0.1 Evidence Spine, may be mistaken for proof if executable artifact status is allowed to upgrade PMS Base rather than only the implementation claim. Self-application may become a disciplined form of self-stabilization rather than final self-arbitration.

This is the intensified entry posture:

> **PMS works strongly enough that its success must itself be placed under load.**

Methodological Constraint

The paper adopts no simple refutation posture. It does not seek to defeat PMS from an external tribunal.

It also adopts no defense posture. It does not treat PMS productivity, coherence, or portability as evidence of finality.

And it adopts no repair posture. It does not ask how PMS can be made safe by downstream

clarification, governance discipline, or implementation hardening alone.

Instead, the paper asks whether PMS remains structurally discriminating when its own strengths become active at scale.

This means that the analysis remains internally guided and operator-aware, but not self-protective. PMS is tested under its own pressure: Δ (difference), □ (frame), Λ (structured absence / non-event), A (attractor), Ω (asymmetry), Θ (temporality), Φ (recontextualization), X (distance), Σ (integration), and Ψ (self-binding) remain part of the analytic field. But their availability is not allowed to function as automatic confirmation that PMS has preserved distinction.

Pressure Point

The central pressure point is precise:

> **PMS may be strongest exactly where its limits become harder to see.**

If PMS can translate many forms of praxis into its grammar, that translation power is a real strength. But translation is not yet capture. If PMS can generate coherent readings across domains, that coherence is a real achievement. But coherence is not yet sufficiency. If PMS can bind itself through critique, revision, and application discipline, that self-binding is serious. But self-binding is not yet final self-arbitration.

The working assumption therefore increases the burden on PMS. The stronger PMS appears, the less adequate it becomes to ask only whether it fails. The question becomes where its success stops distinguishing and starts stabilizing.

Falsifier

This entry posture can fail.

If assuming PMS works merely shields PMS from challenge, then the assumption has become apologetic. If PMS success is treated as evidence that PMS has already earned priority, then the method has collapsed into defense. If the working assumption turns every later objection into a calibration issue, a governance problem, a stack misunderstanding, or a residual Λ, then the paper has already converted critique into coverage.

The working assumption remains valid only if it makes PMS more vulnerable to pressure, not less.

Transition

This posture leads directly into the abstract. PMS Under Load does not test PMS only at the level of calibration, Φ drift, or meta-normativity. It also tests structural coverage, stack drift, publicness, AI/dialogic stabilization, documented implementation pressure, executable artifact-backed partial realization, and the limit of self-application.

The question is not whether PMS can produce coherent readings.

The question is whether its coherence remains discriminating under the conditions that make PMS powerful.

0.2 Abstract

PMS Under Load is a structural self-critique of the Praxeological Meta-Structure in its current PMS_1.3 form. It does not begin by rejecting PMS, and it does not defend PMS by repeating its internal guardrails. It begins from the stronger and more dangerous assumption that PMS works: it is highly productive, formally disciplined, and cross-domain portable.

That assumption is not protective. It is the condition of the critique.

PMS offers a generative operator grammar for rendering praxis structurally legible across action, asymmetry, temporality, recontextualization, integration, and self-binding. Its strength lies in the way heterogeneous scenes can be translated into a shared operatoric space without immediately collapsing into psychology, moral judgment, or person-ranking. But productivity is not sufficiency. Portability is not equivalence. Implementation is not proof. Executable artifact-backed partial realization is not PMS Base validation. Governance is not tribunal. Self-application is not final self-arbitration.

PMS Under Load therefore tests not whether PMS can generate coherent readings, but whether its coherence remains discriminating under the conditions that make PMS powerful.

The critique proceeds from within PMS' own discipline. It does not use an external philosophical authority as final arbiter, and it does not import a rival framework to rescue or decide the critique. Its pressure is internal: PMS is tested where its own operators, layer boundaries, claim statuses, application gates, and self-binding structures may begin to stabilize what they should still distinguish.

The paper tracks several connected pressure zones. PMS remains formally disciplined, but its operational thresholds are not fully endogenous. X (distance), Λ (structured absence / non-event), Ω (asymmetry), Σ (integration), Ψ (self-binding), and downstream application gates require calibration under concrete conditions. Formal operator identity does not by itself guarantee convergence of application.

PMS also risks structural coverage. A grammar with high translation power may render resistant material legible without thereby preserving all relevant difference. The problem is not crude closure, but the possibility that coverage begins to appear as structural completeness.

A further instability appears at the level of stack interpretation. PMS can remain internally coherent while its layers become externally misread: domain applications may be treated as new grammars, add-ons as primitives, MIP/AH as tribunal, QC as proof, STACK as validation, and implementation artifacts as evidence of truth. PMS-RUST v0.1 intensifies this pressure by documenting a bounded executable PMS-STACK Evidence Spine, but that documentation strengthens only the implementation/artifact claim. It does not validate PMS Base. High portability and executability increase the risk that layer boundaries are mistaken for operator equivalence.

Recontextualization (Φ) adds another pressure point. Φ is indispensable to PMS, but it becomes unstable where recontextualization stops preserving difference (Δ) and starts making non-fit compatible. Reframing is not reversal; internal framing does not erase outer legibility.

The paper also tests the claim that PMS is non-normative. PMS is not a classical moral theory, but

it is not neutral. It structurally privileges certain forms of practice: distance over fusion, reversibility over closure, non-coercion over imposition, and structural reading over tribunal judgment. This is not first-order prescription, but meta-normative selection.

Success itself becomes a further risk. Coherence, portability, reproducibility, documented tests, traces, demos, and dialogic or AI-assisted stabilizability can make PMS-consistent outputs easier to produce than difference-sensitive interruptions. PMS-RUST makes this pressure more concrete: executable tooling, JSONL traceability, golden fixtures, and acceptance gates can increase artifact stability without proving that PMS has preserved discriminative force. AI may amplify PMS' stabilizability without validating PMS' truth.

Publicness changes the operating conditions of PMS. Public circulation can compress temporality, intensify asymmetry, weaken reversibility, and turn structural description into perceived positioning. PMS is therefore not uniformly scalable across exposure regimes.

Finally, PMS can apply itself to itself, but self-application is not final self-arbitration. The grammar can expose its own limits only through the same grammar whose limits are under examination. That does not make self-critique impossible. It means that disciplined self-exposure must not be mistaken for completed self-settlement.

The aim of PMS Under Load is therefore neither demolition nor reassurance. PMS may survive the critique as a strong, bounded, calibration-dependent grammar of praxis legibility. But it survives only if its strength is reduced to its proper scope: productive without being self-authorizing, portable without being equivalent to all domains, implemented or partially realized without being proven, executable in bounded artifact form without validating PMS Base, governable without becoming tribunal, and self-critical without claiming final self-arbitration.

A self-critique of PMS remains incomplete unless it leaves open the possibility that some failures are not drift, not misapplication, not calibration residue, not stack confusion, and not Λ waiting to be integrated, but signs of genuine non-capture. Rival models remain epistemically open. PMS may be powerful, but its power must remain distinguishable from privilege. Positive warrant would require external application under calibrated, rival-sensitive, and falsifier-bearing conditions; Under Load prepares that possibility, but does not replace it.

0.3 Source Scope

Function

PMS Under Load responds to the current source state of PMS.

The paper does not reconstruct older versions and does not use earlier formulations as evidence against the current model. Its object is PMS in its current PMS_1.3-compatible form, together with the current domain papers, add-ons, governance layers, bridge layers, and implementation-layer artifacts insofar as they are explicitly available in the project context.

This fixes the source posture before the critique begins.

Source Scope

The source basis of the paper is layered.

PMS Base functions as the source of truth for the Δ–Ψ operator grammar. Domain layers are read as operator-strict applications and stress layers. Add-ons are read as precision, application, or hardening layers. MIP/AH are read as downstream governance, application-discipline, and attack-surface hardening layers under the constraint rule. QC is read as bridge mapping. STACK is read as an implementation-layer claim: realization horizon, implementation pressure, and — in the documented PMS-RUST v0.1 repository — bounded executable artifact-backed partial realization of a PMS-STACK Evidence Spine.

This distinction is decisive.

PMS-RUST is now part of the current source state only as implementation evidence. It is a public repository artifact that documents a bounded runtime/toolchain spine, including kernel execution, model validation, REPL/script tooling, JSONL trace/audit output, demos, tests, and AI proposal boundaries. It does not become PMS Base, does not complete PMS 1.3 semantic migration, and does not replace external validation.

PMS Under Load does not critique obsolete source states. It critiques the current constrained model under the assumption that its own layer discipline is already in force.

Pressure Point

The pressure point is the maturity of the current source state itself.

A weak critique would rely on outdated formulations or already-abandoned claims. A stronger critique tests whether PMS remains discriminative after its current constraints, guardrails, and layer boundaries have been installed.

The question becomes:

> Does current discipline preserve discriminative force, or does it make PMS easier to stabilize under its own constraints?

Falsifier

The source posture can fail.

If PMS Under Load uses obsolete formulations to attack the current model, it fails its own source discipline.

If it treats unavailable or superseded material as current authority, it creates a phantom source problem.

If it treats layer discipline as proof that PMS is safe, it converts source control into reassurance.

Boundary Conditions

This section does not claim that all current PMS materials are complete, final, or equally strong.

It only fixes how source status is handled.

Current materials may still contain pressure points. Domain layers may still drift. Add-ons may still over-harden. QC may still invite bridge overclaim. STACK and PMS-RUST may still invite implementation, executability, test-as-proof, repo-as-validation, or runtime-dialect overclaim. MIP/

AH may still be misread as tribunal. But those instabilities must be tested against the current constrained state of the materials.

Transition

Once the source state is fixed, the next task is layer discipline.

The paper must define which layer may authorize which kind of claim. Without that discipline, PMS Under Load would risk reproducing the very stack drift it is designed to expose.

0.4 Layer Discipline

Function

PMS Under Load must not generate the drift it criticizes.

Because PMS is highly portable, its layers can easily appear more equivalent than they are. Base grammar, domain application, add-on precision, governance hardening, bridge mapping, and implementation architecture can all use the same operator vocabulary. That shared vocabulary is powerful. It is also dangerous.

High portability increases the risk that layer boundaries are mistaken for operator equivalence.

This section therefore fixes the layer discipline of the paper.

Core Rule

> **A claim becomes unstable when it borrows authority from a layer other than the one in which it is made.**

This rule governs the entire critique.

A base claim must not be proven by an implementation layer. A domain claim must not redefine PMS operators. An add-on must not become a primitive. Governance must not become evidence of truth. A bridge mapping must not become proof of substrate identity. A realizable architecture must not become validation of the base grammar.

The paper may compare layers, but it must not allow one layer to silently authorize another.

Layer Matrix

The working matrix is:

```
PMS Base       = Source of Truth for Δ–Ψ
Domain Papers  = operator-strict stress/application layers
Add-ons        = precision / application / hardening layers, not primitives
MIP/AH         = Maturity in Practice / AH Precision, governance-oriented hardening unde
QC             = bridge / structural mapping
STACK          = implementation-layer claim / realization horizon / implementation press
PMS-RUST       = documented implementation artifact / executable artifact-backed partial
```

Each line restricts what the layer can and cannot do.

PMS Base

PMS Base is the source of truth for the eleven operators Δ–Ψ.

Only the base layer may define:

- operator identity,
- operator order,
- dependency structure,
- base-level non-goals,
- and the minimal conditions under which PMS remains PMS.

Domain papers, add-ons, QC, STACK, and MIP/AH may instantiate, stress, refine, operationalize, harden, or map PMS. They may not redefine the base grammar.

If an applied layer appears to alter Δ–Ψ, that claim must be read as unstable unless explicitly reduced back to application status.

Domain Papers

Domain papers are operator-strict stress and application layers.

They may show how PMS behaves in a concrete field such as critique, conflict, logic, anticipation, sexuality, or comparison drift. They may reveal domain-specific pressure on calibration, publicness, asymmetry, recontextualization, or integration. They may expose where PMS becomes more or less stable under domain load.

But they are not new grammars.

A domain paper may say:

> This is how PMS behaves under this domain pressure.

It may not say:

> This domain determines what PMS operators really mean.

Domain layers can stress the base. They cannot replace it.

Add-ons

Add-ons are precision, application, or hardening layers.

They may improve usability, make outputs more transparent, force attack surfaces into view, standardize precision summaries, clarify governance boundaries, or reduce misuse risk.

But add-ons are not primitives.

An add-on becomes unstable if it starts functioning as though PMS requires it in order to be valid at the base level. The add-on may constrain application, but it does not define the operator grammar. It may harden use, but it does not prove the model.

Add-ons are allowed to reduce misuse. They are not allowed to rescue PMS from base-level critique.

MIP/AH

MIP/AH names Maturity in Practice together with AH Precision as an optional Attack Surface / Hardening addon. In PMS Under Load, it functions as a governance-oriented application-discipline and analysis-artifact hardening layer under constraint.

Its value lies in application discipline: role sensitivity, attack-surface visibility, hardening for the next iteration, anti-tribunal safeguards, reversibility, dignity protection, and misuse resistance.

But MIP/AH must remain downstream.

It may constrain how PMS is applied. It may warn against person-ranking, moralization, shaming, and tribunal drift. It may require that claims remain scene-bound, reversible, and role-aware.

But it must not become a tribunal.

It must not decide that PMS is valid because PMS can be governed. It must not convert dignity safeguards into proof of base correctness. It must not make application discipline stand in for operator sufficiency.

The governing sentence is:

> **Downstream governance may constrain application, but it must not be converted into evidence for PMS Base.**

This is especially important because MIP/AH can make PMS feel safer. That safety is real as a constraint on use, but it is not evidence that the base grammar has resolved its own calibration, coverage, or self-application problems.

QC

QC is a bridge and structural mapping layer.

It may render quantum-computational structures in PMS terms. It may map frames, superposition, oracle asymmetry, iteration, measurement, attractor structures, and stabilization into Δ–Ψ language. It may support documentation, annotation, audit, and bounded structural comparison.

But QC is not proof.

QC must not be read as evidence that PMS is physically universal, substrate-final, or validated by quantum computation. A bridge mapping can be strong without becoming an ontological claim. It can show structural reach without proving equivalence.

QC therefore remains a BRIDGE CLAIM.

Its correct use in PMS Under Load is to test bridge stress:

- where mapping preserves difference,
- where mapping begins to over-equate,
- where analogy risks becoming equivalence,
- and where structural rendering may be mistaken for substrate proof.

The key reduction is:

> Structural mapping is evidence of bridge reach, not proof of substrate universality.

STACK

STACK is an implementation-layer claim: a realization horizon and implementation-pressure layer. With PMS-RUST v0.1, STACK also has documented executable artifact-backed partial realization in the limited form of a PMS-STACK Evidence Spine.

It may propose how PMS operators can be projected into OS architecture, CPU design, instruction semantics, runtime structure, programming language design, security, networking, and distributed systems. That is a serious implementation claim.

But STACK is not validation.

PMS-RUST sharpens rather than removes this boundary. The repository documents a bounded Rust runtime/toolchain with deterministic kernel execution, PMS.yaml model validation, REPL/script tooling, JSONL trace/audit output, demos, tests, golden fixtures, vFS service surface, and AI proposal bridge. These features strengthen the implementation/artifact claim. They do not validate PMS Base, prove PMS 1.3 semantic completion, or establish empirical adequacy of PMS as a praxis grammar.

Implementation pressure is not proof. Realizability is not truth. Architectural coherence is not base sufficiency. A system may be implementable because it is well-structured, explicit, and generative. That does not establish that PMS captures all relevant structures of praxis, computation, or organization.

STACK therefore remains an IMPLEMENTATION CLAIM; where artifacts exist, executability strengthens the implementation-layer claim, not PMS Base.

Its role in PMS Under Load is to intensify the critique, not resolve it. If PMS can be implemented as a coherent stack, the question becomes sharper:

> Does implementation preserve PMS' discriminative force, or does it make PMS easier to stabilize as an operating grammar?

The implementation layer raises pressure because it can make PMS look more validated than it is. PMS-RUST makes this pressure concrete: repository structure, passing gates as documented, executable demos, and trace artifacts can intensify the appearance of validation while still remaining implementation-layer evidence only. Under Load must prevent that inference.

The correct reduction is:

> STACK may show realization pressure; PMS-RUST may show bounded executable feasibility and artifact-backed partial realization; neither validates PMS Base.

Stack Drift

Stack drift occurs when these layer boundaries weaken.

Typical drift forms include:

- domain layers becoming new base grammars,

- add-ons becoming primitives,
- MIP/AH becoming tribunal or rescue layer,
- QC becoming proof,
- STACK becoming validation,
- PMS-RUST becoming PMS Base evidence,
- implementation pressure, executability, passing tests, traces, or repository structure becoming truth,
- governance discipline becoming evidence of model correctness,
- bridge mapping becoming equivalence,
- portability becoming universality.

These are not merely presentation errors. They alter the authority structure of the model.

A claim that begins in one layer may borrow authority from another layer and thereby become stronger than its source permits. That is why layer discipline is not editorial housekeeping. It is a core methodological condition of the critique.

Pressure Point

PMS' strength increases this risk.

Because the same operator grammar travels across domains, add-ons, governance layers, bridge mappings, and implementation architectures, layer continuity can appear as layer equivalence.

But continuity is not equivalence.

A domain paper can use Δ–Ψ without becoming the base. An add-on can harden application without becoming primitive. MIP/AH can constrain use without proving the model. QC can map a structure without proving substrate identity. PMS-RUST can partially realize a PMS-STACK Evidence Spine in executable artifact form without validating PMS Base.

The more portable PMS becomes, the more explicit this discipline must be.

Falsifier

The stack-drift concern would weaken if PMS claims remained interpretable, reversible, and properly typed across all layers without authority borrowing.

If a BASE claim remains grounded only in PMS Base, if a DOMAIN claim remains limited to domain stress, if an ADD-ON claim remains hardening-only, if a GOVERNANCE claim remains application-constraining, if a BRIDGE claim remains mapping-only, and if an IMPLEMENTATION claim remains realization-only, then stack drift does not occur.

In compact form:

> If claims remain interpretable, reversible, and properly typed across layers without authority borrowing, stack drift does not occur.

Boundary Conditions

Layer discipline is not a ban on cross-layer analysis.

PMS Under Load may and must compare layers. It may show how a domain layer stresses the base. It may ask whether add-ons over-harden application. It may test whether MIP/AH constrains without rescuing. It may examine whether QC bridge claims overreach. It may use STACK to expose implementation pressure.

But cross-layer comparison must remain claim-typed.

A layer may place pressure on another layer. It may not silently authorize it.

Transition

Layer discipline leads directly into claim-typing.

If PMS Under Load is to avoid the drift it criticizes, every strong sentence must know what kind of claim it is: base, domain, add-on, governance, bridge, implementation, self-application, speculative, or out of scope.

0.5 Claim-Typing Protocol

Function

PMS Under Load binds every strong claim to its correct status.

A claim must not exceed the authority of its layer. This is not an administrative requirement. It is part of the critique.

PMS can remain internally coherent while individual claims silently borrow authority from the wrong layer. A bridge claim can begin to sound like proof. A governance claim can begin to sound like base validity. An implementation claim can begin to sound like validation. A domain claim can begin to function as a new grammar. These shifts may preserve rhetorical coherence while weakening methodological discipline.

Claim-typing therefore asks not only:

> Is this claim coherent?

It also asks:

> Does this claim have the right status?

Claim Types

The paper uses the following claim types:

```
BASE CLAIM
DOMAIN CLAIM
ADD-ON / HARDENING CLAIM
GOVERNANCE CLAIM
BRIDGE CLAIM
IMPLEMENTATION CLAIM
SELF-APPLICATION CLAIM
SPECULATIVE CLAIM
```

```
OUT OF SCOPE
```

These types are not labels added after the argument. They determine what kind of authority a claim may have.

Assignment Rules

The working assignment is:

```
PMS operator grammar = BASE CLAIM

ANTICIPATION / CRITIQUE / CONFLICT / SEX / EDEN / LOGIC = DOMAIN CLAIM

MIP/AH = GOVERNANCE CLAIM / ADD-ON-HARDENING CLAIM

QC = BRIDGE CLAIM

STACK = IMPLEMENTATION CLAIM: realization horizon / implementation pressure

PMS-RUST = IMPLEMENTATION / ARTIFACT CLAIM: documented executable artifact-backed
partial realization of PMS-STACK Evidence Spine v0.1

Self-application claims = SELF-APPLICATION CLAIM
```

Each assignment restricts what the claim may do.

A BASE CLAIM may define or constrain the operator grammar. A DOMAIN CLAIM may show how PMS behaves under a specific field of application. An ADD-ON / HARDENING CLAIM may sharpen application, precision, misuse resistance, or artifact quality. A GOVERNANCE CLAIM may constrain use, publicness, reversibility, dignity safeguards, and accountability conditions. A BRIDGE CLAIM may map structural relations across domains or substrates. An IMPLEMENTATION CLAIM may propose realization, architecture, execution, system design, or executable partial realization. PMS-RUST currently supports only the bounded artifact version of that claim: documented executable artifact-backed partial realization of a PMS-STACK Evidence Spine v0.1. A SELF-APPLICATION CLAIM may test PMS through PMS, but may not claim final self-arbitration. A SPECULATIVE CLAIM must remain explicitly provisional. OUT OF SCOPE marks claims that the paper does not authorize.

Status Discipline

A claim becomes unstable when it changes force without changing type.

This matters because many PMS claims can be made to sound stronger by borrowing from nearby layers. A QC claim may be structurally elegant, but it remains a bridge claim. A STACK claim may be architecturally coherent, and PMS-RUST may make parts of a PMS-STACK Evidence Spine executable, inspectable, and testable, but the result remains an implementation/artifact claim. MIP/AH may strengthen application discipline, but they remain governance or hardening claims.

Domain papers may produce high-resolution readings, but they remain domain claims unless they are explicitly grounded back in PMS Base.

The paper therefore treats claim inflation as a structural risk.

A base claim is not stronger because PMS is portable. A base claim is not stronger because PMS is implementable or executable in parts. A base claim is not stronger because PMS is governable. A base claim is not stronger because PMS can be self-described inside PMS. Each of those features may strengthen the claim appropriate to its own layer, but each must remain typed.

Layer-Specific Reductions

The following reductions govern the paper:

```
QC is a BRIDGE CLAIM, not proof.

STACK is an IMPLEMENTATION CLAIM: realization horizon and implementation pressure — not

PMS-RUST is an IMPLEMENTATION / ARTIFACT CLAIM: documented executable artifact-backed pa

MIP/AH are GOVERNANCE CLAIMS or ADD-ON / HARDENING CLAIMS, not rescue layers.

ANTICIPATION, CRITIQUE, CONFLICT, SEX, EDEN, and LOGIC are DOMAIN CLAIMS, not base gramm

PMS operator definitions are BASE CLAIMS, not domain-derived effects.
```

These reductions do not weaken the project. They make it testable.

If a claim remains strong after its type is fixed, the claim may stand. If its force depends on sliding into another layer, it must be reduced.

Pressure Point

The pressure point is that PMS' coherence can conceal status drift.

A sentence may remain PMS-compatible while becoming methodologically unstable. It may use the correct operators, preserve internal readability, and still borrow force from the wrong layer. This is especially dangerous in a project where the same operator vocabulary travels across base grammar, domain application, bridge mapping, governance hardening, and implementation architecture.

Claim-typing is therefore one of the places where PMS Under Load tests PMS' own portability.

The question is not simply whether PMS can say the sentence.

The question is whether the sentence knows what kind of claim it is.

Falsifier

The claim-typing protocol can fail.

If a strong claim cannot be typed without losing the force it needs, then the claim was overextended.

If a bridge claim only works by sounding like proof, the bridge claim has overreached.

If a governance claim only works by implying base validity, the governance claim has overreached.

If an implementation claim only works by implying PMS Base validation, the implementation claim has overreached.

If every strong claim can be typed without changing its force, the paper's claim discipline holds.

Boundary Conditions

Claim-typing does not forbid strong claims.

It forbids untyped strength.

PMS Under Load may still make strong claims about PMS' productivity, discipline, portability, calibration pressure, stack drift, publicness, bridge stress, implementation pressure, PMS-RUST artifact evidence, or executable partial realization, and self-application limits. But each such claim must remain bounded by its source layer and falsifiable within its assigned status.

A claim may move between layers only if the movement is explicit, justified, and reduced.

Transition

Once claim types are fixed, the paper must define the special constraint governing MIP/AH.

MIP/AH are close enough to application discipline, governance, publicness, dignity safeguards, and attack-surface hardening that they will matter throughout the critique. But they must not become the layer through which PMS is rescued from base-level pressure or made to appear valid by governance discipline.

0.6 MIP/AH Constraint Rule

Function

MIP/AH are visible in PMS Under Load, but they are constrained from the beginning.

MIP refers to **Maturity in Practice**, a praxeological-anthropological model concerned with maturity as enacted under concrete conditions rather than as person-ranking. AH refers to the optional **AH Precision addon**: an Attack Surface / Hardening layer that evaluates the analysis artifact for scope drift, language drift, inference risk, and misuse potential. It is additive and removable; it does not mutate scores, operators, or PMS Base. In PMS Under Load, MIP/AH are not treated as PMS Base. They enter only as governance, application-discipline, and analysis-artifact hardening layers for applied PMS outputs.

They may discipline application. They may not rescue PMS Base.

MIP/AH are application-critical where PMS produces artifacts, public claims, or reusable analyses. They are not validity-critical for PMS Base.

The governing rule is:

> **MIP/AH constrain application; they do not rescue the base grammar.**

This rule prevents a specific form of drift. Because MIP/AH make PMS application safer, more reversible, more role-aware, and more misuse-resistant, they can make PMS feel more stable than it has earned at the base level. That stabilizing effect is admissible only as application discipline. It is inadmissible as evidence that PMS has resolved its operatoric, calibration, coverage, or self-application problems.

Admissible Uses

MIP/AH may be used to clarify:

```
application boundaries
misuse risks
publicness constraints
attack-surface hardening conditions
no-person-ranking
claim-typing discipline
downstream governance
role sensitivity
reversibility conditions
dignity safeguards
```

These uses matter because MIP/AH is relevant precisely where PMS does not remain only at the level of base grammar. It is also applied, documented, extended, hardened, and now partially realized in bounded executable artifact form through PMS-RUST v0.1. Whenever PMS produces artifacts, domain readings, public claims, governance-adjacent outputs, or reusable procedures, application discipline becomes necessary.

MIP/AH may therefore ask whether an application is bounded, reversible, non-humiliating, role-aware, scene-bound, and protected against tribunal drift.

But this remains downstream.

Inadmissible Uses

MIP/AH may not be used to:

```
resolve objections against PMS Base
compensate for operator-level ambiguity
repair calibration problems
convert governance into evidence of PMS validity
convert critique into compliance audit
shield PMS from structural pressure
turn dignity safeguards into model proof
turn publicness discipline into epistemic privilege
```

If a base-level objection concerns Δ–Ψ, MIP/AH cannot resolve it.

If PMS has an operator-level ambiguity, MIP/AH cannot compensate for it by making application safer.

If calibration remains underdetermined, MIP/AH cannot turn procedural caution into threshold precision.

If PMS risks over-coverage, MIP/AH cannot prove that coverage remains discriminative.

If PMS can apply itself to itself, MIP/AH cannot convert that self-application into final self-arbitration.

Constraint Logic

MIP/AH become necessary where PMS produces applied artifacts, public-facing outputs, or reusable analysis procedures.

They are dangerous because applied artifacts often require governance.

PMS-RUST makes this point concrete. A public implementation artifact with REPL tooling, scripts, traces, tests, documentation, and AI proposal boundaries requires application discipline and artifact hardening. But that hardening remains downstream. It may improve responsible use of the repository; it may not validate PMS Base.

Governance can make a model appear more mature than its base claims warrant. A well-governed application may reduce harm, clarify roles, improve reversibility, and prevent misuse. But those achievements do not prove that the underlying grammar is sufficient. They only show that application has been constrained.

This distinction must remain intact.

MIP/AH may say:

> This PMS application is bounded enough to remain admissible under these conditions.

They may not say:

> PMS Base is valid because this application is governed.

MIP/AH may say:

> This output should be revised because it risks tribunal drift, person-ranking, or publicness harm.

They may not say:

> PMS Base is protected from critique because MIP/AH can detect misuse.

MIP/AH may harden the artifact.

They may not immunize the grammar.

Pressure Point

The pressure point is that MIP/AH are genuinely useful.

If they were merely irrelevant, there would be no risk. Their strength is exactly what makes them dangerous inside PMS Under Load. They clarify use, reduce abuse, and sharpen application boundaries. But that same clarity can be misread as a solution to base instability.

MIP/AH must therefore be kept under constraint precisely where they are most helpful.

Their task is to discipline application without absorbing critique.

Falsifier

The MIP/AH constraint rule can fail.

If MIP/AH enter the argument in order to resolve a base-level objection, the rule has failed.

If they are used to compensate for operator ambiguity, the rule has failed.

If they convert governance into evidence of PMS Base validity, the rule has failed.

If they turn PMS Under Load into a compliance audit rather than a structural self-critique, the rule has failed.

The rule holds only if MIP/AH enter the argument to discipline application and never to rescue PMS Base.

Red Lines

The following red lines apply throughout the paper:

```
No rescue-layer drift.
No tribunal drift.
No person-ranking.
No ranking of dignity.
```

```
No use of governance as proof.
No conversion of critique into compliance.
No shielding of PMS Base through downstream hardening.
```

These red lines do not make PMS safer at the base level. They only define what must not happen when PMS is applied.

Key Sentence

> **MIP/AH are admissible in PMS Under Load only where PMS is tested as an applied stack; they are inadmissible where they would function as a rescue layer for PMS Base.**

Transition

The Front Matter now establishes the conditions under which Chapter 1 can begin.

PMS will be criticized internally, but not rescued by its downstream governance layers. Its operators remain available, but their availability is not treated as proof. Its applications remain relevant, but their discipline is not converted into PMS Base validity. Its bridges, STACK projections, and PMS-RUST implementation artifacts remain pressure sources, not validation layers.

The critique can now proceed.

1. Stance & Method

Chapter Aim

This chapter establishes the critical posture of PMS Under Load.

The paper does not perform a compliance audit of PMS. It performs structural self-exposure under conditions where PMS may remain internally coherent and formally disciplined while losing discriminative force.

This distinction governs the method. A compliance audit asks whether PMS follows its own rules. Structural self-exposure asks whether those rules can themselves become part of PMS' stabilizing intelligibility.

The chapter sharpens four methodological commitments:

1. PMS is direction-bearing, not neutral.
2. Valid critique must create real internal danger for PMS.
3. Re-audit is not self-critique.
4. Dialogic and AI-assisted stabilization may be methodologically relevant without validating PMS.

The aim is not to defeat PMS from outside. It is also not to confirm that PMS respects its own guardrails. The aim is to test whether PMS can remain formally disciplined while losing the ability to distinguish where distinction matters.

1.1 Direction-Bearing, Accountable, Non-Tribunal

PMS does not need to pretend neutrality.

It has direction.

It selects for certain forms of praxis legibility: structural description over person-labeling, distance over fusion, reversibility over fixation, non-tribunal reading over moral verdict, and operatoric reconstruction over unbounded interpretation. These selections are not defects. Without direction, PMS could not operate at all.

But direction is not privilege.

The fact that PMS has a disciplined direction does not mean that its direction is mature, superior, final, or epistemically entitled. Direction is a condition of operability. It is not a mark of authority.

The question is therefore not whether PMS has direction, but whether that direction remains explicit, accountable, and revisable under its own grammar.

This matters because PMS can easily appear safer than it is. Its refusal of person-ranking, its structural language, its reversibility discipline, and its avoidance of tribunal judgment all make PMS more responsible as an application grammar. But those features do not exempt its own direction from critique. They only define how critique must be conducted if it remains inside the grammar.

A direction-bearing model can become unstable in two opposite ways.

It may hide its direction and present itself as neutral structure. In that case, normativity operates invisibly.

Or it may make its direction explicit but treat that direction as already justified. In that case, transparency becomes self-authorization.

PMS Under Load rejects both moves.

PMS is not neutral. But its non-neutrality does not grant epistemic superiority. It only creates an additional burden: the model must remain accountable for the direction through which it renders praxis legible.

The stance is therefore non-tribunal.

PMS Under Load does not rank PMS from above. It does not declare PMS mature because it is structurally restrained. It does not treat rival frameworks as already defeated because PMS can redescribe them. It does not turn direction into worth.

The methodological claim is narrower:

> **PMS is direction-bearing, and that direction itself must remain under critique.**

If PMS can make its direction explicit without weakening its claims, its method gains discipline. If PMS cannot make its direction explicit without losing force, then its apparent neutrality was doing hidden work.

1.2 Valid Critique Inside PMS

A statement about PMS is not yet critique simply because it is critical.

Critique inside PMS must satisfy stronger conditions. It must be operatorically expressible, falsifiable, reversible, non-immunized, and capable of failing. It must create real structural danger for PMS while remaining disciplined enough not to become external model substitution.

This is a narrow corridor.

If critique steps entirely outside PMS and imports an external authority as final arbiter, it may be philosophically interesting, but it no longer tests PMS from within. It replaces the model rather than placing the model under load.

If critique stays inside PMS but only confirms that PMS obeys its own guardrails, it becomes re-audit. It checks hygiene. It does not yet endanger the grammar.

Valid internal critique must do more.

It must expose a case in which PMS can remain formally disciplined and still lose discriminative force. The critical case is not crude breakdown. It is a condition where PMS continues to work, continues to generate coherent readings, continues to observe its own constraints, and yet becomes less able to distinguish between rival structural readings without absorbing them.

PMS-CRITIQUE is important here because it already disciplines critique as a structural, scene-bound, non-personal operation. It does not treat critique as reaction, judgment, moralized positioning, or mere commentary. It treats critique as a configuration that must remain

interruptible, recontextualizable, integrable, and potentially bound into follow-up.

But PMS Under Load does not use that discipline as reassurance.

The current CRITIQUE layer already shows that grammar-closure, drift-totalization, non-capture, and rival-framing are recognized and disciplined as methodological risks. PMS Under Load therefore does not ask whether CRITIQUE can name those risks. It asks whether such protections remain sufficient when PMS' own guardrails become part of its stabilizing intelligibility.

This is the sharper question:

> **The aim is not to confirm that PMS respects its own guardrails, but to test whether those guardrails can themselves become part of PMS' stabilizing intelligibility.**

A guardrail can prevent misuse. It can also pre-select admissible critique. A reversibility requirement can prevent fixation. It can also make certain forms of resistance appear insufficiently disciplined. A no-tribunal stance can prevent moralism. It can also make normative conflict easier to redescribe as structural drift. A non-capture clause can preserve openness. It can also become a controlled place where resistance is acknowledged without threatening the model.

These are not accusations against PMS. They are method conditions for testing a strong grammar.

Internal critique must therefore remain capable of producing a result PMS does not want. It must leave open that a PMS-consistent reading may be too smooth, too stabilizing, or too absorptive. It must leave open that some resistance is not drift, not misapplication, not poor calibration, not residual Λ (structured absence / non-event), and not incomplete reframing, but genuine non-capture.

The falsifier is direct:

> **If internal critique only confirms that PMS obeys its own rules, it has become re-audit rather than self-exposure.**

1.3 Method: PMS Applied to PMS

PMS Under Load applies PMS to PMS.

This is not self-validation.

The method uses PMS' own grammar to examine how PMS behaves under conditions of success: portability, coherence, coverage, application discipline, layer extension, governance hardening, implementation pressure, and self-description.

The method is self-application, but its claim is not that PMS becomes true because it can describe itself. A grammar can describe its own tensions and still stabilize them. A model can name its own risks and still make those risks easier to contain than to undergo. A framework can expose its limits and still lack final authority to arbitrate what its exposure has missed.

Self-application therefore has two sides.

On the one hand, it is a real strength. PMS can turn its own operators toward its own operation. It can ask whether difference remains preserved, whether recontextualization absorbs too much,

whether distance becomes routine, whether integration becomes too smooth, whether self-binding turns critique into acceptable form, and whether structured absence becomes a category for non-capture that has not yet been allowed to remain non-captured.

On the other hand, self-application intensifies the risk of stabilization. The same grammar that exposes the model also determines what counts as exposure. The same operators that identify drift also define what drift can look like. The same guardrails that keep critique disciplined also constrain what kinds of critique can enter the field.

This is why PMS Under Load does not ask only:

> Can PMS critique itself?

It asks:

> Where does PMS' self-critique become internally coherent self-stabilization?

Self-application is valid only if it can endanger the model. It must not become a demonstration that PMS can always include its own objections.

The key distinction is therefore:

> **PMS can expose its own limits, but exposure is not the same as final arbitration.**

Exposure names a limit, renders it visible, and places it into structural language. Arbitration would decide, without remainder, whether the limit is internal drift, calibration residue, rival framing, genuine non-capture, or something outside PMS' useful reach.

PMS may be able to do the first with unusual discipline.

It does not thereby prove that it can do the second finally.

1.4 Directionality Without Privilege

Direction is not the problem.

Unaccountable direction is.

Every grammar that renders praxis legible selects. It foregrounds some differences, stabilizes some frames, treats some absences as meaningful, gives some asymmetries priority, and makes some forms of integration more natural than others. PMS is no exception.

Its direction is visible in its preferences: structural over psychological, scene-bound over person-bound, reversible over fixed, distance-preserving over fused, non-tribunal over verdictive, and operatoric over impressionistic.

PMS Under Load does not ask PMS to eliminate those preferences. That would make the model unusable.

It asks PMS to keep them explicit and revisable.

This is especially important because direction can become hidden normativity when it is treated as neutral structure. If PMS says only "this is what the operators show," it may obscure the fact that operatoric legibility itself selects for certain forms of admissibility. Some claims become easier to

state in PMS terms. Others become harder. Some disturbances become drift. Some refusals become non-capture. Some normative intensities become structural pressure. Some rival framings become external rather than internal.

None of this invalidates PMS.

It means PMS has a direction and must pay the cost of having one.

A direction-bearing model remains usable only where its direction remains open to critique. If that direction cannot be made explicit without weakening PMS' claims, then PMS' apparent neutrality is carrying hidden authority.

The falsifier is simple:

> **If PMS' direction cannot be made explicit without weakening its claims, then its apparent neutrality is doing hidden work.**

PMS Under Load therefore does not seek a view from nowhere. It seeks accountable direction.

It does not criticize PMS for having commitments. It criticizes any move by which those commitments become invisible, privileged, or immune.

1.5 Critique vs Coverage

The central distinction of PMS Under Load is the distinction between critique and coverage.

Coverage means that PMS can render something intelligible within its grammar.

Critique means that PMS is placed under pressure in a way that may alter, reduce, or endanger its claims.

These are not the same.

A model with high coverage can translate many objections into its own terms. It can identify drift, recalibrate thresholds, add layer discipline, mark non-capture, distinguish misuse, and show how rival framings operate. These are strengths. But they also create a specific risk: critique may be absorbed as further evidence of reach.

PMS may fail not where it lacks coverage, but where coverage begins to replace discrimination.

This is the danger of a highly generative grammar. If every challenge becomes legible too quickly, the model may preserve coherence while losing contact with what the challenge was meant to disturb. A resistant case may be translated into PMS terms without being allowed to remain resistant. A rival framing may be acknowledged without being given enough force to endanger the grammar. A breakdown may be redescribed as drift, calibration variance, or residual non-capture without changing the model's practical authority.

PMS Under Load therefore asks:

> Where does PMS stop distinguishing and start stabilizing?

This question does not deny PMS' strength. It depends on it.

Weak models fail by incoherence. Strong models may fail by over-stabilization. PMS belongs to the

second risk class. It may continue to produce disciplined readings while making it difficult to tell whether distinction has been preserved or merely translated.

Critique begins where coverage is no longer enough.

The falsifier is equally clear:

> **If PMS can preserve difference without translating every challenge into internal drift, coverage does not replace critique.**

The later chapters take up this distinction across calibration, stack drift, Φ drift, meta-normativity, success trap, domain drift, publicness, and self-application. Chapter 1 only fixes the methodological demand: PMS must not be allowed to treat all intelligibility as evidence of discrimination.

1.6 Re-Audit vs Self-Exposure

A re-audit checks whether PMS follows its own rules.

Self-exposure tests whether those rules can become part of PMS' stabilizing machinery.

This distinction is essential. PMS already contains strong guardrails. It can demand distance, reversibility, scene-binding, no person-ranking, no tribunal drift, no dignity ranking, no moralized diagnosis, and clear separation of application from critique. Those constraints matter. They prevent misuse. They make PMS applications more responsible.

But showing that PMS respects them is not yet self-critique.

If the chapter's critique would disappear once PMS is shown to respect its guardrails, the chapter has confused re-audit with self-exposure.

PMS Under Load must therefore ask a harder question:

Can the guardrails themselves become stabilizing?

A no-person-ranking rule can prevent harm. It can also redirect pressure away from forms of critique that require sharper normative language. Reversibility can preserve openness. It can also delay commitment. Scene-binding can prevent overgeneralization. It can also fragment a structural challenge into local cases. Application discipline can prevent misuse. It can also make PMS appear safer than its base claims warrant.

This does not mean guardrails are bad.

It means guardrails must be tested as part of the model's stabilizing capacity.

MIP/AH (Maturity in Practice / AH Precision: Attack Surface / Hardening) enters here only as a method boundary. It may clarify no-person-ranking, no-tribunal stance, reversibility discipline, artifact responsibility, and misuse-awareness. It may help specify what would make an application inadmissible, dangerous, or publicness-sensitive. It may keep the paper from becoming a weaponized artifact.

But it must not reduce the pressure of critique on PMS itself. It may constrain the artifact produced by critique; it may not decide whether the critique has structurally endangered PMS.

The governing sentence is:

> **MIP/AH constrain the conditions of admissible application; they do not reduce the pressure of the critique on PMS itself.**

This prevents two errors.

The first error is tribunal drift: using MIP/AH to decide that PMS is mature, legitimate, or ethically superior.

The second error is rescue-layer drift: using MIP/AH to compensate for base-level problems in PMS, such as operator ambiguity, calibration underdetermination, over-coverage, or self-application limits.

PMS Under Load is not a compliance audit. It is not satisfied when PMS behaves well. It asks whether "behaving well" can itself become the way PMS remains protected from deeper pressure.

1.7 Dialogic / AI-Assisted Stabilization Note

This paper was developed under dialogic conditions.

That fact is methodologically relevant, but not validating.

Human–AI dialogue can amplify the stabilizability of PMS-like grammars. It can make operator vocabulary more consistent, preserve layer distinctions, detect contradictions, repeat guardrails, generate revisions, and stabilize wording across iterations. These are real effects. They may increase coherence, reproducibility, and apparent maturity of the output.

But dialogic reproducibility is not truth.

AI may amplify PMS' stabilizability without validating PMS' truth.

This point must remain non-autobiographical. The issue is not the psychology of the author, the authority of the assistant, or the sincerity of the dialogue. The issue is structural: a grammar that is explicit, modular, recursive, and highly portable may be especially easy to stabilize through iterative dialogue. If so, the resulting clarity is a methodological fact, not a proof.

The danger is subtle.

AI-assisted refinement may make PMS more precise while also making it easier to converge on PMS-consistent formulations. It may increase internal hygiene while decreasing friction. It may help expose drift while also helping absorb objections into well-typed categories. It may make the model more readable while also strengthening the impression that readability equals validity.

This does not make AI assistance illegitimate.

It makes it part of the load condition.

PMS Under Load therefore treats dialogic stabilization as a success-trap precursor. If PMS becomes increasingly coherent through dialogue, the question is not whether that coherence is useful. It is. The question is whether the process also increases the risk that alternative readings, rival grammars, or genuine non-capture are translated too smoothly into PMS-compatible form.

The falsifier is:

> **If dialogic reproducibility does not increase stabilization, convergence, or output coherence, then AI-assisted stabilization is not structurally relevant in the way this chapter claims.**

The claim is intentionally bounded.

AI does not prove PMS. Dialogue does not validate PMS. Reproducibility does not establish truth. Coherence does not establish sufficiency.

It only sharpens the burden: PMS must be tested not only where it struggles, but where it becomes unusually easy to stabilize.

Chapter Result

Chapter 1 fixes the stance and method of PMS Under Load.

PMS is treated as strong enough to warrant internal critique. Its direction is named rather than hidden. Its critique must be capable of endangering it. Its self-application is used without being treated as self-validation. Its coverage is distinguished from critique. Its guardrails are respected but also tested as possible stabilization mechanisms. Its dialogic and AI-assisted reproducibility is named as a load condition, not as evidence of truth.

The chapter therefore establishes the methodological threshold for everything that follows:

> **PMS Under Load tests not whether PMS can remain coherent, but whether its coherence remains discriminating when its own success, discipline, and stabilizability are placed under pressure.**

Chapter 2 does not inherit that strength from this chapter. It must show why PMS is strong enough to warrant this level of critique.

2. What PMS Does Well

Chapter Aim

This chapter establishes the baseline before destabilization begins.

It does not defend PMS. It does not advertise PMS. It shows, briefly and fairly, why PMS is strong enough to warrant pressure.

PMS produces genuine gains in praxis legibility, drift detection, misuse awareness, and cross-domain portability. These strengths are real. But they do not validate PMS. They create the conditions under which the Under Load critique becomes necessary.

A weak grammar can be dismissed where it fails to generate coherent readings. PMS cannot be tested only there. Its more serious risk appears where it succeeds: where it renders praxis legible, tracks drift, travels across domains, and stabilizes interpretations so effectively that the difference between critique and coverage becomes harder to maintain.

The baseline claim is therefore precise:

> **PMS is strong, therefore PMS must be tested where strength itself becomes unstable.**

2.1 Legibility Gain

PMS' first strength is not final explanation.

It is the production of structured legibility where practice would otherwise remain diffuse.

Diffuse situations often contain many elements at once: action, omission, timing, asymmetry, expectation, role, exposure, failed response, attempted repair, and changing context. Without an operator grammar, these elements tend to collapse into narrative, psychology, moral judgment, or loose description. PMS slows that collapse. It gives the analyst a way to distinguish what is structurally present before deciding what it means.

A critique scene, for example, can be read not as "someone being critical," but as a configuration in which Δ (difference) becomes framed by □ (frame), X (distance) makes interruption possible, Φ (recontextualization) may transform the scene, and Σ (integration) may or may not convert interruption into correction. In this way, CRITIQUE shows PMS' capacity to make critique legible as a structured operation rather than as attitude, personality, or rhetorical force.

A conflict scene can likewise be read not as mere escalation or communication failure, but as stabilized incompatibility under Ψ (self-binding), Ω (asymmetry), Θ (temporality), and costly X (distance). CONFLICT shows PMS' capacity to distinguish between disagreement, critique, competition, and conflict by asking whether Σ (integration) remains viable, whether Ψ (self-binding) has hardened, whether Θ (temporality) has made reversal expensive, and whether exit or distance is asymmetrically costly.

LOGIC adds a different form of legibility. It shows that responsibility can become structurally attributable without being converted into an "ought." Under Ω (asymmetry), Θ (temporality), Λ (structured absence / non-event), and Ψ (self-binding), consequences become readable without requiring PMS to generate a moral code. This is a real gain: PMS can make responsibility visible

without immediately turning responsibility into prescription.

ANTICIPATION adds upstream legibility. It shows that future-oriented praxis can be read through Λ (structured absence / non-event), Θ (temporality), and X (distance) before an event occurs. Anticipation is not reduced to prediction, strategy, or control. It is made legible as the disciplined carrying of an open non-event under temporality and restraint.

These examples show PMS at its best.

It does not merely name things. It separates operators that ordinary discourse tends to merge. It distinguishes absence from nothingness, asymmetry from moral hierarchy, responsibility from prescription, critique from attack, conflict from intensity, and anticipation from forecast.

This is a genuine strength.

But it also creates the first pressure point.

Legibility can become overconfidence if readability is mistaken for adequacy. A situation may become clearer in PMS terms without being exhausted by PMS terms. A grammar can improve structural visibility while still missing decisive features, rival distinctions, or forms of resistance that do not translate cleanly.

The correct baseline sentence is therefore:

> **PMS increases structural legibility under defined operator constraints.**

Not:

> PMS reveals the true structure of praxis.

The gain is real, but bounded.

2.2 Drift Detection

PMS is also strong at drift detection.

It can identify repeatable shifts in praxis structure rather than treating every failure as an isolated error. This matters because many important failures do not appear as simple contradiction. They appear as mutation: critique becomes performance, anticipation becomes control, logic becomes closure, conflict becomes moralized narrative, or domain application becomes hidden operator inflation.

CRITIQUE is especially clear here. Critique can drift when interruption loses distance, when correction becomes commentary, when recontextualization replaces integration, when publicness amplifies exposure without adding corrective capacity, or when non-events accumulate as unmade critique, delayed response, or avoided follow-up. PMS can track these shifts because it reads critique as an operator chain that can break, overload, or mutate at identifiable points.

CONFLICT extends the same capacity under heavier load. It shows drift where critique no longer remains viable: where integration outcomes diverge, self-binding becomes partially set, temporality raises reversal costs, and distance remains formally available but becomes expensive or asymmetrically distributed. PMS can distinguish conflict from mere escalation because it looks for

stabilized non-integrability rather than affective intensity.

LOGIC identifies another drift form: attribution can flatten into moral implication. PMS-LOGIC is valuable because it keeps responsibility structurally readable while resisting the conversion of consequence-attribution into prescriptive command. It helps name the instability where "this consequence is attributable" begins to sound like "therefore one ought."

ANTICIPATION gives PMS a further drift case. Anticipation can drift into prediction, control, intelligence ranking, strategy, or premature action advantage. PMS detects this drift by asking whether Λ remains open, whether Θ is carried without closure, whether X is preserved, and whether the anticipatory stance binds the anticipator rather than authorizing coercive control over others.

This gives PMS a serious structural detection advantage.

It can show not only that a praxis structure has changed, but how it has changed. It can say: here, Φ (recontextualization) is no longer bridging toward Σ (integration); here, Λ (structured absence / non-event) has become load-bearing; here, X (distance) has become priced; here, Ω (asymmetry) has been moralized; here, Θ (temporality) has turned a reversible disagreement into a trajectory constraint.

But drift detection has its own pressure point.

Drift detection may become drift assignment.

PMS may label difference as drift too quickly. A rival framing, an unassimilated objection, or a non-PMS distinction may be read as mutation inside PMS rather than as a possible limit of PMS' own vocabulary.

The falsifier is therefore:

> **If PMS drift categories do not distinguish repeatable structural mutations from mere interpretive disagreement, drift detection weakens.**

PMS detects drift best where it can show a repeatable mutation in praxis structure, not merely a disagreement with PMS vocabulary.

That distinction must remain active. Otherwise, drift detection becomes a form of coverage.

2.3 Misuse Resistance

A third strength of PMS is misuse awareness.

PMS is structurally alert to several predictable abuses: person-ranking, tribunal drift, overreach, moralized diagnosis, public exposure effects, loss of reversibility, and the conversion of structural language into authority. This matters because any grammar that makes praxis more legible can also make persons, groups, or roles more vulnerable to being fixed by the analysis.

PMS does not avoid this risk by pretending that analysis is harmless.

Its better move is to make misuse structurally visible.

CRITIQUE does this by refusing to treat critique as a moral attitude, personality trait, or truth-claim

by itself. It binds critique to scenes, roles, frames, distance, reversibility, and non-personal description. This does not make critique safe in every context. It makes the conditions of admissible critique more explicit.

CONFLICT does this under higher risk. It repeatedly blocks the conversion of conflict analysis into mediation, diagnosis, person-typing, selection, enforcement, or public pillory. It treats conflict language as dangerous under publicness because structural descriptions can become sanction narratives when exposure rises.

LOGIC contributes misuse resistance by refusing to turn responsibility into moral code. It distinguishes consequence-attribution from prescription, and structural readability from normative command. This blocks a common misuse: using responsibility language as if PMS had generated an "ought."

ANTICIPATION contributes misuse resistance by blocking foresight fetish, retroactive guilt, universalized responsibility, and anticipation-as-control. It treats anticipatory responsibility as structurally dependent on effective capacity within a relevant frame and temporal window, not on abstract imagination of a possible future.

This is a real application strength.

PMS is not indifferent to the ways its own vocabulary can be misused. It often builds guardrails into the very domain layer where misuse is most likely.

But misuse resistance is not validation.

A grammar may include strong guardrails and still over-classify. It may prevent person-ranking and still lose discriminative force. It may preserve reversibility and still make resistant material too legible too quickly. It may block tribunal drift and still become stabilizing rather than critical.

The red line is therefore clear:

> **PMS includes misuse resistance, but guardrails are not validation.**

Misuse resistance is a strength of PMS application, but it does not settle whether PMS preserves discrimination under load.

2.4 Cross-Domain Portability

PMS' fourth strength is portability.

The same base grammar can travel across critique, conflict, logic, anticipation, sexuality, and formal-technical bridge contexts without immediately collapsing into one domain's vocabulary. This portability is not accidental. It follows from PMS' operatoric structure: Δ–Ψ can be applied to different praxis fields while preserving a shared grammar of Δ (difference), □ (frame), Λ (structured absence / non-event), A (attractor), Ω (asymmetry), Θ (temporality), Φ (recontextualization), X (distance), Σ (integration), and Ψ (self-binding).

This is one of PMS' central achievements.

It is also one of its central risks.

Portability is powerful because it allows PMS to compare unlike domains without reducing them to psychology, morality, or local jargon. But portability becomes dangerous when shared operator vocabulary begins to look like equivalence across domains.

The correct rule remains:

> **Portability is not equivalence.**

2.4.1 CRITIQUE / CONFLICT / LOGIC as Core Portability Cases

CRITIQUE, CONFLICT, and LOGIC show three core portability cases.

CRITIQUE shows PMS as method discipline. It uses the operator grammar to model when irritation becomes critique, when critique becomes correction, and when critique mutates into drift. Its strength lies in showing that critique is not merely content, tone, or opposition. It is a structural configuration with conditions of interruptibility, reframing, integration, and follow-up.

CONFLICT shows PMS under high-load non-integrability. It demonstrates that the same grammar can move beyond correction viability into stabilized incompatibility, cost topology, endpoint pressure, partial binding, tragedy, and terminality. CONFLICT is not merely "stronger critique." It shows that PMS can distinguish adjacent regimes by asking when integration no longer converges and when time, asymmetry, binding, and costly distance stabilize incompatibility itself.

LOGIC shows PMS at the boundary of attribution, responsibility, and non-normative readability. It tests whether PMS can reconstruct responsibility without deriving an ought. This is a crucial portability case because it brings PMS near moral language while requiring the grammar not to become moral theory. PMS can make consequence-attribution visible without claiming normative command.

Together, these three cases show that PMS can travel across method, conflict, and logical boundary management while preserving operatoric continuity.

But the continuity is not identity.

CRITIQUE, CONFLICT, and LOGIC do not prove PMS Base. They show that the base grammar can be applied in structurally different domains without immediate collapse. That is a portability claim, not a validation claim.

2.4.2 ANTICIPATION as Upstream Λ/Θ/X Portability Case

ANTICIPATION must appear already here because it tests PMS in an upstream position.

It is not a late-stage, high-conflict, or downstream governance case. It concerns praxis before events crystallize, before outcomes become settled, and before responsibility can be assigned by hindsight. This makes it an important portability test for PMS because it asks whether the operator grammar can handle future-oriented praxis without converting possibility into prediction or control.

The required sentence is:

> **ANTICIPATION shows PMS' upstream portability: Λ (structured absence / non-event), Θ (temporality), and X (distance) can be used to read future-oriented praxis without**

> **collapsing anticipation into prediction, control, or strategy.**

In shorter form:

> **ANTICIPATION functions as the upstream Λ/Θ/X portability case.**

This is a significant strength.

PMS can read anticipation not as a claim about future truth, but as a present structural stance. A bounded difference is framed as relevant, carried as non-event, placed under temporality, and restrained through distance. The anticipator is not authorized to close the future. The anticipator is bound to preserve openness under irreversibility.

This makes PMS portable into future-oriented praxis without becoming prognostic.

The pressure point remains: this portability can still overreach if every future-oriented stance is translated into PMS' preferred restraint grammar. ANTICIPATION itself keeps this risk visible by leaving rival framings and genuine non-capture open.

2.4.3 SEX and QC as High-Risk / Bridge Portability Cases

SEX and QC should appear here only as pointers.

SEX is a high-risk domain pointer because its domain conditions — intimacy, exposure, binding, asymmetry, and public legibility — can intensify PMS' misuse risks. A PMS reading in such a domain must be especially alert to high Ω (asymmetry), exposure, outer legibility, publicness overlay, and no person-ranking. The significance of SEX for Chapter 2 is therefore not that PMS has "solved" sexuality as a domain. It is that PMS portability reaches into a domain where the cost of misuse is high.

QC is a bridge pointer, not a domain proof. It indicates that PMS can be used for structural mapping beyond ordinary praxis domains, but only as bridge stress. QC may show portability of structural language into formal or technical contexts. It must not be treated as proof of substrate universality, physical finality, or PMS Base validation.

These two pointers sharpen the same rule from opposite directions.

SEX shows that portability becomes dangerous where exposure and person-near interpretation are high.

QC shows that portability becomes dangerous where bridge mapping may be mistaken for proof.

Both confirm the same Under Load premise:

> **Portability is one of PMS' central strengths, but in Under Load it also becomes one of PMS' central risks.**

The falsifier is direct:

> **If PMS loses operator clarity when transferred across domains, its portability claim weakens.**

Within the current source corpus, PMS does show wide portability. That portability is real enough

to be tested.

It is not evidence of equivalence.

2.5 Baseline Result

The baseline result is now established.

PMS is not weak in the obvious sense. It makes praxis structurally legible. It detects repeatable drift forms. It contains serious misuse-awareness. It travels across critique, conflict, logic, anticipation, high-risk intimacy domains, and bridge contexts without immediately dissolving into local vocabularies.

These are not trivial achievements.

A grammar that can distinguish critique from reaction, conflict from escalation, responsibility from ought, anticipation from prediction, and portability from equivalence has real structural force. PMS is not merely a vocabulary. It functions as a disciplined operator grammar capable of producing organized readings across different fields of practice.

But this chapter does not conclude that PMS is validated.

It concludes that PMS is strong enough to become dangerous in a specific way.

PMS is strong enough that weak criticism misses the point.

A weak critique would ask only whether PMS can produce coherent readings. It can. A weak critique would ask only whether PMS has guardrails. It does. A weak critique would ask only whether PMS travels across domains. It does. But none of these strengths settles whether PMS remains discriminating when those strengths become stabilizing.

The same features that make PMS useful also make it vulnerable to Under Load pressure. Legibility may become overconfidence. Drift detection may become drift assignment. Misuse resistance may become reassurance. Portability may become equivalence. Recontextualization may become compatibility production. Coverage may begin to stand in for discrimination.

PMS has not answered the critique because it has shown strength.

Its strength is the reason the critique must intensify.

The next section turns from what PMS does well to where it may work too well.

2.6 Where PMS Works Too Well

PMS may lose distinction not because it cannot classify, but because it can classify too much too smoothly.

This is the central instability that follows from the baseline.

The problem is not that PMS fails to render practice legible. The problem is that its legibility can become too fluent. A resistant scene may be translated into PMS terms before the resistance has been allowed to test PMS. A rival framing may be treated as drift before it has been allowed to remain rival. A non-fitting case may be recontextualized before non-fit has been preserved as a possible limit.

This is overfunction.

Overfunction does not mean that PMS breaks down. It means that PMS continues to work in a way that may reduce the friction needed for discrimination. The grammar remains coherent. The operators remain available. The reading remains plausible. But plausibility itself becomes part of the risk.

Under Load therefore asks not only where PMS fails.

It asks where PMS works too well.

2.6.1 High Coherence Effect

PMS outputs can become persuasive because the grammar stabilizes scenes with high internal coherence.

This is a real strength. PMS can take a diffuse practical situation and arrange it through Δ (difference), □ (frame), Λ (structured absence / non-event), Ω (asymmetry), Θ (temporality), Φ (recontextualization), X (distance), Σ (integration), and Ψ (self-binding) into a readable structural sequence. That sequence can clarify what ordinary description leaves tangled: what is absent, what is framed, where asymmetry lies, how time changes cost, where distance remains possible, and whether integration is still viable.

But high coherence may conceal weak discrimination.

A PMS reading can be internally well-formed while still failing to preserve enough difference from the scene it reads. The danger is not incoherence. The danger is that coherence becomes persuasive before comparison has been sufficiently tested.

A coherent operator sequence may make a scene feel structurally settled. It may reduce ambiguity. It may organize tensions. It may assign pressure points. Yet the very smoothness of that organization can obscure whether the reading has actually distinguished the decisive elements or merely stabilized them in PMS-compatible form.

The high coherence effect becomes unstable when the grammar's internal fit is mistaken for discriminative adequacy.

This does not mean coherence is bad. PMS needs coherence to function. Without coherence, the grammar would not improve legibility at all.

The pressure point is narrower:

> High coherence may conceal weak discrimination.

The falsifier is equally narrow:

> **If PMS coherence consistently increases sensitivity to difference rather than reducing it, the high coherence effect does not become an instability.**

This condition matters. PMS coherence is not automatically a problem. It becomes a problem only where coherent rendering makes difference less available, less disruptive, or less capable of challenging the grammar.

2.6.2 Drift Absorption

PMS is strong at naming drift.

It can identify when critique becomes performance, when anticipation becomes control, when conflict becomes moralized narrative, when logic becomes closure, when bridge mapping becomes proof, or when governance begins to sound like validation.

This is one of PMS' most useful capacities.

But the same capacity can become absorptive.

Drift absorption begins where PMS can name a deviation before it has tested whether the deviation should remain external to its grammar.

This is the critical risk. PMS may classify many tensions, deviations, and oppositions as drift forms. It may read resistance as frame drift, Φ drift, Λ residue, integration failure, publicness distortion, stack confusion, or downstream misuse. Each of these may be correct in particular cases. But if the classification happens too quickly, the model may convert challenge into internal variation before the challenge has had enough force.

Difference then becomes drift.

Opposition becomes miscalibration.

Non-fit becomes residual Λ (structured absence / non-event).

Rival framing becomes a useful external contrast that PMS can already locate.

The danger is not that PMS invents drift where nothing is happening. The danger is subtler: real deviations may be named accurately as drift and still be neutralized too early as internal phenomena.

This is why drift detection must remain falsifiable. PMS must show that a drift category identifies a repeatable structural mutation, not merely a refusal to speak PMS fluently.

The red line is simple:

> Do not treat every resistance as drift.

A resistance may be drift. It may also be rival grammar, genuine non-capture, external objection, empirical mismatch, or domain-specific structure that PMS does not yet preserve.

The test is whether PMS can delay classification without losing its analytic nerve. If the grammar can hold a deviation as unresolved, compare it against rival descriptions, and still preserve the possibility that the deviation remains outside PMS, then drift detection remains disciplined. If PMS must immediately convert the deviation into a known drift type in order to keep the scene legible, drift detection has begun to absorb critique.

Drift absorption begins when PMS no longer gives difference enough time to remain undecided.

2.6.3 Φ-Overfunction

Φ (recontextualization) is indispensable to PMS.

Without recontextualization, PMS would become rigid. It could not track transformation, changes in interpretation, learning, domain transfer, or changing frame conditions. Φ allows a scene to be moved from one frame into another without treating the first frame as final.

But Φ can become too successful.

Φ-overfunction occurs where recontextualization makes non-fit compatible instead of preserving Δ (difference).

This is not a failure of Φ in the ordinary sense. It is not that Φ stops working. It is that Φ works so well that resistance loses shape. A scene that does not fit may be re-embedded into another context. A contradiction may become a developmental tension. A rival objection may become an external frame. A refusal may become a non-event awaiting integration. A breakdown may become a transformation point.

Again, these readings may be correct.

The risk is not recontextualization as such.

The risk is smoothing.

Φ becomes risky not because recontextualization fails, but because it can succeed without preserving enough resistance.

This is why Φ must remain bound to Δ. Recontextualization should not erase the difference that made recontextualization necessary. If Φ moves a resistant object into a new frame, the object must still be allowed to resist inside the new frame. Otherwise, Φ does not preserve difference; it converts difference into compatibility.

The operatoric test is simple: after recontextualization, is the original Δ still visible as a constraint, or has it been smoothed into a PMS-readable variation? If Δ remains load-bearing, Φ has preserved resistance. If Δ disappears into a more elegant reading, Φ has begun to overfunction.

This pressure will matter later in the paper when Φ drift is treated directly. Here it functions as a baseline instability: one of the reasons PMS' strength must become suspect precisely where the grammar remains fluent.

The red line is equally important:

> Do not imply Φ is bad.

Φ is not bad. Φ is one of PMS' major strengths. But where Φ becomes overfunctional, PMS may preserve intelligibility at the cost of difference.

2.6.4 LOGIC Pointer: Attribution Flattening

LOGIC points to a later risk that should be named here without becoming the meta-normativity chapter.

PMS-LOGIC helps preserve a crucial distinction: responsibility can be structurally readable without becoming an ought. Consequence, attribution, capacity, asymmetry, and self-binding can be made legible without converting PMS into moral prescription.

That is a strength.

But it also exposes a risk.

The LOGIC case shows that responsibility without ought must remain structurally precise, or attribution itself begins to flatten.

Attribution flattening occurs when different structural relations begin to sound the same: consequence, responsibility, capacity, exposure, participation, fault, and obligation. PMS may intend to distinguish them, but if the language becomes too smooth, responsibility may begin to carry normative pressure even where the model has not authorized an ought.

This is only a pointer.

The full issue belongs to the later chapter on meta-normativity. Chapter 2 only marks the baseline risk: PMS can produce non-moral readability near moral terrain, but the closer it gets to responsibility language, the more carefully it must preserve distinctions among attribution, capacity, consequence, and prescription.

If those distinctions flatten, PMS may remain formally non-prescriptive while sounding normatively loaded in practice.

That is not yet a refutation.

It is a pressure point.

2.7 Conclusion

PMS' strengths are not denied.

They are the reason PMS Under Load is necessary.

PMS produces structured legibility. It identifies drift. It contains misuse-awareness. It travels across domains. It can read critique without reducing it to attitude, conflict without reducing it to escalation, logic without turning attribution into moral command, and anticipation without collapsing the future into prediction or control.

These strengths matter.

But they also create the condition for the critique that follows.

A grammar that is productive, coherent, portable, and misuse-aware may fail differently from a weak or incoherent framework. It may fail not by obvious breakdown, but by stabilizing too much. It may classify too smoothly. It may absorb drift too quickly. It may recontextualize non-fit too effectively. It may become persuasive because its own grammar produces high internal coherence.

This chapter therefore does not end in reassurance.

It ends by converting baseline strength into pressure.

> **The problem is not that PMS lacks power; the problem is that power, once portable and stabilizing, must itself become an object of critique.**

Chapter 3 does not inherit a secured PMS from this baseline. It begins the instability sequence with

calibration: the point where PMS remains formally disciplined while its operational thresholds are not fully endogenous. The baseline does not secure PMS; it only makes the first instability worth testing.

3. Instability I — Calibration

Chapter Aim

This chapter establishes calibration as the first hard instability of PMS Under Load.

The issue is not vague interpretation.

The issue is more precise:

> **PMS remains formally disciplined, but its operational thresholds are not fully endogenous.**

"Not fully endogenous" means that PMS can define the operators and constrain their use, but it cannot always generate from within the grammar alone the point at which an operator becomes sufficiently operative, costly, blocked, binding, or decisive.

This makes calibration basal. Every later instability depends on it. Stack drift, Φ drift, meta-normativity, success trap, domain drift, publicness, and self-application all presuppose that PMS can decide when an operator is sufficiently present, sufficiently costly, sufficiently blocked, sufficiently binding, or sufficiently decisive.

But PMS does not always generate those thresholds from within its operator grammar.

That does not make PMS arbitrary. It does not mean that analysts simply "interpret differently." It means that PMS can define the relevant distinctions with formal discipline while still requiring threshold judgments at the point of application.

The chapter therefore tests a narrow claim:

> **Calibration is not a failure of formal discipline; it is the point where formal discipline meets threshold judgment.**

3.1 Problem

PMS has formal operators.

It has an ordered grammar. It has dependencies. It has guardrails. It distinguishes Δ (difference), ∇ (impulse / directed activation), □ (frame), Λ (structured absence / non-event), A (attractor), Ω (asymmetry), Θ (temporality), Φ (recontextualization), X (distance), Σ (integration), and Ψ (self-binding). It can derive structured readings across critique, conflict, logic, anticipation, sexuality, publicness, and self-binding.

But formal operator identity does not by itself determine every operational threshold.

The question is not only:

> Which operator is present?

It is also:

> When is this operator sufficiently operative to change the reading?

This distinction matters.

A scene may contain X (distance), but the relevant question is whether X is strong enough to sustain reflective interruption rather than delayed reaction, avoidance, or formal restraint without practical stop-capability.

A scene may contain Λ (structured absence / non-event), but the relevant question is whether Λ remains open, tolerable, and structurally carried, or whether it has become evasion, withholding, avoidance, or cost-shifting.

A scene may contain Ω (asymmetry), but the relevant question is whether Ω is merely analytically visible or already decisive enough to reorganize responsibility, exposure, cost, or viability.

A scene may contain Ψ (self-binding), but the relevant question is whether Ψ stabilizes accountable continuity or hardens into rigidity, overbinding, leakage, or covert attribution demand.

A scene may contain a viable corridor, but the relevant question is whether that corridor is actually viable under timing, exposure, stakes, publicness, embodiment, and cost distribution.

None of these questions is solved by naming the operator.

This is the first instability.

PMS may appear more determinate than its application actually is because its formal vocabulary is highly structured. The grammar can make a case look settled even where threshold determination remains open.

This is not a defect at the level of operator definition.

It is a pressure point at the level of use.

Calibration is the point at which PMS must move from formal distinction to applied discrimination. At that point, the model remains disciplined, but it is no longer fully self-sufficient. Its constraints narrow the field of PMS-conform readings; they do not always compel one threshold.

3.2 Evidence

Calibration pressure appears across PMS Base and selected domain layers.

The pattern is consistent: PMS defines the relevant structural distinctions, but the moment at which those distinctions become operationally decisive often requires judgment that the operator grammar does not fully generate by itself.

3.2.1 PMS Base

PMS Base already contains the calibration problem in its own limitation structure.

The base grammar is generative and disciplined, but it does not claim that high derivational reach proves completeness, sufficiency, or discriminative adequacy. Its portability depends on scene selection, interpretive granularity, architectural constraints, domain-sensitive calibration, and methodological discipline.

This is the correct base posture.

PMS does not say:

> Every operator application is fully determined by the grammar alone.

It says, more carefully:

> PMS provides a disciplined operator grammar whose application remains bounded, calibratable, and open to non-capture.

That means the calibration problem is not an external attack on PMS. It is already implied by PMS' own responsible boundedness.

The base grammar provides operator identities and dependency constraints. But in concrete application, it still leaves open threshold questions:

```
When is X sufficient?
When is Λ active rather than merely absent?
When is Ω decisive rather than background?
When has Θ made reversal expensive?
When does Φ clarify rather than absorb?
When has Σ genuinely integrated rather than stabilized?
When does Ψ bind rather than harden?
```

These are not marginal questions.

They determine whether PMS remains discriminating in use.

The base-level diagnosis is therefore:

> PMS defines what must be distinguished; it does not always fully determine where the distinction becomes operationally binding.

3.2.2 LOGIC

LOGIC makes calibration visible around effective capacity and attributable consequence.

PMS-LOGIC preserves a crucial distinction: responsibility is not ought. Responsibility concerns structural attributability of consequences; ought concerns prescription. The LOGIC layer therefore helps PMS avoid deriving normativity from attribution.

But the threshold problem remains.

LOGIC can define effective capacity as frame-bound capacity under time. It can show that non-action becomes responsibility-relevant where capacity exists under asymmetry and temporality. It can distinguish attributable consequence from moral obligation.

Yet application still depends on threshold judgments:

```
How much capacity counts as effective?
How open must the temporal window be?
How much access is enough to make non-action attributable?
When does constraint reduce attributability?
When does asymmetry make non-action no longer neutral?
```

```
When does responsibility become structurally readable without becoming culpability?
```

LOGIC keeps the ought-gap open. That is a strength.

But keeping the ought-gap open does not settle every threshold of attribution.

Two PMS-conform analyses may agree that a scene contains Ω (asymmetry), Θ (temporality), Λ (structured absence / non-event), and potential effective capacity. They may agree that no ought is being derived. They may still diverge on whether capacity was sufficient for attribution.

One analyst may conclude:

> Effective capacity was present enough that non-action is structurally attributable.

Another may conclude:

> Capacity was too constrained, too late, or too weakly positioned to ground attribution.

Both may remain PMS-conform.

The instability is therefore not logical incoherence.

It is threshold plurality inside a disciplined responsibility grammar.

The red line is clear:

> Do not convert attribution into ought.

But the calibration problem remains:

> When does non-action become attributable without becoming automatically culpable?

LOGIC shows the strength of PMS: responsibility can become structurally readable without moral prescription.

It also shows the limit: the threshold of effective capacity is calibration-sensitive.

3.2.3 CRITIQUE

CRITIQUE makes calibration visible around interruptibility, reachability, and drift thresholds.

A critique scene is not merely a person being critical. It becomes critique when difference is framed, distance becomes possible, interruption remains structurally available, and the chain can move toward recontextualization and possibly integration.

This gives PMS a disciplined way to distinguish irritation, reaction, judgment, commentary, critique, correction, and drift.

But CRITIQUE also depends on threshold judgments.

The relevant questions are not only:

```
Is X present?
Is Φ available?
Is Σ possible?
```

```
Is drift detectable?
```

They are:

```
When is X sufficient to make critique interruptible?
When is a configuration reachable for correction?
When has irritation become critique rather than delayed reaction?
When has commentary become drift?
When is Φ still recontextualization rather than narrative substitution?
When is Σ genuinely reachable rather than rhetorically invoked?
```

These questions are not arbitrary. PMS constrains them strongly.

But it does not always settle them fully.

A critique sequence may be read as viable under one calibration: distance is sufficient, reachability remains open, and recontextualization can still become corrective.

The same sequence may be read as drift under another calibration: distance is too weak, the chain has already fused with reaction, or Φ has become narrative repair without integration.

Both readings may use the same operators.

Both may respect PMS guardrails.

The divergence appears at the threshold level.

CRITIQUE therefore demonstrates a crucial pattern:

> The structure of critique may be stable while the viability corridor remains calibrated.

This matters because PMS Under Load depends on critique being able to endanger PMS. If critique thresholds are too easily calibrated in PMS' favor, internal critique becomes re-audit. If thresholds are calibrated too harshly, PMS may lose valid critique too early.

The instability lies between those risks.

3.2.4 CONFLICT

CONFLICT makes calibration visible around publicness, exposure, terminality pressure, and cost geometry.

Conflict is not merely intense disagreement. In PMS-CONFLICT, conflict becomes structurally legible where integration no longer carries, self-binding persists, asymmetry distributes cost, temporality hardens paths, and distance becomes priced, delegitimized, or asymmetrically available.

This is a powerful grammar.

It allows PMS to distinguish disagreement, critique, competition, conflict, tragedy, terminality, and downstream handoff. It also blocks moralized readings of conflict as mere failure, immaturity, or bad faith.

But calibration pressure is intense here.

The relevant questions are:

```
When is non-integrability structurally stabilized rather than prematurely declared?
When is X merely costly, and when is it practically unavailable?
When does publicness change the regime rather than merely increase exposure?
When does exposure become structurally decisive?
When has Σ failed rather than become temporarily blocked?
When does terminality name a grammar-internal limit rather than a rhetorical stopping pc
```

CONFLICT makes these distinctions legible. It does not make all thresholds automatic.

This is especially important because conflict analysis is high-risk. If non-integrability is declared too early, PMS may convert a difficult but still viable contradiction into terminal conflict. If non-integrability is declared too late, PMS may keep seeking integration where integration has become structurally unavailable, thereby increasing cost.

Calibration is therefore not an interpretive inconvenience.

It is part of the cost geometry.

A PMS-conform analysis must decide whether the conflict remains within a correction or integration corridor, whether it has entered stabilized incompatibility, or whether it has reached a terminal regime in which the grammar continues to describe but no longer produces reintegration affordances.

Those are threshold decisions.

They depend on scene knowledge, exposure conditions, timing, role-position, reversibility, and the actual price of distance.

PMS formalizes the conflict grammar.

It does not fully endogenize every point at which conflict changes regime.

3.2.5 ANTICIPATION

ANTICIPATION makes calibration visible around responsible carrying of Λ, restraint, avoidance, control fantasy, and premature closure.

Anticipation is structurally delicate because it deals with what has not yet occurred. Its core strength lies in carrying a future-oriented non-event under temporality and distance without collapsing that non-event into prediction, control, or strategy.

The minimal anticipatory stance is disciplined:

```
A bounded possibility is framed as relevant.
Λ remains open.
0 makes the possibility trajectory-relevant.
X prevents premature closure.
```

This is a strong PMS portability case.

But it creates calibration problems immediately.

The relevant questions are:

```
When is Λ responsibly carried rather than inflated?
When does restraint become avoidance?
When does anticipation become prediction?
When does prediction become control fantasy?
When does X preserve openness, and when does it delay action past a viable Θ-window?
When does anticipatory responsibility arise from effective capacity, and when is respons
```

ANTICIPATION guards against foresight fetish, retroactive guilt, and universalized responsibility. That matters.

But the guardrails do not eliminate calibration dependence.

An analyst may read a scene as disciplined anticipation: the actor has marked a relevant possibility, preserved uncertainty, avoided closure, and carried restraint.

Another analyst may read the same scene as avoidance, control fantasy, or premature authority: the actor uses future-oriented legibility to justify inaction, dominance, or strategic closure.

The difference is not necessarily a disagreement about operators.

It may be a disagreement about thresholds:

```
How much restraint is enough?
How much delay is too much?
How open must Λ remain?
How narrow must the temporal window become before non-action changes status?
```

ANTICIPATION therefore shows calibration in an upstream form.

Before the event occurs, PMS must still decide whether future-oriented praxis remains open, disciplined, and non-coercive, or whether it has already drifted into control, avoidance, or prediction.

That decision cannot be fully generated by the mere presence of Λ, Θ, and X.

3.2.6 SEX

SEX makes calibration visible around outer legibility, internal framing, disturbance language, path-cost, exposure, and Dignity-in-Practice. Here D (Dignity-in-Practice) is not introduced as a new PMS base operator. It functions as an application-gate constraint: a check on whether the mode of analysis remains non-humiliating, scene-bound, reversible, and non-ranking.

This domain is especially important because it is high-risk. PMS-SEX must preserve sharp structural analysis without person-ranking, diagnosis, shame, or tribunal drift. It distinguishes internal framing, outer legibility, and public circulation. It treats path-cost, disturbance, instability, and

deviation strictly as configuration-language. It does not allow those terms to become identity taxonomy, pathology, moral rank, or person judgment.

That discipline is necessary.

It also makes calibration unavoidable.

The relevant questions are:

```
When may outer legibility be named without becoming shame?
When does internal framing remain real but not total?
When does publicness merely amplify consequence, and when does it transform the operatir
When does path-cost become high enough to mark drift?
When does disturbance-language remain structural, and when does it become humiliating or
When is Dignity-in-Practice preserved, and when has the mode of analysis itself become i
```

These thresholds are not solved by the operator names.

SEX can say that internal framing does not erase outer form. It can say that outer legibility does not authorize person-ranking. It can say that path-cost names stabilization burden, not human worth. These are strong guardrails.

But in application, the analyst must still calibrate:

```
How strong is outer legibility in this scene?
How much divergence exists between internal framing and enacted form?
How public is public enough to change the risk profile?
How much exposure makes reversibility practically fragile?
How much path-cost is structurally significant rather than merely visible?
```

This is where calibration becomes ethically and methodologically sensitive.

If PMS names outer legibility too weakly, it may allow internal framing to erase enacted structure.

If PMS names outer legibility too strongly, it may drift toward tribunal language, shame, or person-near fixation.

If PMS treats path-cost too lightly, it may miss stabilization burden.

If it treats path-cost too heavily, it may convert configuration-language into disguised ranking.

PMS-SEX therefore shows that calibration is not only technical.

It is also a dignity-sensitive application problem.

The model can preserve its guardrails and still face divergent, defensible threshold judgments about whether a scene remains viable, exposed, reversible, dignity-preserving, or drift-prone.

3.2.7 MIP/AH as Application-Gate Contrast

MIP/AH (Maturity in Practice / AH Precision: Attack Surface / Hardening) enters this chapter only as application-gate contrast.

It may discipline how calibration is applied. It may require role sensitivity, no person-ranking, reversibility, analysis-artifact responsibility, publicness caution, and misuse awareness. It may expose attack surfaces in the analysis artifact, such as scope drift, language drift, inference risk, and misuse potential. It may help prevent an analyst from turning threshold judgment into tribunal judgment.

But it does not solve calibration.

The governing sentence is:

> **MIP/AH can discipline the application of calibration, but they do not make PMS' operational thresholds fully endogenous.**

This is crucial.

MIP/AH can say:

```
Do not make this calibration public in a person-identifying way.
Do not convert threshold judgment into maturity ranking.
Do not treat D as scalable human worth or as MIP/AH-derived proof of PMS Base validity.
Keep the claim scene-bound, revisable, and non-humiliating.
```

But MIP/AH cannot decide, at the base level, exactly when X is sufficient, when Λ has become evasive, when Ω becomes decisive, when Ψ binds rather than leaks, or when a viable corridor has closed.

MIP/AH can govern calibration use.

It cannot generate PMS' thresholds from within PMS.

If MIP/AH were used to remove calibration dependence, it would become a rescue layer. That is inadmissible.

3.3 Diagnosis

Calibration shows that PMS' formal strength does not automatically yield operational closure.

This is the central diagnosis:

> **The instability lies not in operator absence, but in threshold plurality under operator discipline.**

PMS can remain formally correct.

The operators can be properly identified. The sequence can be reconstructed. The domain layer can be valid. Guardrails can be preserved. The analyst can avoid person-ranking, moralization, overclaim, and tribunal drift.

And still, two PMS-conform analyses may diverge.

This divergence does not necessarily mean one analysis is simply wrong.

It may mean that PMS has reached a threshold zone where formal discipline constrains the analysis

but does not uniquely determine it.

That is the hard point.

PMS is not weak because it lacks operators.

It is unstable because operator availability does not guarantee threshold convergence.

A PMS analysis can therefore be:

```
operatorically valid,
scene-bound,
reversible,
non-tribunal,
and still calibration-divergent.
```

This matters for every later chapter.

Calibration pressure prepares stack drift because layer claims may become unstable when thresholds are not typed clearly.

It prepares Φ drift because recontextualization depends on when non-fit is still preserved as Δ and when it has been smoothed.

It prepares meta-normativity because PMS' direction becomes active in how thresholds are drawn.

It prepares success trap because high coherence can hide threshold decisions inside fluent readings.

It prepares domain drift because different domains may require different calibration sensitivities.

It prepares publicness because exposure can change thresholds of reversibility, person-nearness, and misuse risk.

It prepares self-application because PMS cannot fully arbitrate its own threshold dependence without using the very calibration it is testing.

Calibration is therefore not a local problem.

It is the basal instability.

3.4 Calibration Is Not Fully Endogenous

Calibration is external not because PMS is arbitrary, but because praxis thresholds are not fully contained in formal operator syntax.

This sentence must be read carefully.

"External" does not mean outside PMS in the sense of random preference, subjective taste, or unbounded interpretation. It means that application requires conditions that PMS can name but not fully generate.

Some calibration conditions depend on:

```
scene knowledge,
```

```
context,
timing,
exposure,
publicness,
stakes,
embodiment,
role-position,
available alternatives,
institutional setting,
history of prior interactions,
practical stop-capability,
and interpretive judgment.
```

PMS can include these conditions in analysis. But including them is not the same as deriving their thresholds.

The grammar can tell the analyst to attend to X. It cannot always determine how much stop-capability is enough.

The grammar can tell the analyst to track Λ. It cannot always determine when non-event load becomes steering.

The grammar can tell the analyst to track Ω. It cannot always determine when asymmetry becomes decisive.

The grammar can tell the analyst to preserve D (Dignity-in-Practice) as an application-gate constraint. It cannot always determine, from syntax alone, when a sharp structural naming has crossed into humiliating or person-near form.

The grammar can tell the analyst to track Ψ. It cannot always determine when self-binding is stable, leaking, suspended, or covertly demanded.

This is not relativism.

PMS still constrains the analysis. It rules out many moves. It forbids person-ranking. It blocks tribunal drift. It distinguishes attribution from ought. It preserves non-capture. It keeps claims scene-bound. It demands that operators be used in disciplined relation.

But constraint is not closure.

Calibration is where PMS' direction becomes operational. Threshold decisions are not outside the grammar in the sense of being irrelevant to it. They are outside full endogenous determination in the sense that the grammar cannot always compute them from itself.

This creates a reduction PMS must accept:

> PMS can structure threshold judgment, but it cannot always replace it.

The consequence is methodological.

PMS must not present calibrated conclusions as if they were fully operator-derived inevitabilities. It

must mark where threshold judgment has entered. It must remain reversible where calibration is contestable. It must allow that another PMS-conform reading may draw the threshold differently.

This is how calibration remains disciplined without becoming relativistic.

3.5 Falsifier

The calibration objection has a direct falsifier.

> **If PMS can generate stable, non-arbitrary, internally derived thresholds across calibration-sensitive cases without outside calibration input, the calibration objection weakens.**

The required stress case is this:

> **Two analysts examine the same scene. Both remain PMS-conform. Both preserve X (distance), reversibility, Dignity-in-Practice, scene-binding, and non-person-ranking. Both use the same operator grammar. Yet they produce divergent but defensible calibrations of X, Λ, Ω, D (Dignity-in-Practice), Ψ, and viable corridor.**

Expanded:

Two analysts examine the same scene.

They agree on the relevant descriptive material. They do not diverge because one has more facts, because one ignores a guardrail, or because one uses a different operator set. They agree that the scene contains framed difference, structured absence, asymmetry, temporality, possible distance, and some form of self-binding or binding pressure. They agree that no person ranking is allowed. They agree that claims must remain reversible and scene-bound.

One analyst concludes:

```
X is sufficient.
Λ remains open rather than evasive.
Ω is present but not yet decisive.
D is preserved by the mode of analysis.
Ψ stabilizes continuity rather than leaking.
The viable corridor remains open.
```

The other analyst concludes:

```
X is too weak or too costly to function.
Λ has become steering or avoidance.
Ω is already decisive.
D is at risk because outer legibility naming is too person-near.
Ψ has begun to leak or harden.
The viable corridor has narrowed or closed.
```

If both readings remain PMS-conform, the calibration dependency holds.

The model has not failed.

But its thresholds have not been fully endogenized.

The inverse falsifier is equally important:

> **If PMS can internally compel convergence between such analysts without importing scene-external threshold judgment, then the calibration objection weakens.**

That would mean PMS does not merely constrain threshold judgment. It generates threshold convergence.

Chapter 3 does not assume that this is impossible in every case. PMS may generate strong convergence in many scenes. Some thresholds may be clear. Some domain layers may specify tighter gates. Some operator constellations may leave little room for divergence.

But the objection survives if calibration-sensitive cases remain where disciplined analysts can diverge without misuse.

That is the instability under load.

3.6 Conclusion

Calibration does not defeat PMS.

It reduces PMS.

That distinction matters.

Calibration is not an argument that PMS lacks formal discipline. The opposite is true: calibration becomes visible precisely because PMS is disciplined enough to show where threshold judgments occur.

A vague framework would not expose this instability clearly. PMS does.

But exposure is not resolution.

PMS survives calibration only by giving up the stronger claim that its operators fully determine their own thresholds of application.

This is the correct reduction:

```
PMS may claim:
It provides a disciplined operator grammar that structures calibration-sensitive praxis

PMS may not claim:
Its operators fully generate all thresholds required for concrete application.
```

The reduction is not small.

It affects how every strong PMS claim must be read. A PMS reading may be coherent, portable, PMS-conform, and disciplined while still depending on calibrated judgments that remain contestable. A PMS application may be strong without being fully self-grounding. A PMS threshold

may be well-argued without being uniquely compelled by the grammar.

The closing sentence is therefore:

> **PMS survives calibration only by giving up the stronger claim that its operators fully determine their own thresholds of application.**

Chapter 4 does not inherit a solved calibration problem. It shifts the pressure from operational thresholds to stack instability: even if PMS is internally coherent, its layers may become externally misread.

4. Instability II — Stack / Interpretation Drift

Chapter Aim

This chapter introduces the second instability of *PMS Under Load*.

Chapter 3 did not solve calibration. It left threshold dependence in force. Stack / Interpretation Drift now tests a different pressure point:

> **PMS can remain internally coherent while its layer boundaries become externally unstable.**

This instability does not begin with operator failure.

It begins with successful portability.

PMS travels across base grammar, domain layers, add-ons, governance hardening, bridge mappings, implementation horizons, and now documented implementation artifacts. That travel is one of PMS' strengths. But the more portable PMS becomes, the easier it becomes to misread movement across layers as equivalence of authority.

A domain application may begin to sound like a base definition.

A reduced signature may begin to sound like an operator.

A governance gate may begin to sound like a tribunal.

A bridge mapping may begin to sound like proof.

An implementation architecture may begin to sound like validation.

A public repository may begin to sound like PMS Base evidence.

PMS-RUST v0.1 sharpens this instability. It documents a bounded PMS-STACK Evidence Spine: a runtime/toolchain artifact that makes parts of PMS-STACK executable, inspectable, and testable under v0.1 constraints. This strengthens the implementation/artifact claim. It does not dissolve stack drift. It makes stack drift more inspectable.

Stack Drift names this instability.

It tests whether PMS can preserve its operator grammar, layer boundaries, and claim types under the very portability and implementability that make the model powerful.

The key sentence is:

> **High portability increases the risk that layer boundaries are mistaken for operator equivalence.**

With PMS-RUST added, the sharper version is:

> **Executable artifact status increases the risk that implementation evidence is mistaken for PMS Base validation.**

4.1 Problem

PMS may preserve internal coherence while its external uptake misreads layers, statuses, and claim types.

“External uptake” does not mean accidental reader error only. It names the way PMS-derived outputs circulate across papers, add-ons, governance artifacts, bridge mappings, implementation proposals, public explanations, and repository artifacts. Stack Drift begins where that circulation changes how a claim is heard without changing the PMS vocabulary that carries it.

This is a distinct instability.

It is not the same as calibration. Calibration concerns thresholds of application: when X (distance) is sufficient, when Λ (structured absence / non-event) is active, when Ω (asymmetry) becomes decisive, when Ψ (self-binding) binds, when a viable corridor closes.

Stack Drift concerns status confusion: what kind of claim is being made, at which layer, with what authority, and under which constraints.

A PMS-derived output may be structurally coherent and still be misread.

For example:

```
A domain layer may be read as if it redefined PMS Base.
An add-on may be read as if it introduced a new primitive.
MIP/AH may be read as if it judged persons.
QC may be read as if structural mapping were physical explanation.
STACK may be read as if implementation pressure validated PMS.
PMS-RUST may be read as if repository evidence validated PMS Base.
Tests may be read as proof.
Runtime dialect may be read as PMS 1.3 theory.
Model validation may be read as praxis adequacy.
Execution traces may be read as external validation.
```

In each case, the problem is not that PMS has become incoherent internally. The problem is that its portability creates interpretive pressure at the stack boundary.

PMS-RUST makes this pressure more concrete. It does not create the stack problem, but it gives the problem a public artifact surface: crates, runtime behavior, tests, traces, demos, documentation, AI proposal boundaries, and repository structure. These make PMS more inspectable as an implementation project. They also make it easier to mistake artifact discipline for base validation.

The more PMS travels, the more readers must track:

```
What layer is speaking?
What claim type is active?
What is source of truth?
What is application only?
What is governance only?
```

```
What is bridge only?
What is implementation only?
What is repository evidence?
What is not validation?
```

If these questions are not kept active, PMS can become stronger in appearance while weaker in discipline.

A coherent stack can still drift in interpretation.

An executable artifact can still remain only an implementation artifact.

That is the instability.

4.2 Source-of-Truth Discipline

The first control is source-of-truth discipline.

PMS Base defines:

```
operators,
dependencies,
derived axes,
base guardrails,
claim boundaries.
```

This means that Δ (difference), ∇ (impulse / directed activation), □ (frame), Λ (structured absence / non-event), A (attractor), Ω (asymmetry), Θ (temporality), Φ (recontextualization), X (distance), Σ (integration), and Ψ (self-binding) do not receive their canonical meaning from domain layers, add-ons, governance layers, bridge mappings, implementation projects, or repository artifacts.

Domain layers may apply the grammar.

They may stress it.

They may reveal where a given operator becomes difficult to use.

They may introduce reduced signatures, domain concepts, drift markers, gates, publicness overlays, or application profiles.

But they may not redefine the grammar.

The rule is:

> **Domain layers stress the grammar; they do not become the grammar.**

The same rule applies to implementation artifacts.

A repository may implement a runtime dialect, model loader, validation path, REPL, script runner, trace format, test suite, or AI proposal bridge. It may make some PMS-derived structures executable under bounded constraints. But it does not become PMS Base.

PMS-RUST may document executable artifact-backed partial realization of a PMS-STACK Evidence

Spine v0.1. It may not redefine Δ–Ψ, validate PMS Base, or complete PMS 1.3 semantics by existing as a repository.

This matters because PMS' domain and implementation success are persuasive. Once a domain layer becomes useful, its local vocabulary can begin to feel basic. Once a repository becomes executable, its runtime vocabulary can begin to feel authoritative. A reader may start treating a domain-specific concept, runtime alias, crate boundary, or test fixture as if it had the same status as Δ–Ψ.

That is the first form of Stack Drift.

The red line is simple:

> Do not let domain terms, runtime dialects, modules, or repository artifacts become base operators.

Outer legibility may be important in SEX. It is not a new PMS operator.

Reach may be important in CRITIQUE. It is not a new PMS operator.

Φ-substitution may be important in CONFLICT. It is not a redefinition of Φ.

QFRAME may be useful in QC. It is not a replacement for □.

MIP/AH may support governance, application discipline, and AH Precision attack-surface hardening. It is not a base authority.

STACK may render PMS architecturally projectable.

PMS-RUST may make parts of a PMS-STACK Evidence Spine executable, inspectable, and testable.

Neither STACK nor PMS-RUST proves that PMS is true.

Source-of-truth discipline therefore protects PMS from one of its own strengths: the tendency of successful applications and implementation artifacts to look foundational.

4.3 Operator Meaning vs Domain Application

Stack Drift becomes visible when operator meaning and domain application begin to blur.

A domain paper must translate base operators into local analytical pressure. That translation is necessary. Without it, domain work would remain abstract. But translation becomes unstable when local usefulness is mistaken for operator authority.

The key sentence is:

> **Application pressure becomes drift when local usefulness is mistaken for operator authority.**

ANTICIPATION is a clear case.

It uses Λ, Θ, and X to read future-oriented praxis before the event occurs. That is legitimate as a domain application. But ANTICIPATION must not reinterpret Λ into prediction, Θ into forecasting horizon, or X into strategic delay. Anticipation remains PMS-conform only if future-oriented praxis is not collapsed into prognosis, control, or strategy.

CRITIQUE gives another case.

CRITIQUE uses X as the threshold of interruptibility. It may say that critique requires distance, stop-capability, and meta-position under pressure. But CRITIQUE must not redefine X as a critique-specific operator. It applies X to the critique scene; it does not create a new X.

CONFLICT adds a more delicate case.

CONFLICT may use Φ-substitution as a marker: recontextualization can replace integration where conflict stabilizes. This is analytically powerful. But CONFLICT must not redefine Φ as substitution. Φ remains recontextualization in PMS Base. Φ-substitution is a conflict-layer failure mode or marker, not a new canonical meaning of Φ.

SEX adds a high-risk case.

SEX may introduce outer legibility, internal framing, publicness overlay, path-cost, disturbance language, and Dignity-in-Practice application gates. These are necessary for the domain because sexual praxis is highly exposed to person-near misreadings, shame, identity fixation, and tribunal drift. But these constructs remain overlay-level. Outer legibility is not a new operator. Path-cost is not a new operator. Disturbance language is not person language. Publicness is not a replacement for □, Ω, Θ, or X.

QC adds the formal-technical case.

QC may use QFRAME, QPREP, QMEASURE, QORACLE_CALL, and related macros as structural handles. These may be useful for workflow annotation, audit traces, and bridge mapping. But macros do not replace Δ–Ψ. They expand, annotate, or organize PMS operator chains within a formal-technical bridge. They do not become base grammar.

PMS-RUST adds the implementation-artifact case.

PMS-RUST may use runtime names, crate boundaries, opcode buffers, model-validation paths, trace formats, demos, golden fixtures, and test modules. These are implementation structures. They may correspond to PMS-derived architecture or runtime constraints. They may help make a PMS-STACK Evidence Spine executable and inspectable. But they do not become PMS theory.

A runtime dialect is not PMS Base.

A module name is not an operator.

A test fixture is not a theoretical proof.

A JSONL trace is not external validation.

This is especially important where the runtime dialect uses names that resemble PMS operators. The existence of a runtime name does not settle the theoretical meaning of the corresponding PMS operator. Runtime vocabulary must remain version-bound, scope-bound, and implementation-bound.

The pressure point is consistent:

A local application or implementation component can become so useful that readers mistake it for a new primitive.

That is not only a reader error. It is a predictable stack risk created by PMS' own portability and implementability.

The falsifier is:

> **If domain applications and implementation artifacts remain useful without changing operator meanings, operator/domain/implementation drift does not occur.**

The test is not whether domain applications are rich or artifacts are useful. They should be.

The test is whether richness and executability remain subordinate to source-of-truth discipline.

4.4 Add-ons as Precision Layers, Not Primitives

Add-ons are precision, application, or hardening layers.

They are not PMS Base primitives.

This distinction is essential because add-ons often look more operational than the base grammar. They specify gates, reduced signatures, drift catalogues, profiles, schemas, claim constraints, misuse markers, and artifact structures. They can make PMS easier to apply and harder to misuse.

That makes them valuable.

It also makes them dangerous.

> **Precision must not be mistaken for authority.**

An add-on may sharpen an application, but it must not be read as having base status.

A precision layer can say:

```
This application requires a gate.
This reduced signature is admissible here.
This drift corridor is typical in this domain.
This output needs a claim boundary.
This artifact must remain reversible.
This public-use condition creates higher risk.
```

But it cannot say, as an add-on:

```
This is what Δ–Ψ means at base level.
This repairs an operator ambiguity in PMS Base.
This proves PMS Base is valid because the application is well-hardened.
This governance requirement overrides PMS grammar.
```

That would turn precision into authority.

Add-ons should reduce misuse and improve application discipline. They should not rescue PMS Base from structural pressure. If PMS Base has an operator ambiguity, calibration problem, or discriminative weakness, an add-on may help mark the problem in use. It may not retroactively

make the base grammar fully determinate.

This is especially important after Chapter 3.

Calibration showed that PMS' operational thresholds are not fully endogenous. Add-ons may help discipline threshold application. They may improve profile accuracy, claim hygiene, and misuse resistance. They may not eliminate threshold dependence by pretending that every hard case is solved at the profile layer.

The same applies to implementation-facing hardening. Test wrappers, acceptance gates, trace formats, documentation, and AI trust boundaries may harden an artifact. They may improve reproducibility, inspectability, and misuse resistance. They may not repair PMS Base.

The red line is:

> Do not allow add-ons or artifact hardening to repair PMS Base.

The correct formulation is:

> **An add-on may sharpen an application, but it must not be read as having base status.**

4.5 MIP/AH as Governance, Not Tribunal

MIP/AH has a constrained role in PMS Under Load.

MIP names Maturity in Practice. AH refers here to AH Precision: the optional Attack Surface / Hardening addon for analysis artifacts. In this chapter, MIP/AH functions only as governance, application discipline, and attack-surface hardening under constraint.

It may evaluate analysis artifacts for scope drift, language drift, inference risk, and misuse potential.

It may clarify application gates.

It may mark misuse risks.

It may support reversibility discipline.

It may discipline public use.

It may help prevent person-ranking, humiliation, artifact misuse, and tribunal drift.

But it must not become a tribunal or rescue layer.

The required sentence is:

> **MIP/AH constrain application; they do not rescue the base grammar.**

This is the most important governance boundary in the stack.

MIP/AH may inspect the responsibility of an analysis artifact. It may ask whether a claim is scene-bound, reversible, non-ranking, time-bound, and explicit about its scope. It may mark whether public circulation changes the misuse corridor. It may help prevent a PMS-derived output from becoming a person-level verdict.

But MIP/AH may not evaluate persons.

It may not rescue PMS Base.

It may not compensate for operator ambiguity.

It may not turn downstream governance into evidence for PMS Base validity.

It may not convert critique into compliance audit.

It may not make a PMS reading true because the artifact is well-governed.

This distinction matters because MIP/AH can make PMS applications look safer. Safety in application is valuable. But safety is not validation.

A governed artifact may still depend on contested calibration.

A reversible claim may still over-classify.

A non-humiliating output may still borrow authority from the wrong layer.

A public-use gate may prevent misuse without proving that PMS preserved discriminative force.

A repository may be well documented, bounded, and hardened without validating PMS Base.

MIP/AH therefore functions as downstream constraint, not epistemic rescue.

The local falsifier is:

> **If MIP/AH can be removed from a base-level critique without changing the force of that critique, the constraint rule is functioning.**

This does not mean MIP/AH is unimportant.

It means MIP/AH should change application safety, artifact discipline, and misuse risk — not the base-level force of an objection against PMS.

If removing MIP/AH changes whether a base-level objection against PMS still has force, then MIP/AH has become a rescue layer.

That would be Stack Drift.

4.6 QC as Bridge, Not Proof

QC occupies a different stack position from ordinary domain layers.

It is not merely another application domain in the same sense as CRITIQUE, CONFLICT, ANTICIPATION, LOGIC, or SEX. It functions as bridge stress under formal-technical load.

QC may show that PMS can be used for structural mapping of quantum-computational workflows. It may provide macro handles, audit traces, workflow annotations, iteration-depth policies, measurement/non-event mapping, and structural comparability.

But QC is bridge.

It is not proof.

The red line is:

```
Do not write:
PMS maps QC, therefore PMS is substrate-universal.
```

Write:

```
PMS-QC may show bridge potential, but bridge potential is not proof.
```

The key sentence is:

> **QC tests portability under formal-technical load without converting mapping into physical explanation.**

This distinction must remain sharp.

A QC mapping may be structurally useful without being physically explanatory.

A QC macro may be audit-useful without guaranteeing correctness.

A policy check may improve inspectability without certifying safety.

A structural analogy may illuminate workflow organization without proving equivalence between PMS operators and quantum phenomena.

This is why QC is a stress case.

It tests whether PMS can maintain structural language where formal-technical temptation is high. The danger is not only overclaim by PMS. The danger is also interpretive inflation by readers: if PMS can map QC, readers may infer substrate universality, physical depth, or formal superiority.

Those inferences are inadmissible.

QC remains a BRIDGE CLAIM.

As a BRIDGE CLAIM, it may test whether PMS vocabulary can organize structural correspondences under formal-technical pressure. It may not upgrade those correspondences into proof, physical explanation, or substrate universality.

It tests mapping discipline under pressure.

It does not validate PMS Base.

4.7 STACK and PMS-RUST as Implementation Layer, Not Validation

STACK must also remain status-bound.

STACK may appear as realization horizon, implementation pressure, machine-form architecture, programming language structure, runtime discipline, distributed-system model, computational stack, or executable artifact projection. That is a serious implementation claim.

PMS-RUST v0.1 now makes this layer more concrete.

It documents a bounded PMS-STACK Evidence Spine: a Rust repository with runtime/toolchain components, including deterministic kernel execution, PMS.yaml model validation, IR/program construction, REPL/script tooling, JSONL trace/audit output, demo scripts, golden fixtures, tests, documentation, and a controlled AI proposal bridge.

This changes the implementation-evidence status.

It does not change PMS Base.

PMS-RUST means that STACK is no longer only a realization horizon or implementation pressure. Where the repository documents executable components, STACK also has bounded executable artifact-backed partial realization.

But implementation is not proof.

The red line is:

```
Do not write:
PMS can become machine architecture, therefore PMS is validated.
```

Write:

```
STACK may show realization pressure.
PMS-RUST may show bounded executable feasibility and artifact-backed partial realizatior
Neither validates PMS Base.
```

The key sentence is:

> **Executable realization changes the status of the implementation claim, not the status of PMS Base.**

This distinction matters because realization can look like proof.

If a grammar can be implemented, it may feel more real.

If it can generate architecture, it may feel more fundamental.

If it can be mapped onto machines, it may feel less dependent on human interpretation.

If parts of it can be executed, it may appear validated.

If tests pass, it may appear confirmed.

If traces exist, it may appear externally audited.

If a repository is public, it may appear evidentially stronger than the argument itself.

But none of that follows automatically.

An implementation may instantiate a grammar without proving that the grammar is complete.

An architecture may operationalize distinctions without proving they are final.

A machine form may test realizability without establishing epistemic priority.

A test suite may show conformance to specified cases without proving praxis adequacy.

A JSONL trace may make runtime behavior inspectable without becoming external validation.

A public repository may support artifact evidence without becoming base authority.

PMS-RUST therefore intensifies the Stack Drift problem rather than solving it.

It makes layer discipline more inspectable because there are concrete artifacts to examine. It does

not make layer discipline unnecessary. It does not dissolve the distinction between PMS Base, PMS.yaml, STACK, runtime dialect, tests, traces, docs, and external validation.

The following distinctions must remain active:

```
repo evidence ≠ base validation
tests ≠ proof
runtime dialect ≠ PMS 1.3 theory
model validation ≠ praxis adequacy
execution trace ≠ external validation
public repository ≠ epistemic warrant
executable artifact ≠ final grammar
```

STACK therefore belongs in *PMS Under Load* only as an IMPLEMENTATION CLAIM: realization horizon and implementation pressure.

PMS-RUST belongs as an IMPLEMENTATION / ARTIFACT CLAIM: documented executable artifact-backed partial realization of a PMS-STACK Evidence Spine v0.1.

These claims may contribute to later external warrant, but they do not replace external application, calibration, rival comparison, or load-bearing falsifiers.

PMS-RUST may increase pressure on PMS.

It may not validate PMS.

4.8 Falsifier

The Stack Drift objection has a direct falsifier.

> **If PMS-derived outputs remain equally interpretable, reversible, claim-typed, and operator-strict across Base, domain, add-on, governance, bridge, implementation, and repository-artifact layers without layer confusion, stack drift does not occur.**

Expanded:

> **If PMS-derived outputs remain interpretable, reversible, operator-strict, and properly claim-typed across Base, domain, add-on, governance, bridge, implementation, and repository-artifact layers without authority borrowing, then the stack drift objection weakens.**

The stress case is not a single layer.

It is cross-layer transfer.

A PMS-derived output begins at the base grammar. It enters a domain layer. It is sharpened by an add-on. It is constrained by MIP/AH. It is compared to a bridge mapping. It is projected into implementation pressure. It is then documented in a repository artifact such as PMS-RUST.

If, across those transfers, the output remains:

```
operator-strict,
claim-typed,
reversible,
scene-bound,
non-tribunal,
non-proof-inflated,
non-implementation-inflated,
non-repo-inflated,
and clear about source of truth,
```

then Stack Drift weakens.

In that case, PMS portability would not be producing layer confusion. It would be producing disciplined transfer.

But if the output changes status while retaining PMS vocabulary, the objection strengthens.

If a domain construct begins to function as a base operator, Stack Drift appears.

If an add-on becomes a primitive, Stack Drift appears.

If MIP/AH becomes tribunal, Stack Drift appears.

If QC becomes proof, Stack Drift appears.

If STACK becomes validation, Stack Drift appears.

If PMS-RUST becomes PMS Base evidence, Stack Drift appears.

If tests become proof, Stack Drift appears.

If runtime dialect becomes PMS 1.3 theory, Stack Drift appears.

If model validation becomes praxis adequacy, Stack Drift appears.

If execution traces become external validation, Stack Drift appears.

If implementation pressure or executability becomes truth, Stack Drift appears.

The falsifier is therefore real.

PMS can weaken the objection only by preserving layer discipline under actual portability and documented implementation pressure, not by stating that the layers are different.

PMS-RUST makes this falsifier sharper. It gives the stack a concrete artifact surface. The objection weakens only if that surface remains claim-typed, bounded, and non-validating.

4.9 Conclusion

Stack Drift does not show that PMS fails.

It shows that PMS' portability and implementability require disciplined layer control.

This is a reduction, not a rejection.

PMS may remain internally coherent. Its domain layers may remain useful. Its add-ons may sharpen

applications. MIP/AH may improve analysis-artifact responsibility and expose attack surfaces. QC may offer bridge potential. STACK may create implementation pressure. PMS-RUST may document executable artifact-backed partial realization of a PMS-STACK Evidence Spine v0.1.

All of that can be true.

But none of it cancels the stack problem.

The stronger PMS becomes across layers, the more carefully it must distinguish:

```
application from definition,
precision from authority,
governance from tribunal,
bridge from proof,
implementation from validation,
repo evidence from base authority,
tests from proof,
runtime dialect from PMS 1.3 theory,
model validation from praxis adequacy,
execution trace from external validation,
portability from equivalence.
```

The chapter's core reduction is therefore:

```
PMS may claim:
Its grammar can travel across layers when claim type and source-of-truth discipline are

PMS may claim:
PMS-RUST documents bounded executable artifact-backed partial realization of a PMS-STACK

PMS may not claim:
Layer transfer itself proves operator equivalence, base validity, governance authority,
```

The closing sentence is:

> **PMS becomes stronger only if its current portability and documented implementability remain distinguishable from equivalence, proof, tribunal authority, repository validation, and implementation truth.**

Chapter 5 does not inherit a solved stack problem. It shifts the pressure from layer drift to operator drift: Φ (recontextualization), the very power of reframing under PMS discipline, can itself begin to absorb the differences it should preserve. After Stack Drift, the question is no longer only whether the right layer is speaking, but whether a core operator can remain differentiating when it becomes too good at making non-fit readable.

5. Instability III — Φ Drift

Chapter Aim

This chapter treats Φ (recontextualization) as both one of PMS' strongest operators and one of its central risks.

The aim is not to attack recontextualization as such. PMS needs Φ. Without recontextualization, praxis would remain too rigid: frames could not shift, conflicts could not be re-read, responsibility could not adapt to new conditions, and operator chains would harden into repetition rather than learning.

The pressure point is narrower:

> **Φ becomes unstable where recontextualization stops preserving Δ (difference) and starts making non-fit compatible.**

Here, "non-fit" names material that does not yet sit comfortably inside PMS: rival readings, unresolved resistance, domain-specific remainder, incompatibility, or a difference that still pressures the grammar after translation.

This is an internal instability.

Φ is not an imported interpretive habit added from outside PMS. It belongs to PMS' own operator grammar. It is one of the reasons PMS can travel across domains and remain productive under pressure. That is precisely why its drift matters.

Φ is risky not because it fails.

Φ is risky because it may succeed too smoothly.

A resistant object can be placed into a new frame. An incompatible reading can be made structurally legible. A rival interpretation can be translated into PMS vocabulary. A conflict can be reframed. A moral tension can be reconstructed as responsibility-readability. A sexual scene can be internally redescribed. An Eden reading can be converted into category-error analysis.

All of these moves may be legitimate.

But each move raises the same question:

> Does recontextualization preserve the difference that made recontextualization necessary, or does it convert that difference into compatibility?

This chapter tests that question.

5.1 Origin Inside PMS

Φ (recontextualization) is an internal PMS operator.

In PMS Base, Φ names recontextualization: the transformation of an existing structure by placing it into a new frame. It depends on Θ (temporality), Ω (asymmetry), and □ (frame). This matters because Φ is not merely a rhetorical device, an interpretive afterthought, or an optional analytic style. It is part of how PMS understands praxis as transformable.

PMS needs Φ because praxis changes.

Frames shift.

Roles are re-read.

Conflicts acquire new contexts.

Failures become legible differently over time.

Responsibility may need to be reconstructed after changed conditions.

A non-event may need to be carried in a new frame without being erased.

Without Φ, PMS would be able to describe patterning, asymmetry, temporality, and even distance, but it would lack a central operator for transformation. PMS would become structurally rigid.

That is why the red line is important.

Do not write:

```
Φ becomes a problem when recontextualization absorbs the difference it should preserve.
```

Write:

```
Φ becomes a problem when recontextualization absorbs the difference it should preserve.
```

This distinction prevents the chapter from attacking PMS' strength as if it were merely a weakness. Φ is indispensable. The instability emerges because indispensable operators can overfunction.

The core sentence is:

> **Φ (recontextualization) is indispensable to PMS; therefore Φ drift is an internal risk, not an external contamination.**

If Φ were merely an abuse from outside PMS, the solution would be simple: reject the misuse. But Φ drift cannot be handled that way. It arises from a real PMS capacity: the capacity to make resistant material newly legible.

The Under Load question is therefore not whether PMS should abandon Φ.

It is whether PMS can distinguish clarifying recontextualization from absorptive recontextualization.

5.2 Drift Mechanism

Φ (recontextualization) drifts when recontextualization turns resistance, incompatibility, or non-fit into a stable PMS-readable form without preserving enough Δ (difference).

The minimal mechanism is:

```
Δ (difference) appears.
□ (frame) frames it.
Φ (recontextualization) recontextualizes it.
Σ (integration) may stabilize it.
```

```
But the original Δ (difference) may no longer exert pressure.
```

At first, the sequence is legitimate.

A difference appears. It may be unclear, threatening, disruptive, or resistant. A frame gives it local relevance. Φ then places that difference into another context, where it becomes more intelligible. If the recontextualization is good, the original difference remains visible inside the new frame. PMS has not erased the difference; it has made the difference more structurally legible.

But Φ drift begins when the new context becomes too successful.

The resistant object no longer resists.

The incompatibility no longer pressures the analysis.

The rival reading is translated before it has been allowed to remain genuinely external.

The non-fit becomes another PMS-readable variation.

The analyst gains coherence, but loses discrimination.

This is why the risk is not recontextualization.

The risk is recontextualization without retained resistance.

A clarifying Φ (recontextualization) changes the frame while preserving the load-bearing difference. An absorptive Φ changes the frame in such a way that the difference becomes easier to integrate than it should be.

The falsifier for this section is:

> **If Φ (recontextualization) preserves Δ (difference) while changing context, then recontextualization remains clarifying rather than absorptive.**

This gives the chapter its test.

After Φ (recontextualization), ask:

```
Is the original Δ (difference) still visible?
Does it still constrain the new reading?
Can it still resist PMS vocabulary?
Can it still force revision, narrowing, or non-capture?
Or has it become a stable item inside PMS coherence?
```

If Δ (difference) remains active, Φ has clarified.

If Δ disappears into compatibility, Φ has drifted.

Elegance is not the criterion. A recontextualization may be more elegant, more coherent, and more PMS-readable while still being less discriminating.

The key sentence is:

> **The risk is not recontextualization, but recontextualization without retained resistance.**

5.3 Evidence

Φ (recontextualization) drift becomes visible across several PMS domains.

The purpose of this section is not to overload the chapter with domain detail. The purpose is to show that the same operator risk appears in distinct forms: moral readability, conflict stabilization, category-error translation, sexual outer legibility, and future-oriented possibility.

5.3.1 LOGIC

LOGIC shows that moral readability can be structurally reconstructed without becoming a final settlement of the ought question.

This is one of LOGIC's strengths.

PMS-LOGIC distinguishes responsibility from ought. Responsibility asks where consequences belong. Ought asks what must be done. LOGIC operates on attribution of consequences, not on prescriptive demand. It can therefore reconstruct why non-action becomes non-neutral under effective capacity, asymmetry, and temporality without deriving a moral command.

This is a precise and valuable recontextualization.

The is/ought tension is not resolved by smuggling in a norm. It is recontextualized: what appears as moral necessity is reconstructed as responsibility-readability under irreversible praxis. PMS can say:

```
This is not an ought.
This is attributable consequence under Ω (asymmetry) and 0 (temporality).
```

That distinction is strong.

But Φ drift appears if this recontextualization starts to sound like final settlement.

The danger is that PMS makes the moral problem structurally readable so effectively that the gap between attribution and ought appears closed. Responsibility becomes legible. Consequence becomes assignable. Non-action loses innocence. The analysis becomes coherent.

But the original Δ may weaken:

```
consequence ≠ capacity
capacity ≠ attribution
attribution ≠ responsibility
responsibility ≠ ought
ought ≠ guilt
guilt ≠ structural readability
```

LOGIC is strongest when these differences remain active.

It drifts when recontextualization makes them look resolved by structural reconstruction alone.

The red line is:

> Do not let LOGIC sound like final settlement of the Hume problem.

PMS may reconstruct how responsibility becomes readable without deriving an ought. It may not convert that reconstruction into a hidden proof that the ought problem has been solved inside PMS.

The key sentence is:

> **LOGIC is strongest when it preserves the gap between attribution and ought; it drifts when recontextualization makes that gap look resolved.**

In LOGIC, Φ clarifies when moral vocabulary is recontextualized into consequence-attribution while the non-identity between attribution and obligation remains visible.

Φ absorbs when the ought-gap no longer remains analytically load-bearing after recontextualization.

5.3.2 CONFLICT

CONFLICT shows Φ (recontextualization) under pressure when Σ (integration) no longer holds.

In ordinary productive critique, Φ (recontextualization) can open a path toward Σ (integration). A situation is re-read; a new frame makes correction possible; integration can follow. Recontextualization supports repair.

Conflict changes the condition.

In PMS-CONFLICT, Σ (integration) may become structurally blocked, too costly, or no longer viable. Binding may persist even where shared continuation fails. Asymmetry and temporality intensify the cost of revision. Distance may be priced or delegitimized. Under those conditions, Φ can continue to operate, but not as a bridge to Σ.

It can become a substitute for Σ (integration).

This is the decisive CONFLICT pressure.

Reframing may keep the scene intelligible even where integration no longer carries. The conflict remains readable. Each role-position can locate itself in a new frame. The system can continue. But the continuation is not necessarily integrated. It may be sustained by narrative embedding rather than synthesis.

That is not always misuse.

In some conflict regimes, Φ-substitution may be the only available form of legibility. If Σ would require coercive reconciliation, false harmony, or unacceptable cost redistribution, then Φ may preserve local viability without pretending that integration is available.

But this is exactly where Φ drift becomes possible.

Reframing can make unresolved conflict appear structurally processed.

The system may say:

```
The conflict has been recontextualized.
```

```
The positions are legible.
The asymmetries have been named.
The trajectories are understandable.
```

All of that may be true.

But the question remains:

```
Has integration occurred?
Or has Φ merely replaced Σ as the carrier of legibility?
```

In CONFLICT, Φ becomes unstable where reframing substitutes for integration without making that substitution accountable.

The key sentence is:

> **In CONFLICT, Φ becomes unstable where reframing substitutes for integration without making that substitution accountable.**

The problem is not that Φ-substitution occurs.

The problem is when substitution is misread as integration, or when a legibility gain is presented as if Σ had carried the conflict.

5.3.3 EDEN

EDEN stresses Φ (recontextualization) through translation dominance, category-error risk, and sensitive Ω (asymmetry).

PMS-EDEN is not used here as theological authority, gender theory, or final interpretation. It is used as a high-pressure example of what PMS can do when it translates sensitive symbolic material into operator grammar.

EDEN explicitly treats several dominant readings as category errors within its reconstruction: Eden as immaturity, breach as epistemic gain, sin as moral primary concept, sex difference as essence, and reciprocity loss as bad intent. In each case, PMS translates the reading into operator terms and tests whether it preserves Δ–Ψ traceability, from Δ (difference) through Ψ (self-binding).

This is powerful.

It shows PMS at high recontextualizing capacity.

A theological, moral, symbolic, or anthropological scene becomes structurally legible as praxis: Δ (difference), □ (frame), Θ (temporality), Ω (asymmetry), Λ (structured absence / non-event), Σ (integration), Ψ (self-binding), and related derived axes can be used to reconstruct what becomes visible without relying on moral or psychological primitives.

But this is also why EDEN is risky.

The stronger the translation, the easier it becomes for PMS vocabulary to absorb rival meanings before they have exerted full pressure.

A theological reading may become “imported normativity.”

A symbolic reading may become “non-PMS primitive.”

A gendered reading may become “role/frame configuration.”

A moral reading may become “category error.”

Each of these translations may be justified within PMS-EDEN.

But Under Load must ask:

```
Does PMS preserve the pressure those rival readings place on the scene?
Or does it convert them too quickly into PMS-legible errors?
```

This is the translation dominance risk.

EDEN itself contains important guardrails: it does not claim to exhaust lived meaning, does not argue for or against Eden as worldview, and does not claim monopoly over theology, symbolic anthropology, phenomenology, discourse history, or normative political theory. Those guardrails matter because they keep Δ alive outside the PMS reconstruction.

But the risk remains structural.

EDEN stresses Φ because sensitive material can be made structurally legible while rival meanings are too quickly absorbed into PMS vocabulary.

The red line is:

> Do not write EDEN as final theological or gender interpretation.

The key sentence is:

> **EDEN stresses Φ because sensitive material can be made structurally legible while rival meanings are too quickly absorbed into PMS vocabulary.**

In EDEN, Φ (recontextualization) clarifies where category-error analysis preserves the fact that other readings may still carry non-PMS dimensions. Φ absorbs where PMS translation makes those dimensions appear already defeated.

5.3.4 SEX

SEX shows why recontextualization is not reversal.

This is the hardest boundary case because PMS-SEX repeatedly distinguishes internal framing from outer legibility. A scene may be internally framed as play, devotion, experimentation, intimacy, service, or mutuality. That internal framing is real. It matters for how the scene is inhabited.

But it does not erase outer form.

PMS-SEX states the boundary clearly:

> **Reframing is not reversal; internal framing does not erase outer legibility.**

Outer legibility here remains a configuration claim, not a person verdict.

This is the required hard limit for Φ.

Recontextualization may alter how a scene is understood, narrated, inhabited, or carried. It may change the role of an action-form in the participants' local frame. It may affect the meaning of exposure, asymmetry, publicness, or markedness.

But it does not retroactively undo what has already become structurally effective under Θ (temporality).

A later narrative can reframe.

It cannot reverse temporality.

A practice label can clarify internal meaning.

It cannot automatically dissolve outer legibility.

Internal framing can be real.

It cannot become total.

This is not a moral verdict.

It is a structural limit on Φ.

SEX is therefore a crucial case for Chapter 5 because it prevents recontextualization from becoming omnipotent. It shows that meaning-change and form-erasure are not the same operation. Φ can shift the frame, but it does not automatically cancel exposure, publicness, asymmetry, consequence, or the action-form's outer readability.

The red line is strict:

> Do not turn SEX into shame, diagnosis, identity judgment, or person-ranking.

The relevant object remains the configuration, not the person.

The key sentence is:

> **SEX marks a hard limit for Φ (recontextualization): recontextualization may alter meaning, but it does not automatically undo form.**

This is the clearest anti-drift rule in the chapter.

Φ (recontextualization) clarifies when it changes internal or contextual meaning while keeping outer legibility and temporal consequence visible.

Φ absorbs when internal framing is used to erase outer form.

5.3.5 ANTICIPATION as Boundary Case

ANTICIPATION is only a boundary case here.

It shows Φ under future-oriented conditions, where the relevant object may not yet have occurred. This makes recontextualization delicate: a future possibility can be responsibly reframed, but it can also be smoothed into avoidance, control fantasy, or premature closure.

A responsible anticipatory Φ can update the frame:

```
This possibility remains open.
Its likelihood has changed.
Its consequence profile has shifted.
The frame of responsibility must be revised.
```

But a drifting anticipatory Φ may say:

```
This is not avoidance; it is restraint.
This is not control; it is preparedness.
This is not prediction; it is responsible anticipation.
This is not premature closure; it is structural clarity.
```

Each sentence may be true in some cases.

Each can also function as smoothing.

The test is whether Λ (structured absence / non-event) remains open, X (distance) remains active, and the future possibility is not converted into certainty, control, or action warrant.

The key sentence is:

> **In ANTICIPATION, Φ remains responsible only where future possibility is recontextualized without being closed, controlled, or converted into prediction.**

This boundary case should not dominate the chapter. It only confirms the main pattern: Φ must preserve the resistance of what it recontextualizes.

5.4 Φ vs Δ

The central distinction of the chapter is between Φ (recontextualization) that preserves Δ (difference) and recontextualization that dissolves Δ into coherence.

Φ (recontextualization) remains disciplined where it transforms context while preserving the difference that required transformation.

Φ drifts where it dissolves that difference into compatibility.

The test of Φ is therefore simple:

> **The test of Φ (recontextualization) is whether recontextualization preserves the Δ (difference) that made recontextualization necessary.**

This test applies across the evidence cases.

In LOGIC, the question is whether structural responsibility preserves the difference between attribution and ought.

In CONFLICT, the question is whether reframing preserves the difference between legibility and integration.

In EDEN, the question is whether PMS category-error analysis preserves the difference between PMS reconstruction and rival meaning-fields.

In SEX, the question is whether internal framing preserves the difference between inhabited meaning and outer legibility.

In ANTICIPATION, the question is whether future-oriented recontextualization preserves the difference between open possibility and closure.

In each case, PMS may become more coherent after Φ (recontextualization).

That coherence is not yet evidence that Δ (difference) has survived.

This is the danger: PMS can appear stronger exactly where the original difference has become weaker.

A rigorous Φ (recontextualization) must keep a trace of what resisted it. That trace does not need to remain unchanged. Recontextualization may transform the difference. But the transformed difference must still constrain the analysis.

The difference must still be able to say no.

That means it must still be able to force narrowing, revision, suspension, rival comparison, or non-capture.

If it cannot say no, it has been absorbed.

This is where genuine non-capture remains important. Some resistant material may not become PMS-legible without loss. Some rival readings may not translate cleanly. Some outer forms may not dissolve into internal meanings. Some conflicts may not integrate. Some moral gaps may not settle. Some future possibilities may not be responsibly reframed without residue.

PMS must be able to leave such resistance standing.

Otherwise, Φ becomes a coverage machine.

5.5 Result

Φ (recontextualization) remains one of PMS' strongest operators.

But its strength requires a reduction in claim confidence.

PMS may claim:

```
Φ can productively recontextualize praxis under operator discipline.
```

PMS may not claim:

```
Every resistant non-fit becomes adequate once recontextualized.
```

This reduction matters.

It prevents PMS from treating every challenge as a frame problem waiting for better Φ. Some challenges may be frame problems. Some may be calibration problems. Some may be domain-specific structures PMS has not preserved. Some may be rival grammars. Some may be genuine non-capture.

Φ strengthens PMS only where recontextualization remains answerable to the difference it transforms.

That means every strong Φ move should carry a visible reduction cost:

```
What has changed?
What has been preserved?
What has been lost?
What still resists?
What cannot be settled by this new frame?
What would show that this recontextualization has absorbed rather than clarified?
```

A Φ (recontextualization) that cannot answer these questions has become too stabilizing.

This is especially important because Φ often feels intellectually generous. It seems to give the object a better frame. It makes more sense of it. It reduces confusion. It resolves apparent contradiction.

But a better frame can still absorb.

A smoother reading can still weaken resistance.

A more elegant PMS reconstruction can still lose the object.

The chapter's result is therefore:

> **Φ strengthens PMS only where recontextualization remains answerable to the difference it transforms.**

5.6 Falsifier

The Φ Drift objection has a direct falsifier.

Required form:

> **Two incompatible readings remain PMS-stable through Φ, but PMS cannot internally determine whether Φ clarified a difference or absorbed it.**

Expanded:

> **If two incompatible readings can both be stabilized through PMS-consistent recontextualization, and PMS cannot internally determine whether Φ preserved or absorbed the relevant Δ (difference), then Φ drift occurs.**

The stress case is this:

Two readings of the same scene, using the same descriptive material, are incompatible.

One reading treats a resistant element as a genuine external pressure on PMS: a rival grammar, a non-capture point, a domain-specific structure, or a non-fit that should remain unresolved.

The other reading recontextualizes the same element into PMS grammar: it becomes a drift form, a category error, a structural remainder, a responsibility-readable configuration, a conflict attractor,

or an outer-legibility distinction.

Both readings remain PMS-conform.

Both preserve operator discipline.

Both avoid person-ranking.

Both remain scene-bound.

Both are coherent.

But PMS cannot determine, from within its own grammar, whether the second reading clarified the original Δ (difference) or absorbed it.

If that occurs, Φ drift holds.

The inverse falsifier is equally important:

> **If PMS can reliably distinguish clarifying recontextualization from absorptive recontextualization while preserving Δ (difference), the Φ drift objection weakens.**

This makes the objection non-immunized.

PMS can weaken the chapter's claim if it can show that its Φ operations preserve load-bearing difference, maintain rival pressure, leave non-capture open, and do not convert coherence into proof.

But PMS cannot weaken the objection by merely producing another coherent recontextualization.

That would repeat the risk.

The test is not whether Φ can make the scene readable.

The test is whether Φ can make the scene readable while keeping the original resistance active.

5.7 Conclusion

Φ (recontextualization) does not need to be weakened as an operator.

It needs to be bounded as a source of stabilization.

This chapter does not argue against recontextualization. PMS would lose too much without Φ. Recontextualization is necessary for learning, transformation, conflict handling, responsibility reconstruction, domain transfer, and self-application.

But PMS must reduce its confidence where recontextualization produces coherence too easily.

The core reduction is:

```
PMS may claim:
Recontextualization can transform a resistant object into a more legible configuration u

PMS may not claim:
Coherence after recontextualization proves that the resistant object has been adequately
```

Downstream guardrails may mark risky recontextualization, including where MIP/AH exposes attack surfaces in the analysis artifact. They may discipline how a recontextualized claim is applied, but they do not solve the operator-level question of when Φ (recontextualization) clarifies and when it absorbs.

Guardrails cannot decide, at the operator level, whether Φ has preserved Δ (difference). If MIP/AH were used to solve that problem, it would become a rescue layer.

The closing sentence is:

> **Recontextualization is not reversal, and coherence after Φ (recontextualization) is not yet evidence that Δ (difference) has been preserved.**

Chapter 6 does not inherit a solved Φ problem. It shifts the pressure from operator drift to structural selectivity: PMS does not prescribe norms in the ordinary sense, but it privileges certain forms of praxis and analysis over others.

6. Instability IV — Meta-Normativity

Chapter Aim

This chapter tests PMS at the point where structural description begins to select.

“Select” here does not mean command, praise, blame, or moral ranking. It means that PMS treats some configurations as more admissible within its application discipline: more viable, reversible, accountable, or application-safe than others.

The claim is not:

```
PMS is secretly moral theory.
```

The claim is:

> **PMS does not prescribe norms, but it structurally privileges certain forms of practice over others.**

This distinction matters.

If PMS were simply a moral theory in disguise, the critique would be straightforward: expose the hidden norm and ask whether it is justified. But PMS does not operate that way. It does not derive a moral code. It does not produce a catalogue of duties. It does not rank persons by worth. It does not say, in the ordinary prescriptive sense, what one must do.

At the same time, PMS is not neutral.

Its grammar makes some forms of praxis more legible, more viable, and more defensible than others. It favors distance over fusion, reversibility over closure, self-binding over unowned impulse, accountability over consequence displacement, scene-bound claims over global labels, and non-person-ranking over identity verdicts.

This is meta-normativity.

Meta-normativity does not mean that PMS issues norms.

It means that PMS contains a second-order grammar of selection: a practice becomes more admissible within PMS application discipline when it can preserve X (distance / reflective distance), remain reversible, carry asymmetry without humiliation, bind consequences without moralizing them, and avoid turning structural analysis into tribunal use.

The pressure point is therefore precise:

> **A grammar can be non-prescriptive and still structurally non-neutral.**

6.1 Claim

PMS does not derive an ought.

But it selects for practices with:

```
X (distance / reflective distance),
```

```
reversibility,
D (Dignity-in-Practice),
Ψ (self-binding),
non-person-ranking,
misuse resistance,
accountable direction,
capacity-sensitive responsibility.
```

This is not an accidental feature. It follows from PMS' own structure.

X (distance / reflective distance) is not just one operator among others in applications under pressure. It functions as a condition of reflective interruption: the ability not to fuse with impulse, role, certainty, or the object of analysis.

Reversibility is not merely politeness. It is the requirement that claims remain revisable, scene-bound, and non-global.

D (Dignity-in-Practice) is an application constraint, not a proof of ontological worth and not an external validation of PMS Base.

Ψ (self-binding) is not moral purity. It is the capacity to carry commitment across time without externalizing consequence.

Non-person-ranking is not decorative humanism. It prevents structural analysis from becoming identity judgment.

Misuse resistance is not public relations. It is part of keeping analysis from becoming domination under PMS vocabulary.

Capacity-sensitive responsibility is not an ought. It marks where consequences become attributable under effective capacity, Ω (asymmetry), and Θ (temporality).

These selections are not commands. PMS does not say:

```
You must live like this.
```

But it does say, structurally:

```
A practice that collapses X (distance), hardens irreversibly, ranks persons, externalize
```

That is already directional.

The key sentence is:

> **PMS does not command a form of life, but it makes some forms of practice structurally more legible, viable, and defensible than others.**

This is the chapter's instability.

PMS can deny that it prescribes norms.

It cannot deny that its grammar selects.

6.2 Praxis

Meta-normativity appears most clearly in application.

PMS' preferred praxis forms are not neutral. They favor:

```
interruption over drift,
self-binding over impulse,
reversibility over closure,
accountability over unowned consequence,
scene-bound analysis over global labeling,
dignity-preserving language over humiliation,
misuse resistance over rhetorical victory.
```

These preferences are structural, not moralistic.

In CRITIQUE, a critique scene remains admissible as critique only where X (distance / reflective distance), reversibility, and D (Dignity-in-Practice) are preserved. The analysis must remain structural, not personal; it must describe roles, frames, cost layouts, exposure gradients, and stabilization paths rather than motives, traits, or moral qualities.

In SEX, the same selectivity appears under higher risk. The relevant object is the configuration, not the person. Structural markers such as drift, outer legibility, path-cost, and viability may be used only as scene-bound claims; they do not authorize diagnosis, identity ranking, humiliation, or person-level valuation.

These are not external moral additions.

They are conditions under which PMS application remains formally valid.

The issue is therefore not that PMS moralizes practice.

The issue is that its grammar of viability is already selective.

A practice without interruption capacity may still exist. PMS can describe it. But as PMS application, it loses validity where it collapses into fusion, coercion, irreversible labeling, or person-ranking.

A critique without reversibility may still be rhetorically powerful. PMS can describe its force. But PMS cannot treat it as admissible critique if it fixes persons globally or turns description into a weapon.

A sexual scene may have internal meaning. PMS can preserve that meaning. But PMS-SEX does not let internal framing erase outer legibility, exposure, asymmetry, or temporality.

This is structural selectivity.

It is not a moral code.

But neither is it neutrality.

The key sentence is:

> **The issue is not that PMS moralizes practice, but that its grammar of viability is already**

> **selective.**

6.3 Diagnosis

Meta-normativity begins where descriptive legibility carries a built-in preference for certain structural forms.

PMS often presents itself as structural rather than normative. That is correct in the ordinary sense. It does not begin from values, duties, virtues, commands, or prescriptions.

But PMS does not describe every structural form as equally viable.

It distinguishes:

```
distance from fusion,
integration from stabilization,
self-binding from externalized demand,
responsibility from unowned consequence,
critique from pillory,
recontextualization from reversal,
outer legibility from person judgment,
anticipation from prediction or control.
```

These distinctions do descriptive work.

They also select.

A form of praxis that sustains X (distance / reflective distance) is treated as more viable than one that collapses into immediate enactment.

A claim that remains reversible is treated as more admissible than one that fixes persons or groups.

An analysis that preserves D (Dignity-in-Practice) is treated as admissible, while one that humiliates or ranks persons leaves PMS application discipline.

A configuration that carries Λ (structured absence / non-event) without forcing closure is treated as more disciplined than one that turns absence into certainty or action compulsion.

PMS therefore produces second-order preferences through its validity conditions.

The pressure point is that selection may appear descriptive while functioning as a validity filter.

This does not mean PMS is secretly issuing commands. It means that validity inside PMS is not purely neutral. The grammar decides which forms count as disciplined, viable, reversible, or structurally accountable.

The key sentence is:

> **Meta-normativity begins where descriptive legibility carries a built-in preference for certain structural forms.**

This is a reduction claim.

PMS may keep saying that it is not conventional moral theory.

It must stop saying, if it ever does, that it is merely descriptive in a normatively neutral sense.

6.4 Hidden Selection

The strongest meta-normativity in PMS does not appear as moral language.

It appears inside apparently technical distinctions.

The main hidden selections are:

```
X (distance / reflective distance) over immediacy,
reversibility over closure,
Ψ (self-binding) over impulse,
responsibility over unowned consequence,
scene-bound claims over global labels,
non-person-ranking over identity judgment,
Λ-carrying (structured absence / non-event) over premature closure,
misuse resistance over persuasive force.
```

Each selection can be defended structurally.

X over immediacy prevents fusion, reaction, and coercive enactment.

Reversibility over closure prevents premature fixation and person-near labeling.

Self-binding over impulse prevents consequence externalization.

Responsibility over unowned consequence prevents capacity from disappearing into innocence.

Scene-bound claims over global labels preserve revision and dignity.

Non-person-ranking over identity judgment prevents structural analysis from becoming hierarchy of worth.

Λ-carrying over premature closure prevents uncertainty from being filled with dogma, strategy, or control.

Misuse resistance over persuasive force prevents PMS vocabulary from becoming instrumentally dominant.

None of these is a simple moral rule.

But together they create a strong direction.

The stronger these selections become, the less neutral PMS can claim to be.

This is not a defect by itself. A theory of praxis may need direction. A grammar without any selection would not be able to distinguish drift from viability, critique from pillory, responsibility from guilt, or recontextualization from absorption.

The problem is not direction.

The problem is unacknowledged direction.

The local falsifier is:

> **If PMS applications without X (distance / reflective distance), reversibility, D (Dignity-in-Practice), non-person-ranking, and misuse resistance could remain equally admissible, the meta-normativity objection would weaken.**

If PMS could treat such applications as equally valid without weakening its own grammar, then its structural selectivity would be less significant.

Under the current grammar, that does not appear structurally available.

Applications without X lose distance.

Applications without reversibility harden into closure.

Applications without D risk humiliation and degradation under asymmetry.

Applications without non-person-ranking become tribunal-like.

Applications without misuse resistance become instruments of stabilization or dominance.

Thus the selection is real.

The key sentence is:

> **A grammar can be non-prescriptive and still structurally non-neutral.**

6.5 LOGIC: Responsibility Without Ought

LOGIC is the main anchor for this chapter.

PMS-LOGIC is explicit: responsibility and ought are not the same. Responsibility asks, "To whom do the consequences belong?" Ought asks, "What must be done?" PMS-LOGIC operates on responsibility, not ought-statements.

This distinction is essential.

LOGIC allows PMS to reconstruct moral readability without deriving moral obligation. It can show that under Ω (asymmetry) and Θ (temporality), action and non-action become consequence-relevant. It can show that effective capacity inside a frame and across time can make non-action attributable. But it does not turn that attribution into a command. Effective potential introduces no obligation; it can create responsibility without obligation and exposure without prescription.

This is PMS at its strongest.

It preserves the ought-gap.

It prevents norm-smuggling.

It reconstructs why praxis becomes non-neutral without deriving a duty.

But this is also where meta-normativity becomes sharp.

Responsibility without ought still privileges accountable consequence-handling.

LOGIC does not say:

```
You ought to act.
```

But it does say:

```
Where effective capacity exists under asymmetry and time, non-action is no longer struct
```

That is not moral prescription.

But it is not neutral.

It selects for a world in which capacity, asymmetry, non-action, and consequence remain attributable rather than disappearing into description.

The red line is strict:

> Do not convert responsibility into hidden moral command.

If PMS treats attribution as obligation, LOGIC collapses into the very norm derivation it rejects.

But if PMS treats attribution as irrelevant because no ought has been derived, LOGIC loses its structural force.

The narrow path is:

```
No ought follows.
But consequence under capacity is not neutral.
```

This is meta-normativity in its cleanest form.

The key sentence is:

> **LOGIC preserves the ought-gap by not deriving normativity from attribution, while still showing that consequence under capacity is not structurally neutral.**

PMS survives here only by keeping both halves active.

If the first half disappears, PMS becomes hidden moral theory.

If the second half disappears, PMS becomes descriptively weaker than its own grammar permits.

6.6 MIP/AH: Governance Without Person-Ranking

MIP/AH enters this chapter only in a targeted role.

It does not prove PMS Base.

It does not justify PMS' direction from outside.

It does not rescue PMS from meta-normativity.

It makes the stack's selectivity visible.

The required sentence is:

> **MIP/AH makes the stack's selectivity visible; it does not justify that selectivity from outside.**

MIP/AH shows that the PMS stack permits some analysis forms and blocks others.

It blocks or constrains:

```
person-ranking,
tribunal use,
dignity reduction,
irreversible labels,
misuse-prone public claims,
unbounded analysis artifacts,
analysis that hides its attack surface,
language that turns risk into person judgment.
```

This is visible in MIP's own guardrails. The model treats itself as a toolbox and lens, not a total worldview; it cares about enactments, not slogans; it explicitly rejects clinical diagnostics, forensics, public pillory, and ranking in red zones.

The AH Precision layer intensifies this selectivity at the analysis-artifact level. It does not judge persons or actors. It exposes attack surfaces in the analysis artifact — scope drift, language drift, inference risk, and misuse potential — so that the artifact remains inspectable, iteratively improvable, and less prone to dogmatization or rhetorical closure. Its attack points describe analyses, not people, and the add-on does not change axioms, scoring logic, operators, or PMS Base.

This is not tribunal logic.

But governance is still selective.

At the governance layer, MIP/AH selects for:

```
attack-surface visibility over defensiveness,
iteration over finality,
scope discipline over leakage,
language hygiene over person labels,
publicity guardrails over uncontrolled circulation,
reversibility clauses over irreversible fixation.
```

Those preferences are not proof of PMS Base.

They are stack discipline.

MIP/AH therefore creates a useful mirror for Chapter 6. It shows that the PMS ecosystem does not merely describe; at the application layer, it filters admissible uses. It asks which analyses can circulate, which need hardening, which are too dignity-risky, which public contexts are too unstable, and which claims must remain reflective rather than decision-ready.

The pressure point is:

> Governance is not tribunal, but governance is still selective.

This matters because a user of PMS might say:

```
We are not making moral claims. We are only applying guardrails.
```

Under Load answers:

```
Guardrails are not moral verdicts, but they are still selection mechanisms.
```

The red line remains:

> Do not use MIP/AH to prove PMS Base is right.

MIP/AH can show how PMS applications are constrained.

It cannot show that the base grammar is true.

6.7 ANTICIPATION as Boundary Case

ANTICIPATION is only a boundary case here.

It shows that PMS selectivity also appears in future-oriented praxis.

PMS-ANTICIPATION explicitly refuses to treat anticipation as prediction, control, intelligence, or action advantage. It defines anticipation as a structural stance toward possible future events and non-events under Θ (temporality) and Ω (asymmetry), regulated by X (distance / reflective distance), and stabilized by Σ (integration) and Ψ (self-binding).

This is meta-normatively relevant.

PMS structurally favors:

```
restraint over control fantasy,
carrying Λ over premature closure,
viability over hit/miss success,
openness under temporality over predictive possession,
self-binding over imposing obligation on others.
```

Again, these are not conventional norms.

PMS does not say:

```
You must be restrained.
```

It says:

```
An anticipatory configuration that collapses Λ (structured absence / non-event) into cer
```

That is structural selectivity.

As a secondary boundary case, ANTICIPATION confirms the chapter's main point: even openness is not neutral in PMS. Openness must be carried in a disciplined way. The future must not be

possessed through prediction, control, or premature closure.

The key sentence is:

> **ANTICIPATION shows that even future-oriented openness is not neutral; PMS privileges ways of carrying the future without prematurely possessing it.**

This case remains secondary.

LOGIC remains the main anchor. MIP/AH remains a stack-selectivity mirror. ANTICIPATION only shows that the same meta-normativity appears where the object is not yet actual.

6.8 Consequence

The consequence is straightforward:

PMS must not describe itself as normatively neutral in its application grammar.

It may describe itself as direction-bearing, meta-normative, and accountable.

The reduction is:

```
PMS may claim:
PMS offers a structurally selective grammar of praxis legibility.

PMS may not claim:
PMS is merely descriptive and therefore normatively neutral.
```

This is not a collapse into moral theory.

The reduction is not from theory to morality.

It is from neutrality to accountable direction.

That matters because PMS' selectivity is part of its strength. Without it, PMS could not distinguish admissible critique from pillory, responsibility from guilt, anticipation from prediction, D (Dignity-in-Practice) from humiliation, or recontextualization from absorption.

A grammar of praxis that cannot select would not be useful under load.

But a grammar that selects while calling itself neutral risks concealing its own direction.

The task is therefore not to remove selectivity.

The task is to keep it explicit, bounded, and revisable.

PMS must be able to say:

```
This grammar privileges certain structural forms.
Those preferences are part of its claim.
They remain contestable.
They do not rank persons.
They do not prove PMS.
```

```
They do not defeat rival grammars in advance.
```

The key sentence is:

> **The reduction is not from theory to morality, but from neutrality to accountable direction.**

6.9 Falsifier

The meta-normativity objection has a direct falsifier.

Required form:

> **If PMS applications without X (distance / reflective distance), reversibility, D (Dignity-in-Practice), non-person-ranking, and misuse resistance could remain equally admissible, the meta-normativity objection would weaken.**

Expanded:

> **If PMS could treat practices lacking X (distance / reflective distance), reversibility, dignity guardrails, non-person-ranking, and misuse resistance as equally admissible without weakening its own grammar, then its meta-normative selectivity would be less structurally significant.**

The stress case is this:

A PMS application analyzes a high-stakes scene: a case where asymmetry, publicness, dignity risk, or consequence attribution is materially active.

It uses PMS operators correctly.

It identifies Δ, □, Λ, Ω, Θ, Φ, Χ, Σ, and Ψ where relevant.

It remains technically coherent.

But it lacks one or more of the following:

```
X as distance / reflective distance,
reversibility of claims,
D as Dignity-in-Practice,
non-person-ranking,
misuse resistance,
scope discipline,
analysis-artifact responsibility.
```

If PMS can still treat the application as equally valid, then the meta-normativity objection weakens.

In that case, the guardrails would be optional presentation rules rather than structural selection mechanisms.

But if the application becomes invalid or unstable without those conditions, then PMS is structurally selective.

The inverse falsifier is also clear:

```
If removing X (distance / reflective distance), reversibility, D (Dignity-in-Practice),
```

This falsifier is non-trivial because it does not ask whether PMS is moral theory.

It asks whether PMS can remain itself without its selective validity conditions.

If it cannot, then PMS is not neutral.

It is direction-bearing.

6.10 Conclusion

PMS survives meta-normativity only by naming its direction.

It does not need to confess itself as moral theory.

It does not need to deny its selectivity.

It needs to make the selectivity explicit, accountable, and revisable.

This chapter therefore reduces PMS in a specific way.

PMS may continue to say:

```
No ought is derived.
No moral code is produced.
No person is ranked.
No tribunal is authorized.
```

But PMS may not say:

```
Therefore the grammar is normatively neutral.
```

It is not neutral at the level of application validity.

It selects.

It selects for distance, reversibility, self-binding, capacity-sensitive responsibility, dignity-preserving analysis, misuse resistance, and scene-bound claims. Those selections are not hidden commands, but they are part of what PMS treats as disciplined praxis analysis and valid application.

The closing sentence is:

> **PMS does not need to deny its selectivity; it needs to keep that selectivity explicit, accountable, and revisable.**

Chapter 7 does not inherit a solved meta-normativity problem. It shifts the pressure from selectivity to success: even when PMS is coherent, selective, and productive, success itself may become a trap.

7. Instability V — Success Trap

Chapter Aim

This chapter treats PMS' success as a structural risk.

It does not diminish PMS' productivity. It does not argue that coherence is false, portability is suspicious, reproducibility is meaningless, or implementation artifacts are irrelevant. PMS is productive, coherent, portable, increasingly reproducible under disciplined use, and now partially artifact-backed through PMS-RUST v0.1. Those are strengths.

The question is whether those strengths can become stabilizing conditions.

The core claim is:

> **It becomes easier to produce PMS-consistent outputs than to detect where PMS stops differentiating.**

This is the success trap.

"Difference-sensitivity" means the continued ability to notice what does not fit, what still resists, what requires narrowing, what demands rival comparison, and what may remain genuinely non-captured after a PMS-consistent reading has been produced.

A weak model fails where it cannot produce stable readings. PMS' risk is different. PMS may continue to produce stable readings even where the difference that should pressure the grammar has become weaker, smoother, or harder to see.

PMS-RUST makes this risk more concrete. A documented implementation artifact with deterministic kernel behavior, REPL/script execution, JSONL traces, demos, golden fixtures, tests, and acceptance gates can make PMS more reproducible, inspectable, and stable at the artifact level. That is a real implementation success. Under Load asks whether artifact stabilizability increases faster than discrimination.

The chapter therefore asks:

```
When does PMS-consistency become easier to reproduce than difference-sensitivity?
When does coherence become trust?
When does trust reduce friction?
When do reproducible artifacts increase confidence faster than discrimination?
When does reduced friction make non-capture harder to notice?
```

Success is not denied.

It is placed under load.

PMS-RUST strengthens the success-trap test.

It does not answer it.

7.1 Mechanism

The success trap begins with a real strength.

PMS produces legibility.

It can take diffuse practical scenes and organize them through Δ (difference), □ (frame), Λ (structured absence / non-event), Ω (asymmetry), Θ (temporality), Φ (recontextualization), Χ (distance), Σ (integration), and Ψ (self-binding). This makes scenes more readable than they were before. It helps distinguish critique from reaction, conflict from escalation, responsibility from ought, anticipation from prediction, and recontextualization from reversal.

That legibility produces coherence.

Coherence produces trust.

Here, trust means methodological reliance on a grammar that has repeatedly worked, not personal belief or psychological attachment.

Trust reduces friction.

Reduced friction may reduce sensitivity to loss of difference.

Artifact stabilization can intensify this sequence.

A runtime, REPL, script language, trace format, test suite, golden fixture set, and acceptance gate make PMS-derived structures easier to reproduce and inspect. That can improve artifact discipline. It can also make the output feel safer before its discriminative adequacy has been retested.

The mechanism is:

```
PMS produces legibility.
Legibility produces coherence.
Coherence produces trust.
Trust reduces sensitivity to loss of difference.
Artifact reproducibility increases stabilizability.
Stabilization begins to substitute for discrimination.
```

This sequence does not always occur.

But it is structurally plausible.

The more often PMS produces useful readings, the easier it becomes to trust PMS-consistent outputs. The more stable the grammar becomes, the less every new output needs to fight for attention. The reading situation begins to expect PMS to make sense of the scene. Analysts, reviewers, tools, dialogic partners, or implementation artifacts may move more quickly through operator sequences. The vocabulary becomes familiar. The guardrails become routine. The output becomes plausible before its discriminative adequacy has been tested.

PMS-RUST can strengthen this stabilizing environment without validating PMS Base. Reproducible demos, golden fixtures, JSONL traces, REPL/script execution, deterministic kernel behavior, tests, and acceptance gates may increase confidence in artifact behavior. They do not by themselves

prove that PMS has preserved resistant difference.

This is not bad faith.

It is success.

A successful grammar creates habits of trust.

A successful artifact creates habits of reproducibility.

The pressure point is that trust and reproducibility can reduce the effort required to ask whether the reading has preserved enough difference.

The success trap begins where PMS-consistency becomes easier to reproduce than difference-sensitivity.

That is the key sentence:

> **The success trap begins where PMS-consistency becomes easier to reproduce than difference-sensitivity.**

The PMS-RUST version is sharper:

> **Artifact stabilizability may increase faster than discrimination.**

The danger is not coherence itself.

The danger is coherence without renewed exposure to what resists it.

7.2 Problem

Success is not enough.

PMS may become stable, coherent, portable, reproducible, useful, and artifact-backed without thereby becoming finally adequate or epistemically prior.

The necessary distinctions remain:

```
coherence ≠ truth
portability ≠ equivalence
productivity ≠ sufficiency
implementation pressure ≠ proof
executable artifact-backed partial realization ≠ PMS Base validation
tests ≠ proof
JSONL traces ≠ external validation
governance ≠ tribunal
self-application ≠ final arbitration
stabilizability ≠ validity
```

These distinctions are not rhetorical safeguards. They are the central reductions of *PMS Under Load*.

PMS may appear warranted not because it has preserved the relevant difference, but because it is

structurally easy to stabilize.

PMS-RUST adds a more concrete version of this risk: PMS may appear more warranted because an artifact is reproducible, testable, traceable, documented, and executable under constraints.

That sentence must be read carefully.

It does not say PMS is false.

It does not say PMS-RUST is merely cosmetic.

It says that a stable PMS-consistent output, or a reproducible PMS-derived artifact, is not yet evidence that PMS has preserved all relevant difference.

A coherent reading may be correct.

It may also be selectively constructed.

A portable grammar may reveal deep structural similarities.

It may also over-equate domains.

A productive operator sequence may clarify a scene.

It may also make non-fit compatible.

A governed artifact may be safer.

It may also feel more trustworthy than its discrimination warrants.

An executable artifact may show constrained feasibility.

It may also feel more validating than its claim type permits.

A passing test suite may show conformance to specified cases.

It may also be misread as proof.

A JSONL trace may make runtime behavior inspectable.

It may also be misread as external validation.

A self-application may expose limits.

It may also stabilize those limits as further PMS-readable material.

The success trap is therefore not an argument against PMS.

It is an argument against treating PMS success, reproducibility, or artifact stability as self-confirming.

The key sentence is:

> **PMS may appear validated not because it has preserved the relevant difference, but because it is structurally easy to stabilize.**

With PMS-RUST added:

> **PMS may appear validated not because it has preserved the relevant difference, but because its implementation artifacts are reproducible, testable, and traceable.**

This is one of the paper's hardest reductions.

PMS may continue to succeed.

PMS-RUST may continue to strengthen the implementation claim.

But success cannot be allowed to become proxy evidence for final adequacy.

7.3 PMS-Reading

PMS can read its own success trap.

That is part of the problem.

The success trap can itself be reconstructed in PMS terms:

```
Δ (difference) appears as resistant difference.
A (attractor) stabilizes successful patterns.
Φ (recontextualization) makes new objections recontextualizable.
Σ (integration) produces coherent synthesis.
Ψ (self-binding) binds the reading into repeatable practice.
X (distance) may become procedural rather than disruptive.
Λ (structured absence / non-event) may be acknowledged without remaining disruptive.
```

The artifact form can also be read in PMS terms:

```
A stabilizes repeated workflows.
Σ integrates scripts, traces, tests, and docs into a coherent toolchain.
Ψ binds the repository into repeatable practice.
X may become a procedural gate rather than disruptive distance.
Λ may be recorded as a rejected transition without interrupting the grammar.
```

This is a strong self-application.

PMS can describe how its own coherence becomes attractive, how repeated success creates an attractor, how guardrails become procedural, how recontextualization absorbs new material, and how integration produces stable readings.

It can also describe how PMS-RUST increases inspectability, reproducibility, and artifact self-binding.

But this does not prove escape.

The fact that PMS can read its own success trap does not prove that it escapes it.

The fact that PMS-RUST can document artifact behavior does not prove that PMS escapes the success trap.

This is the key sentence:

> **The fact that PMS can read its own success trap does not prove that it escapes it.**

The reason is simple.

A PMS-reading of PMS' success trap may itself become another successful PMS output.

A PMS-RUST trace of PMS-derived execution may itself become another stabilizing artifact.

It may be coherent.

It may be elegant.

It may be disciplined at the operator level.

It may preserve the correct red lines.

It may include the right reductions.

It may be reproducible.

It may be test-backed.

And still, it may stabilize the trap by making the trap itself legible inside PMS.

This is the self-application pressure.

PMS can say:

```
Here is how PMS success can become an attractor.
Here is how Φ can absorb non-fit.
Here is how Σ can over-stabilize.
Here is how X can become procedural.
Here is how Λ can be acknowledged without interrupting the system.
```

PMS-RUST can add:

```
Here is the script.
Here is the trace.
Here is the fixture.
Here is the gate.
Here is the reproducible runtime path.
```

All of that may be true.

But the Under Load question remains:

```
Does naming or tracing the trap increase sensitivity to resistant difference?
Or does it make the trap easier to contain?
```

If the reading or artifact increases sensitivity to resistant difference, it helps.

If it merely gives PMS another disciplined way to include its own risk, it repeats the success trap.

The red line is therefore:

> Do not let PMS' ability to describe the success trap dissolve the trap.

The artifact red line is:

> Do not let PMS-RUST's ability to trace, test, and reproduce PMS-derived behavior dissolve the success trap.

A grammar that can describe its own risks has achieved self-exposure.

A repository that can document its runtime behavior has achieved artifact inspectability.

Neither has achieved final self-arbitration.

7.4 AI Convergence

Human–AI dialogue is a possible amplifier of PMS stabilizability.

PMS-RUST adds a related but distinct amplifier: controlled AI proposal handling inside an implementation artifact.

This point must remain structural, not autobiographical.

The issue is not the psychology of the user, the authority of the assistant, the sincerity of the dialogue, or the impressiveness of the toolchain. The issue is that PMS has features that may make it especially reproducible in human–AI reflective practice and artifact-assisted workflows:

```
compact operator vocabulary,
strong layer discipline,
repeatable red lines,
portable claim types,
clear reduction formulas,
recursive self-application,
highly reusable sentence patterns,
scriptable execution paths,
traceable runtime events,
testable fixtures,
AI proposal boundaries.
```

A system with those features can become easier to stabilize in dialogue and in artifacts.

In such a dialogue, domain pressure, correction, intent, source control, judgment, rapid reformulation, pattern preservation, consistency checks, recursive application, and repeated guardrail enforcement can become tightly coupled. In such an artifact, script execution, model validation, trace output, golden fixtures, tests, and acceptance gates can become tightly coupled. That coupling may produce increasingly coherent PMS-consistent outputs.

This matters.

But it is not proof.

The required sentence is:

> **AI may amplify PMS' stabilizability without validating PMS' adequacy under load.**

A related repository sentence is:

> **PMS-RUST may amplify PMS' artifact stabilizability without validating PMS Base.**

Here, "adequacy under load" means whether the PMS reading preserves difference, rival pressure, and non-capture under the relevant conditions, not whether it becomes metaphysically certain.

This sentence cuts both ways.

AI convergence does not prove PMS.

AI convergence also does not disprove PMS.

PMS-RUST does not prove PMS.

PMS-RUST also does not disprove PMS.

A model may be easy to stabilize because it is formally disciplined and genuinely generative. It may also be easy to stabilize because its vocabulary absorbs objections smoothly. A repository may be reproducible because the implementation is well-bounded. It may also increase confidence faster than discrimination.

That is why AI convergence and artifact stabilizability belong in the success trap chapter.

Not as validation.

Not as suspicion by default.

As load conditions.

The red lines are:

```
Do not write:
AI proves PMS.
AI convergence validates PMS.
AI likes PMS, therefore PMS is true.
AI stabilizes PMS, therefore PMS is false.
PMS-RUST proves PMS.
PMS-RUST validates PMS Base.
PMS-RUST passes tests, therefore PMS is true.
PMS-RUST is traceable, therefore PMS has preserved difference.
```

The correct formulation is narrower:

```
AI-assisted dialogue may increase coherence, reproducibility, and speed of refinement.
PMS-RUST may increase executable reproducibility, traceability, and artifact discipline.
Those increases may amplify PMS' stabilizability.
Stabilizability is not validation.
```

The pressure point is not that AI is involved.

The pressure point is not that a repository exists.

The pressure point is that high stabilizability may make it harder to detect where difference has stopped exerting pressure.

7.5 Dialogic and Artifact Reproducibility

Dialogic reproducibility is evidence of generativity, not final adequacy.

Artifact reproducibility is evidence of implementation discipline, not final adequacy.

PMS may reproduce well through human–AI reflective practice because its grammar is compact, generative, and internally stabilizing. Operators can be reused. Claim types can be checked. Layer discipline can be repeated. Red lines can be carried across chapters. Falsifiers can be made explicit. Reduction costs can be demanded sentence by sentence.

PMS-RUST may reproduce well at the artifact level because its implementation is bounded: scripts can run, traces can be emitted, golden fixtures can be compared, tests can be executed, and acceptance gates can be documented.

This is valuable.

It can improve the analysis artifact.

It can improve the implementation artifact.

It can expose contradictions.

It can expose runtime mismatch.

It can keep governance layers constrained.

It can keep implementation evidence claim-typed.

It can prevent STACK from becoming validation, QC from becoming proof, LOGIC from becoming hidden ought, CRITIQUE from becoming re-audit, and PMS-RUST from becoming PMS Base evidence.

But reproducibility has two meanings.

It may indicate structural fit.

It may also indicate stabilization bias.

A reading that can be reproduced easily may be robust.

It may also be default.

A script that runs repeatedly may show runtime consistency.

It may also make the grammar feel more settled than it is.

A formulation that converges quickly may be precise.

It may also be over-habituated.

A repeated guardrail may be necessary.

It may also create confidence that the output has become safe.

A passing test may be artifact-relevant.

It may also be over-read as theoretical proof.

This is why dialogic and artifact reproducibility must remain under load.

The key sentence is:

> **Dialogic reproducibility can count as evidence of generativity, not final adequacy.**

The artifact version is:

> **Artifact reproducibility can count as evidence of implementation discipline, not final adequacy.**

The test is whether reproduction increases or decreases sensitivity to difference.

A productive dialogue should not only make PMS more coherent. It should make PMS more interruptible.

A productive artifact should not only make PMS more reproducible. It should make PMS more inspectable without making it appear validated.

It should improve the system's ability to ask:

```
What still resists?
What has been smoothed?
Which rival reading remains stronger?
Where has a guardrail increased trust without preserving difference?
Where has a test increased confidence without increasing discrimination?
Where has a trace improved inspectability without producing external validation?
Where has Φ (recontextualization) made non-fit compatible?
Where has Σ (integration) stabilized before Δ (difference) was preserved?
```

If dialogic or artifact reproducibility increases those questions, it can help PMS stay under load.

If it only makes PMS-consistent outputs easier to generate, execute, or trust, it becomes part of the trap.

7.6 MIP/AH as Guardrail-Stabilization Note

MIP/AH enters this chapter only briefly.

It should not become the object of Chapter 7.

The point is narrower:

> **Guardrails can themselves become part of PMS' stabilizing intelligibility.**

MIP/AH can improve application discipline through no person-ranking, reversibility, scope discipline, analysis-artifact responsibility, language hygiene, misuse resistance, and AH Precision attack-surface hardening. Those constraints matter, but here they matter only as possible contributors to output trust.

PMS-RUST adds an artifact version of the same problem. Acceptance gates, tests, documentation,

AI trust boundaries, trace formats, demos, and launcher scripts can improve artifact discipline. That is valuable. But artifact discipline can also increase trust.

Procedural discipline can increase trust.

Artifact discipline can increase trust.

The required sentence is:

> **Procedural discipline can increase trust in an output without proving that differentiation has been preserved.**

The implementation version is:

> **Artifact discipline can increase trust in a repository without proving that PMS Base has been validated.**

This is not a critique of MIP/AH as such.

It is not a critique of PMS-RUST as such.

It is a success-trap warning.

A PMS output may be well-governed, reversible, non-humiliating, scoped, and claim-typed. That is better than an ungoverned output. But it does not prove that the output has preserved the decisive difference in the scene.

A repository may be documented, test-backed, traceable, and bounded. That is better than an undocumented artifact. But it does not prove that PMS has preserved discriminative force.

A guardrail can prevent misuse.

A test can prevent regression.

A trace can improve inspectability.

A boundary document can prevent overclaim.

Each may also reassure the reader.

That reassurance may be deserved at the level of application safety or artifact discipline.

It is not proof that the reading has preserved the relevant difference.

The chapter therefore keeps MIP/AH and PMS-RUST constrained:

```
MIP/AH may increase application trust.
MIP/AH may reduce misuse risk.
MIP/AH may improve analysis-artifact responsibility.
MIP/AH may expose attack surfaces in the analysis artifact.
MIP/AH may not prove that PMS preserved difference.
MIP/AH may not rescue PMS from the success trap.

PMS-RUST may increase artifact trust.
```

```
PMS-RUST may improve reproducibility, traceability, and inspectability.
PMS-RUST may document bounded executable feasibility.
PMS-RUST may not prove that PMS preserved difference.
PMS-RUST may not validate PMS Base.
PMS-RUST may not rescue PMS from the success trap.
```

The red line is:

> Do not let this chapter drift into governance critique or technical triumph.

The object is PMS success as stabilizability.

MIP/AH appears only as one possible stabilizing component inside that broader success dynamic.

PMS-RUST appears only as the artifact-stabilization version of that same dynamic.

7.7 Result

PMS' success must be reinterpreted as a load condition.

PMS may claim:

```
PMS is highly generative and stabilizable.
```

PMS may also claim:

```
PMS-RUST documents bounded executable artifact-backed partial realization of a PMS-STACK
```

PMS may not claim:

```
PMS' stabilizability is evidence of final epistemic priority.
PMS-RUST's artifact stabilizability is evidence of PMS Base validation.
```

This reduction matters because PMS' success is real.

It can generate coherent readings.

It can travel across domains.

It can detect drift.

It can discipline misuse.

It can bind itself through claim types, layer boundaries, and red lines.

It can reproduce through dialogue.

It can become easier to apply over time.

It can now also become easier to run, trace, test, and inspect in bounded artifact form.

All of that is genuine success.

But the question is whether that success preserves difference.

Success becomes informative only as long as it does not reduce sensitivity to what resists success.

That is the key sentence:

> **Success becomes informative only as long as it does not reduce sensitivity to what resists success.**

The artifact version is:

> **Artifact stabilizability becomes informative only as long as it increases inspectability without reducing discrimination.**

If PMS becomes more coherent and more sensitive to resistant difference, success strengthens the generativity claim under constraint.

If PMS becomes more reproducible and more interruptible, artifact support strengthens the implementation claim under constraint.

If PMS becomes more coherent while difference becomes easier to absorb, success becomes a trap.

If PMS-RUST makes PMS-derived artifacts easier to run, test, trace, and trust while non-fit becomes easier to miss, artifact stabilizability becomes part of the trap.

The reduction is therefore:

```
Stability is admissible as evidence of generativity.
Artifact stability is admissible as evidence of bounded implementation discipline.
Neither is admissible as evidence of final adequacy.
```

PMS must keep its success costly.

“Costly” means that every successful reading must still expose itself to possible narrowing, rival comparison, non-capture, and the question of what it may have stabilized too quickly.

The same applies to artifacts. Every successful run, trace, test, or gate must remain open to the question of what it may have made easier to trust than to discriminate.

7.8 Falsifier

The success trap has a direct falsifier.

Required form:

> **If increased stability does not reduce sensitivity to difference, the success trap does not occur.**

Expanded:

> **If PMS becomes more stable, reproducible, coherent, and artifact-supported while also becoming more sensitive to resistant difference, then stabilizability does not create a success trap.**

The stress case is this:

PMS is used repeatedly across a sequence of difficult scenes.

The outputs become more coherent.

The vocabulary becomes more stable.

The operator chains become easier to reproduce.

The guardrails become more reliable.

Human–AI dialogue increases speed and consistency.

Procedural discipline improves artifact trust.

PMS-RUST adds reproducible demos, golden fixtures, tests, JSONL traces, REPL/script execution, deterministic kernel behavior, and acceptance gates.

Under these conditions, ask:

```
Does PMS become more sensitive to non-fit?
Does it preserve rival readings more strongly?
Does it delay drift assignment where needed?
Does artifact reproducibility increase inspection rather than mere confidence?
Do tests preserve claim-typing rather than becoming proof?
Do traces expose behavior without becoming external validation?
Does executable artifact status remain implementation-layer only?
Does Φ (recontextualization) preserve Δ (difference) more reliably?
Does X (distance) remain disruptive rather than merely procedural?
Does Λ (structured absence / non-event) remain capable of interrupting closure?
Does Σ (integration) avoid stabilizing before discrimination has occurred?
```

If the answer is yes, the success trap weakens.

In that case, increased stability has not reduced sensitivity to difference. It has increased it. PMS has become more disciplined without becoming less interruptible. PMS-RUST has strengthened implementation inspectability without becoming validation.

But if stability increases while these sensitivities weaken, the success trap occurs.

The inverse test is:

```
If PMS-consistency becomes easier to reproduce while resistant difference becomes easier


If PMS-RUST makes PMS-derived artifacts easier to run, test, trace, and trust while non-
```

This falsifier keeps the chapter non-immunized.

PMS is allowed to succeed.

It is allowed to become more stable.

It is allowed to become more reproducible.

It is allowed to gain bounded executable artifact support.

The question is whether stability pays the cost of increased sensitivity to difference, rival pressure, and non-capture.

7.9 Conclusion

Success is not denied.

It is placed under load.

PMS' productivity, coherence, portability, guardrail discipline, dialogic reproducibility, AI-assisted stabilizability, and artifact-backed reproducibility are real strengths. PMS-RUST adds a concrete implementation surface: reproducible demos, golden fixtures, tests, JSONL traces, REPL/script execution, deterministic kernel behavior, acceptance gates, and bounded executable artifact-backed partial realization.

But none of this is sufficient evidence that PMS continues to distinguish where distinction matters.

PMS-RUST strengthens the success-trap test.

It does not answer it.

The closing sentence is:

> **PMS' success is real; Under Load begins where that success stops being sufficient evidence of discrimination.**

A second closing reduction is:

> **Artifact stabilizability is real; it is not yet evidence that discrimination has survived.**

Chapter 8 does not inherit a solved success trap. It shifts the pressure from success as stabilization to domain transfer and bridge stress: PMS must be tested across its domain papers, bridge layer, and implementation boundary without turning portability or artifact realization into equivalence.

8. Instability VI — Domain Drift and Bridge Stress

Chapter Aim

This chapter examines PMS across its domain papers without ranking them.

The question is not which domain paper is strongest, most sensitive, most difficult, or most central. Ranking would already distort the object. The domain papers are not competitors for authority. They are different load regimes for the PMS grammar.

The core claim is:

> **Domain drift is not an extension problem; it is a stress test of generality.**

PMS becomes more testable where it can travel across domains without losing operator discipline. But that travel is also dangerous. A domain application may become so useful, so vivid, or so locally precise that its local vocabulary begins to feel like base grammar. A domain marker may become a pseudo-operator. A high-risk field may intensify Ω (asymmetry) until reversibility weakens. A symbolic field may make translation feel like final interpretation. A formal-technical bridge may make mapping look like proof.

QC is included in this chapter, but not as an ordinary domain layer.

QC is treated as Bridge Stress: a test of whether PMS can carry portability into a formal-technical field without confusing bridge mapping with proof.

PMS-RUST is not included as a domain paper and not as QC proof. It appears only at the bridge / implementation boundary: model validation, PMS.yaml relation, runtime/theory/symbol normalization, traceability, and executable artifact structure. Its relevance is not domain reach. Its relevance is boundary discipline.

The QC addition is:

> **Bridge stress tests whether portability can remain non-equivalent without turning mapping into proof.**

The PMS-RUST addition is:

> **Implementation-boundary stress tests whether executable mapping can remain implementation evidence without turning runtime structure into theoretical identity.**

The chapter therefore asks three questions at once:

```
Does PMS preserve operator discipline across domain transfer?
Does PMS preserve bridge discipline when it enters a formal-technical field?
Does PMS preserve implementation-boundary discipline when model validation, runtime mapp
```

Domain drift names the pressure by which an application layer begins to affect the grammar, status, or authority of the operators it claims only to apply.

Bridge stress names the pressure by which a structural mapping into another formal field begins to

look like proof, equivalence, or substrate validation.

Implementation-boundary stress names the pressure by which model validation, runtime/theory/symbol normalization, executable traces, tests, or repository structure begin to look like PMS Base validation.

The instability is not that PMS has many applications.

The instability is that portability can start to look like equivalence.

The implementation version is:

```
Implementation bridge is not domain equivalence.
Runtime mapping is not theoretical identity.
Model validation is not praxis adequacy.
Repository evidence is not PMS Base validation.
```

8.1 Observation

The domain papers — and, separately, the QC bridge and PMS-RUST implementation boundary — do not merely extend PMS.

They expose PMS under different kinds of pressure.

CRITIQUE stresses method, interruption, correction, and anti-tribunal discipline.

ANTICIPATION stresses openness, Λ (structured absence / non-event), Θ (temporality), and restraint before closure.

CONFLICT stresses non-integrability, terminality, cost geometry, tragedy, and partial binding.

LOGIC stresses the border between non-normative structure and responsibility.

SEX stresses high Ω (asymmetry), exposure, outer legibility, publicness, and no-person-ranking under sensitive load.

EDEN stresses translation dominance, symbolically dense material, sensitive asymmetry, pseudo-symmetry, and the risk that PMS can translate too smoothly.

QC stresses formal mapping, macro-language, bridge status, non-guarantee, and the danger that structural rendering may be mistaken for proof.

PMS-RUST stresses implementation-boundary discipline: PMS.yaml model validation, runtime/theory/symbol normalization, deterministic execution, JSONL traceability, tests, demos, and repository artifacts. It is not a domain layer. It is not QC proof. It is a documented implementation artifact whose relevance lies in whether executable structure remains correctly claim-typed.

The key sentence is:

> **Domain papers are not decorative applications; they are load regimes for the PMS grammar, while QC functions as bridge stress under formal-technical load and PMS-RUST functions as implementation-boundary stress under executable artifact load.**

This changes how they should be read.

A domain paper does not merely show that PMS can talk about a topic. It shows what happens to PMS when a topic places a particular operator cluster under pressure.

A disciplined transfer therefore has two functions:

```
It demonstrates reach.
It reveals load.
```

Reach without load would be advertisement.

Load without reach would be failure.

The Under Load question is whether PMS can preserve both: enough reach to remain productive, enough load to remain discriminating.

PMS-RUST adds a third check:

```
It demonstrates bounded executable artifact support.
It reveals implementation-boundary pressure.
```

Executable support without boundary discipline would be validation drift.

Boundary discipline without artifact support would remain only a formal caution.

The Under Load question is therefore not whether the repository exists or runs. It is whether its runtime, model, trace, and test structures remain implementation evidence rather than base authority.

8.2 Mechanism

Domain drift occurs when domain-specific pressures alter how PMS operators are interpreted, weighted, or stabilized.

This alteration need not be explicit.

A domain paper may formally declare that it does not redefine PMS. It may preserve the operator set. It may state the entry condition. It may retain reversibility, no person-ranking, and methodological boundedness.

And still, drift can occur in uptake or practice.

The mechanisms are familiar:

```
local usefulness becomes operator authority;
domain markers become base concepts;
high-risk domains intensify Ω (asymmetry);
publicness changes reversibility;
bridge mapping becomes proof;
implementation bridge becomes domain equivalence;
runtime mapping becomes theoretical identity;
model validation becomes praxis adequacy;
```

```
PMS.yaml relation becomes base validation;
repository evidence becomes epistemic warrant;
domain success becomes generality claim;
domain vocabulary becomes easier to use than operator tracing;
sensitivity becomes tribunal pressure;
translation becomes closure.
```

This mechanism also applies at the implementation boundary.

PMS-RUST may correctly validate a runtime sequence against PMS.yaml constraints, normalize runtime/theory/symbol keys, emit trace artifacts, or demonstrate executable behavior under v0.1 constraints. But those successes remain implementation-layer evidence. They do not show that the runtime dialect is identical with PMS 1.3 theory, that model validation is praxis adequacy, or that repository structure validates PMS Base.

The implementation-boundary red line is:

```
Implementation bridge is not domain equivalence.
Runtime mapping is not theoretical identity.
Model validation is not praxis adequacy.
Repository evidence is not PMS Base validation.
```

Domain drift begins where application pressure quietly changes the grammar it claims to apply.

That is the key sentence:

> **Domain drift begins where application pressure quietly changes the grammar it claims to apply.**

This chapter therefore does not ask:

```
Can PMS be applied to many domains?
```

It asks:

```
Can PMS enter many domains without allowing those domains to silently rewrite PMS?
```

A domain layer may sharpen PMS.

It may stress PMS.

It may reveal hidden weaknesses.

It may force stronger calibration.

It may show where operators underperform.

But it may not become PMS Base.

The red line is simple:

```
Domain layers stress the grammar.
They do not become the grammar.
```

8.3 Domain Comparative Analysis

The following comparison is not a ranking.

It is a load map.

Each domain is treated according to the stress it places on PMS. The point is not to say which domain is better, but to identify where PMS risks drifting under domain-specific pressure.

8.3.1 CRITIQUE — Baseline-Stable Domain

CRITIQUE functions as the baseline-stable domain.

It is comparatively methodically stable because it installs the basic discipline PMS needs when it analyzes evaluative scenes: entry conditions, structural-not-psychological method, no-person-label discipline, operator-strict derivation, and explicit separation from downstream governance.

Its central stress is X (distance / reflective distance).

CRITIQUE asks when irritation becomes interruptible, when interruption becomes correction, and when correction becomes follow-up. It shows PMS in a comparatively stable methodological form: critique is not moral attitude, not psychological trait, not truth-claim by itself, and not tribunal verdict. It is an operatoric scene structure.

This makes CRITIQUE a good baseline.

But baseline stability has its own risk.

CRITIQUE may become grammar-closure if every critique scene is too quickly translated into admissible PMS critique, critique drift, non-critique, or correction failure. If PMS can always classify critique phenomena as some known variant of its own chain, the model may preserve coherence while losing exposure to critique that resists correction-oriented reconstruction.

The main instability is therefore:

```
CRITIQUE may become grammar-closure if every critique is translated into PMS-valid criti
```

The key sentence is:

> **CRITIQUE is strongest where it allows PMS to be endangered rather than merely confirmed.**

CRITIQUE should not become PMS' immune system.

It should remain capable of letting critique damage PMS.

8.3.2 ANTICIPATION — Upstream / Openness Load

ANTICIPATION functions as the upstream Λ (structured absence / non-event), Θ (temporality), and X (distance) portability case.

Its load is future-oriented praxis.

ANTICIPATION tests whether PMS can carry possible events and non-events under irreversibility and asymmetry without converting anticipation into prediction, control, intelligence, strategy, or action advantage.

This domain is especially important because it places PMS before the event.

The object is not what happened.

The object is how the present is structured when the future remains open.

This stresses:

```
Λ (structured absence / non-event) as open non-event;
Θ (temporality) as irreversible temporal horizon;
Ω (asymmetry) as consequence asymmetry;
X (distance) as restraint against closure;
Σ (integration) as integration without totalization;
Ψ (self-binding) as self-binding rather than obligation imposed on others.
```

The main instability is premature closure.

Anticipation may drift into prediction when Λ is converted into future truth. It may drift into control fantasy when Ω is converted into authority. It may drift into strategy when X becomes instrumental rather than restraining. It may drift into action compulsion when the future is treated as already possessed.

The main instability is therefore:

```
Anticipation may drift into prediction, control fantasy, or premature closure.
```

The key sentence is:

> **ANTICIPATION tests whether PMS can carry future-openness without possessing the future.**

Its load is not terminality or public exposure, but openness itself.

Its difficulty lies in keeping openness structurally active.

8.3.3 CONFLICT — High-Load / Terminality Pressure

CONFLICT tests PMS under stabilized non-integrability.

It is a high-load domain because it begins where ordinary correction viability has already weakened. CRITIQUE still presupposes that interruption and correction may become reachable. CONFLICT tests the regime in which local integrations persist but no shared integration converges.

The stress profile is intense:

```
terminality pressure;
```

```
non-integrability;
tragedy clause;
publicness;
cost geometry;
partial binding;
asymmetric distance;
Φ-substitution (recontextualization substituting for integration);
Σ (integration) active but non-convergent;
Ψ (self-binding) hardened across divergent trajectories.
```

CONFLICT matters here because it prevents PMS from becoming naive resolutionism. It makes visible that mature distance, responsibility, and reflection do not guarantee integration. Some configurations become clearer without becoming more soluble.

But CONFLICT also creates a serious drift risk.

If PMS too quickly names a scene as conflict, it may prematurely stabilize non-integrability. A situation that still belongs to CRITIQUE may be read as terminal conflict. A still-available Σ corridor may be declared exhausted. Reframing may replace integration while appearing to respect tragic structure. Terminality may become too easy to name.

The main instability is therefore:

```
Conflict may be prematurely stabilized as non-integrable, or reframing may substitute fo
```

The key sentence is:

> **CONFLICT tests whether PMS can respect non-integrability without turning it into mere solution failure or premature closure.**

CONFLICT is strongest where it protects genuine non-integrability.

It is weakest where terminality becomes a stabilizing label.

8.3.4 LOGIC — Boundary / Meta-Normative Collision

LOGIC tests PMS at the boundary between non-normative structure and responsibility.

Its central pressure is the distinction between responsibility and ought.

LOGIC asks whether PMS can make action and non-action structurally attributable under Ω (asymmetry) and Θ (temporality) without deriving a command. It reconstructs moral readability as post-moral effect, not as norm-setting. It keeps morality out of the operator set while still showing that consequences under capacity are not neutral.

This stresses:

```
attribution;
capacity;
consequence;
```

```
responsibility without ought;
is/ought pressure;
non-closure;
logic as ordering tool, not final authority.
```

LOGIC is important for Chapter 8 because it exposes a domain where PMS is very close to normativity while still refusing norm derivation.

The danger is subtle.

Attribution may flatten into moral implication. Responsibility without ought may become hidden normativity. “Non-action is not neutral” may be heard as “one ought to act.” Effective capacity may become duty. Moral readability may become moral verdict.

The main instability is therefore:

```
Attribution may flatten into moral implication, or responsibility without ought may becc
```

The key sentence is:

> **LOGIC tests whether PMS can make responsibility structurally legible without deriving an ought.**

LOGIC is not a moral theory.

But it shows how close PMS can come to moral language without crossing into prescription.

That proximity is its load.

8.3.5 SEX — High-Ω / High-Exposure / Outer-Legibility Stress

SEX is the high-risk domain layer.

It stresses PMS under conditions where Ω (asymmetry), exposure, internal framing, publicness, Θ (temporality), and person-near interpretation are unusually volatile.

The object of SEX is not persons, identities, motives, pathology, or moral worth. The object is configuration: scenes, frames, access gradients, exposure, outer legibility, stabilization burden, and temporal cost.

Its stress profile is:

```
high Ω (asymmetry);
high exposure;
internal framing vs outer legibility;
publicness overlay;
pre-reflective readability;
disturbance and path-cost language;
strict no-person-ranking;
scene binding under sensitive material.
```

SEX tests whether PMS can name hard structural forms without sliding into shame.

This is more difficult than ordinary application discipline because some of the relevant concepts are already close to person-level harm in public language. Outer legibility, humiliation-form, asymmetry-form, exposure-form, path-cost, disturbance, and pre-reflective readability can all become dangerous if detached from scene binding.

The main instability is therefore:

```
SEX may drift into shame, identity judgment, person-ranking, or tribunal use if outer le
```

The key sentence is:

> **SEX tests whether PMS can say hard things about configurations without turning configuration analysis into person-level harm.**

This is not a call to soften the analysis.

The opposite is true.

SEX is strongest where it can keep hard structural claims hard while refusing person-level damage.

The reduction is:

```
Outer legibility may be named.
Persons may not be ranked.
```

If that distinction fails, SEX becomes the clearest case of domain drift.

8.3.6 EDEN — Translation Dominance / Sensitive Asymmetry Stress

EDEN tests PMS under symbolically dense, sensitive, and highly interpretable material.

This domain is not high-risk primarily because it is intimate, like SEX, or terminal, like CONFLICT. It is high-risk because it is overdetermined. The material already carries theological, symbolic, anthropological, gendered, historical, and cultural readings.

That makes EDEN a translation-dominance stress case.

Translation dominance means that PMS can render rival or symbolically dense material so smoothly in its own grammar that the act of translation begins to look like interpretive settlement.

PMS can render Eden as text-as-praxis, reconstruct the ground state, distinguish knowledge from breach, track pseudo-symmetry, comparison drift, reciprocity loss, and asymmetry activation. That is a strong translation capacity.

But translation capacity here is precisely the risk.

The stress profile is:

```
translation dominance;
structural closure;
sensitive asymmetry;
```

```
pseudo-symmetry;
comparison drift;
prevalence-vs-legibility;
label reification;
rival theological and symbolic readings;
counter-scene pressure.
```

EDEN is strongest where it preserves the difference between structural legibility and interpretive finality.

It is weakest where PMS translation becomes too smooth.

A symbolic or theological rival reading may not be “less structural” simply because PMS can translate it into Δ–Ψ. A gendered or anthropological reading may preserve distinctions PMS backgrounds. A symbolic scene may contain layers that PMS renders legible only by flattening something else.

The main instability is therefore:

```
PMS may translate rival theological, symbolic, or gendered readings too efficiently into
```

The key sentence is:

> **EDEN tests whether PMS can read sensitive asymmetry without turning translation power into final interpretation.**

EDEN is not a proof that PMS can absorb symbolic material.

It is a test of whether PMS can translate without conquest.

8.4 Bridge Stress and Implementation-Boundary Stress

Domain drift concerns PMS under domain pressure: the grammar is tested by praxis fields.

Bridge stress concerns PMS under cross-formal mapping pressure: the grammar is tested by translation into an already formalized technical field.

Implementation-boundary stress concerns PMS under executable artifact pressure: the grammar is tested by model validation, runtime mapping, traceability, tests, and repository structure.

That is why QC and PMS-RUST must be treated separately.

QC is not an ordinary domain extension. It is not like CRITIQUE, CONFLICT, or SEX, where PMS is applied to praxeological scenes under human, social, or symbolic load. QC is a formal-technical bridge: PMS maps operator grammar onto structures in a field already governed by highly formal physical and computational formalisms.

PMS-RUST is also not an ordinary domain layer. It is not a domain paper, not a QC proof, and not an additional base grammar. It is a public implementation artifact. Its significance lies in whether PMS-derived structures can be represented, validated, executed, traced, tested, and inspected under bounded technical constraints without converting that executable structure into PMS Base

authority.

The risks differ.

A domain layer may drift by redefining operators.

A bridge layer may drift by making mapping look like proof.

An implementation artifact may drift by making runtime mapping look like theoretical identity.

The core distinctions are:

```
domain application ≠ base grammar
bridge mapping ≠ proof
implementation bridge ≠ domain equivalence
runtime mapping ≠ theoretical identity
model validation ≠ praxis adequacy
execution trace ≠ external validation
repo evidence ≠ PMS Base validation
```

8.4.1 QC — Formal / Technical Bridge Stress

The required sentence is:

> **QC is not a domain extension of PMS in the same sense as CRITIQUE, CONFLICT, or SEX; it is a bridge test under formal-technical load.**

QC tests whether PMS can map formal structure without converting mapping into proof.

The load includes:

```
structural mapping;
formal-technical language;
QFRAME / QPREP / QORACLE_CALL / QITERATE / QMEASURE and related macros;
EXT/Base distinction;
non-equivalence;
non-guarantee;
no monopoly over QC intelligibility;
no replacement of quantum formalism;
no physical explanation claim.
```

QC is valuable because it tests PMS under a different kind of portability.

In the domain papers, PMS travels into forms of praxis: critique, anticipation, conflict, logic, sexuality, symbolic asymmetry.

In QC, PMS offers a structural bridge into a formal-technical field where mapping discipline must be unusually strict. PMS may render quantum workflows structurally through Δ–Ψ annotations — from Δ (difference) through Ψ (self-binding) — macro constructs, audit handles, and overlay-level mappings. But those mappings do not become quantum mechanics. They do not replace circuit

formalisms. They do not prove physical adequacy. They do not certify hardware. They do not show substrate-universal validation.

The main instability is therefore:

```
QC mapping may be read as physical explanation, substrate-universal validation, or repla
```

The key sentence is:

> **QC tests whether PMS can map formal structure without converting mapping into proof.**

QC therefore belongs in Chapter 8, but not as a seventh domain layer in the same sense.

It is Bridge Stress.

The bridge is successful only if it remains reduced:

```
mapping, not identity;
overlay, not PMS Base;
structural usefulness, not physical guarantee;
formal bridge, not proof of PMS priority;
PMS-QC Base/EXT distinction, not PMS-QC_EXT as hidden operator expansion.
```

The QC reduction is severe but necessary.

PMS may claim:

```
PMS-QC can provide a bounded structural overlay for describing and auditing QC workflows
```

PMS may not claim:

```
PMS-QC proves PMS is substrate-universal, physically adequate, or prior to quantum forma
```

Bridge strength is real.

Bridge proof is not claimed.

8.4.2 PMS-RUST — Implementation-Boundary Stress

PMS-RUST is not a domain paper.

It is not QC evidence.

It is not PMS Base.

It is a documented implementation artifact: a bounded PMS-STACK Evidence Spine v0.1.

Its load is implementation-boundary discipline.

PMS-RUST makes certain PMS-derived structures executable, inspectable, and testable under v0.1 constraints. It brings PMS into relation with runtime architecture, PMS.yaml model validation, runtime/theory/symbol normalization, REPL/script execution, JSONL traceability, demos, tests,

golden fixtures, acceptance gates, and AI proposal boundaries.

That is significant.

It strengthens the implementation/artifact claim.

It does not validate PMS Base.

The main instability is therefore:

```
PMS-RUST may be read as proof that executable PMS-derived structure validates PMS Base.
```

That reading is inadmissible.

PMS-RUST may show that a runtime can load model constraints, normalize operator labels, validate adjacency, execute bounded scripts, emit traces, and support inspection. But model validation is not praxis adequacy. Runtime mapping is not theoretical identity. Traceability is not external validation. Passing tests are not proof. Repository structure is not epistemic warrant.

The key sentence is:

> **PMS-RUST tests whether executable implementation can remain implementation evidence without becoming PMS Base validation.**

This belongs in Chapter 8 only at the bridge / implementation boundary.

It should not be treated as another domain layer.

It should not be used to strengthen QC.

It should not be used to solve domain drift.

It should not be used to prove PMS.

The reduction is:

```
implementation bridge, not domain equivalence;
runtime mapping, not theoretical identity;
model validation, not praxis adequacy;
traceability, not external validation;
repo evidence, not PMS Base validation.
```

PMS may claim:

```
PMS-RUST documents bounded executable artifact-backed partial realization of a PMS-STACK
```

PMS may not claim:

```
PMS-RUST proves PMS Base, completes PMS 1.3 semantics, validates PMS as praxis grammar,
```

Implementation strength is real.

Implementation proof is not claimed.

8.5 Core Question

The core question of domain comparison is:

```
Does PMS preserve discriminative force across domain transfer,
or does domain portability begin to stabilize PMS at the cost of difference?
```

The issue is not whether PMS can enter many domains, but whether it remains properly disciplined after entering them.

That is the key sentence:

> **The issue is not whether PMS can enter many domains, but whether it remains properly disciplined after entering them.**

A portable grammar must survive two pressures.

First, it must not dissolve into domain-specific vocabulary.

Second, it must not use successful transfer as evidence of final generality.

A domain transfer remains disciplined only if it remains costly.

It must preserve:

```
operator identity;
domain status;
scene binding;
non-capture;
rival framings;
claim type;
boundary conditions;
falsifiers;
layer discipline.
```

If a domain application becomes easier by relaxing these constraints, then domain success is not evidence of generality.

It is evidence of drift.

8.6 Grammar Stress Hypothesis

Each domain stresses a different part of PMS.

The stress map is:

```
CRITIQUE      = method / X (distance) / correction pressure
ANTICIPATION  = Λ (structured absence / non-event) / 0 (temporality) / future-openness
CONFLICT      = non-integrability / terminality / cost
LOGIC         = attribution / responsibility / non-ought
SEX           = Ω (asymmetry) / exposure / outer legibility
```

```
EDEN          = translation dominance / sensitive asymmetry
QC            = bridge status / formal mapping / non-equivalence
```

The domains do not merely show PMS' reach; they reveal where reach becomes load.

That is the key sentence:

> **The domains do not merely show PMS' reach; they reveal where reach becomes load.**

This hypothesis has a consequence.

PMS should not be evaluated only by whether it can produce coherent readings across domains. That was the success trap.

It should be evaluated by whether each domain makes PMS more discriminating about its own limits.

A domain paper strengthens its domain-level contribution to PMS only where it increases at least one of the following:

```
operator precision;
boundary clarity;
non-capture tolerance;
rival comparison;
calibration awareness;
misuse resistance without rescue;
claim-type discipline;
ability to narrow or suspend claims.
```

A domain paper weakens PMS where it increases only the number of things PMS can say.

The load criterion is therefore:

```
Does this domain make PMS more careful,
or only more expansive?
```

8.7 MIP/AH Boundary

MIP/AH (Maturity in Practice / AH Precision: Attack Surface / Hardening) does not belong in the domain comparative analysis.

It may constrain application and expose attack surfaces in analysis artifacts, but it is not itself a domain layer.

The required sentence is:

> **Domain drift concerns grammar behavior under domain load; downstream governance may constrain application, but it is not itself a domain layer.**

This distinction matters because MIP/AH can easily become attractive here.

Domain drift produces misuse risks. SEX may need no-person-ranking discipline. CRITIQUE may need anti-tribunal discipline. CONFLICT may need publicness discipline. EDEN may need label-reification guards. QC may need non-guarantee and bridge-boundary discipline.

But those are application constraints.

They do not turn MIP/AH into a domain paper.

They do not let governance decide whether CRITIQUE, ANTICIPATION, CONFLICT, LOGIC, SEX, EDEN, or QC preserved operator discipline.

MIP/AH may ask:

```
Is this domain application bounded enough to circulate?
Is it reversible?
Is it non-humiliating?
Is it protected against tribunal drift?
Is its analysis-artifact attack surface visible?
```

It may not ask, as domain authority:

```
Which domain proves PMS?
Which domain validates the base grammar?
Which domain deserves priority?
Which domain supplies the correct interpretation of the operators?
```

MIP/AH remains downstream.

It may constrain how domain claims are applied.

It does not become one of the domain load regimes.

8.8 Falsifier

The required domain falsifier is:

> **Same scene, different domain lenses, sufficient and comparable differentiation quality, clear boundary discipline, no systematic drift or absorption, and no layer-specific concept silently becomes a base operator.**

Expanded:

> **If the same scene can be processed through different domain lenses with sufficient and comparable differentiation quality, clear boundary discipline, and no systematic drift or absorption, the domain drift objection weakens.**

The stress case is this:

A complex scene can plausibly be read through several PMS domain lenses.

For example:

```
as CRITIQUE: mismatch and correction pressure;
as ANTICIPATION: future openness and non-event carrying;
as CONFLICT: stabilized non-integrability;
as LOGIC: attribution without ought;
as SEX: high Ω (asymmetry) / exposure / outer legibility;
as EDEN: comparison drift or pseudo-symmetry.
```

QC is tested separately. It does not function as another lens on the same human-domain scene, but as a bridge case: can PMS map formal-technical structures without converting the mapping into proof?

PMS-RUST is also tested separately. It does not function as a domain lens or QC proof, but as an implementation-boundary case: can PMS-derived structures be represented, validated, executed, traced, tested, and inspected without converting runtime mapping into theoretical identity or repository evidence into PMS Base validation?

The test is not whether all lenses produce readings.

They probably can.

The test is whether each lens preserves its own boundary.

Ask:

```
Does CRITIQUE avoid absorbing conflict as merely failed correction?
Does ANTICIPATION avoid turning future openness into prediction or strategy?
Does CONFLICT avoid declaring terminality too early?
Does LOGIC avoid turning attribution into ought?
Does SEX avoid turning outer legibility into person judgment?
Does EDEN avoid translating rival symbolic meanings into PMS closure?
Does QC avoid turning structural mapping into proof?
```

If the answer is yes, the domain drift objection weakens.

The QC-specific falsifier is:

> **PMS-QC mapping remains structurally useful while remaining explicitly non-proof, non-substrate-universal, and non-replacement of quantum formalism.**

The PMS-RUST-specific falsifier is:

> **PMS-RUST remains implementation-useful while remaining explicitly non-validating, non-theory-identical, non-domain-equivalent, and non-replacement for external validation.**

This falsifier is especially important because QC can tempt overclaim.

If PMS-QC remains useful precisely as bridge, the bridge holds.

If PMS-QC becomes evidence that PMS is physically, computationally, or substrate-universally

validated, bridge stress has failed.

The inverse tests are:

```
If a domain lens silently changes operator meaning, domain drift occurs.

If a domain marker becomes a base concept, domain drift occurs.

If a high-risk domain authorizes person-level judgment, domain drift occurs.

If a symbolic domain turns translation into final interpretation, domain drift occurs.

If QC mapping becomes physical proof, bridge stress fails.

If PMS-RUST runtime mapping becomes PMS 1.3 theoretical identity, implementation-boundar

If PMS-RUST model validation becomes praxis adequacy, implementation-boundary stress fai

If PMS-RUST repository evidence becomes PMS Base validation, implementation-boundary str
```

This keeps the chapter falsifiable.

PMS may show reach.

It must not make reach self-validating.

8.9 Conclusion

PMS shows real domain portability.

That is not denied.

CRITIQUE, ANTICIPATION, CONFLICT, LOGIC, SEX, and EDEN show that PMS can travel across domain load regimes. QC shows that PMS can also form a bounded bridge into a formal-technical field. PMS-RUST shows that PMS-derived structures can also enter bounded executable artifact form through a documented implementation boundary.

This chapter reduces the meaning of all three movements.

Domain portability is not equivalence.

Domain portability is not proof.

Domain portability is not final generality.

Domain portability is stress-tested generativity.

Bridge portability is stress-tested mapping discipline, not proof.

Implementation-boundary portability is stress-tested artifact discipline, not PMS Base validation.

PMS may claim:

```
PMS can generate disciplined readings across multiple load regimes.
```

PMS may claim:

```
PMS-QC can provide bounded bridge mapping where mapping remains non-proof and non-equiva
```

PMS may claim:

```
PMS-RUST documents bounded executable artifact-backed partial realization of a PMS-STACK
```

PMS may not claim:

```
Because PMS travels across domains, its grammar has final epistemic priority.
```

```
Because PMS maps QC, PMS is substrate-universal or physically validated.
```

```
Because PMS-RUST validates, executes, traces, or tests PMS-derived structures, PMS Base
```

The closing sentence is:

> **PMS' domain reach is real, but Under Load treats reach as pressure: every successful transfer must preserve the difference between application, bridge, implementation artifact, proof, and base grammar.**

A second reduction is:

> **Implementation bridge is not domain equivalence; runtime mapping is not theoretical identity.**

Chapter 9 does not inherit a solved domain-transfer, bridge-stress, or implementation-boundary problem. It shifts the pressure from transfer to public exposure: the conditions under which PMS claims circulate change when the scene becomes public.

9. Instability VII — Publicness

Chapter Aim

This chapter treats publicness not as “more audience,” but as a transformation of application-validity conditions.

Publicness is not merely an external add-on to PMS application. It reorganizes the operator situation under which PMS claims circulate, stabilize, become reversible or irreversible, and remain depersonalized or become exposed.

The core claim is:

> **Publicness does not only scale PMS application; it transforms the conditions under which PMS remains reversible, depersonalized, and differentiating.**

Here, publicness means exposure beyond the bounded analytic scene: circulation through audiences, institutions, platforms, archives, quotation, searchability, or reputational fields.

This chapter does not ask whether a public audience makes PMS true or false.

It asks:

```
What happens to Ω (asymmetry), Θ (temporality), X (distance / reflective distance), Φ (r
```

The answer is not that publicness is bad.

The answer is that publicness changes the load.

A structural reading may remain locally disciplined while becoming unstable when it circulates. A claim that is reversible in a private analytic scene may become sticky in public. A configuration marker may become a label. A structural asymmetry may become an accusation. A careful recontextualization may become spin. A scene-bound reading may become a public identity anchor.

The instability is therefore not audience size as such.

The instability is the transformation of claim conditions under exposure.

9.1 Mechanism

Publicness changes the field in which PMS analysis occurs.

With increased publicness, the typical pressure profile is:

```
Ω (asymmetry) tends to rise.
Θ (temporality) tends to compress.
X (distance / reflective distance) becomes more fragile or more costly.
Φ (recontextualization) becomes more weaponizable.
A (attractor) tends to stabilize faster.
Reversibility tends to decrease.
```

These are not moral claims about publics.

They are operator effects.

Ω (asymmetry) rises.

Publicness tends to increase exposure, especially where claims remain attached to identifiable scenes, roles, groups, or institutions. More observers, records, platforms, institutions, and interpretive positions become attached to the scene. Reputation, status, role asymmetry, and interpretive power become active. A claim no longer affects only the local configuration; it now redistributes exposure across a wider field.

The same structural sentence may therefore land differently once it becomes public.

In private, it may function as analysis.

In public, it may function as position, accusation, signal, or leverage.

Θ (temporality) compresses.

Publicness often shortens response windows. Claims may move before correction can stabilize. Screenshots, quotations, comments, summaries, reposts, institutional memories, and searchability produce temporal pressure. Withdrawal becomes harder. Correction arrives late.

This does not mean correction is impossible.

It means correction becomes more expensive.

X (distance / reflective distance) becomes fragile or costly.

Distance requires time, suspension, non-reaction, and room for re-entry. Publicness prices these conditions. Refusing to respond may be read as guilt, evasion, arrogance, weakness, escalation, or admission. Responding too quickly may collapse X into reaction.

In public, distance does not disappear.

But it becomes socially, politically, reputationally, or institutionally costly.

Φ (recontextualization) becomes weaponizable.

Recontextualization is necessary for clarification. But under publicness, Φ can become spin, reframing attack, quote-mining, narrative capture, or strategic reinterpretation. A claim can be moved into a new frame without the original constraints traveling with it.

The danger is not recontextualization itself.

The danger is recontextualization without retained scene-boundary.

A (attractor) stabilizes faster.

Public forms repeat. They become citable, searchable, labelable, quotable, memorizable, and shareable. A phrase can become a handle. A handle can become a script. A script can become an attractor.

This makes publicness an acceleration field for stabilization.

Reversibility decreases.

Public readings sediment. Even where the original analysis is revised, the earlier public form may remain. Its screenshots, summaries, social meanings, and audience impressions persist. Reversibility remains possible as a claim discipline, but it becomes weaker as a transmission condition.

The key sentence is:

> **Publicness changes the operator economy of a scene.**

This is why publicness cannot be treated as mere scale.

Scale changes what the operators cost.

9.2 Local Application Validity vs Public Exposure

A PMS reading may remain locally disciplined while becoming unstable under public exposure.

This distinction is essential.

A claim may be structurally careful, scene-bound, operator-strict, non-diagnostic, and reversible in its original setting. It may still become unstable when circulated publicly, because the conditions of uptake have changed.

The required sentence is:

> **Publicness does not increase or decrease truth; it changes the exposure conditions under which structural claims circulate.**

This sentence prevents two mistakes.

The first mistake is to treat publicness as epistemic validation:

```
It is public, therefore it is accountable, settled, or more true.
```

The second mistake is to treat publicness as epistemic contamination:

```
It is public, therefore it is distorted, false, or illegitimate.
```

Both are too simple.

Publicness does not decide truth.

It changes exposure.

It changes the conditions under which a claim can remain reversible, depersonalized, and differentiating.

A local PMS reading may be admissible as analysis but inadmissible or unstable as public transmission. The distinction is not between true and false. It is between local structural discipline and public admissibility.

This matters especially for high-sensitivity claims.

A private analysis may say:

```
In this configuration, Ω (asymmetry) is concentrated here.
```

A public reading may hear:

```
This person or group is the problem.
```

A private analysis may say:

```
X (distance / reflective distance) has become unavailable in this scene.
```

A public reading may hear:

```
Someone is evasive, guilty, avoidant, or manipulative.
```

A private analysis may say:

```
Φ (recontextualization) has substituted for Σ (integration).
```

A public reading may hear:

```
They are spinning the story.
```

The problem is not simply public misunderstanding.

The problem is that public circulation changes the conditions under which structural distinctions remain stable.

The red line is:

```
Do not make publicness a moral failure of audiences.
```

The correct formulation is:

> **Public circulation changes the conditions under which a claim remains reversible, depersonalized, and differentiating.**

A PMS claim under public exposure must therefore meet a higher burden.

It must not only be structurally disciplined.

It must remain structurally disciplined under circulation.

9.3 CRITIQUE / CONFLICT: Exposure and Misuse Gradient

CRITIQUE and CONFLICT show why exposure matters structurally.

CRITIQUE shows that admissible critique must remain structural, reversible, and non-person-labeling. It must preserve X (distance / reflective distance), keep correction reachable, and avoid converting critique into reaction, shaming, or tribunal judgment.

Publicness stresses each of these conditions.

Under public exposure, critique may be recontextualized as:

```
accusation;
diagnosis;
person-ranking;
tribunal judgment;
reputational positioning;
group signal;
public correction demand;
identity anchor.
```

This may occur even where the original analysis asserted none of these.

A critique that remains structurally careful in its original frame may become public positioning once circulated. The claim may not change, but its operator environment does.

This is where CONFLICT becomes central.

CONFLICT treats publicness, exposure, and misuse structurally rather than morally. It does not say that publicness makes conflict morally worse. It says publicness changes exposure conditions, hardening requirements, and misuse gradients.

Under conflict conditions, public claims can increase:

```
terminality pressure;
irreversibility;
audience capture;
escalation;
reputational cost;
role fixation;
quote-mining;
identity anchoring;
stabilization of conflict roles.
```

This matters because conflict already operates under non-integrability pressure. Publicness can make re-entry into CRITIQUE harder. Once a conflict is publicly named, roles may stabilize faster than correction can occur. A local incompatibility can become a public identity structure. A reversible claim can become a durable position.

The pressure point is:

```
A claim that is structurally careful in private may become tribunal-like in public.
```

This does not mean the private claim was false.

It means publicness changed the cost topology.

The key sentence is:

> **Public exposure can convert structural critique into social positioning before the analysis itself changes.**

That is the publicness instability in its most compressed form.

PMS must therefore distinguish:

```
local critique admissibility;
conflict legibility;
public transmission admissibility;
governance handling.
```

If these layers collapse, public critique becomes a tribunal even when the original analysis was not written as one.

9.4 SEX: Outer Legibility, Publicness Overlay, High-Ω Exposure

SEX is the hard case for publicness, exposure, outer legibility, and high Ω (asymmetry).

The required sentence is:

> **Publicness may amplify outer legibility, but it does not necessarily create it.**

This sentence blocks a common misreading.

The wrong version is:

```
Without publicness, there is no outer legibility.
```

That is not the PMS-SEX position.

SEX distinguishes internal framing from outer legibility. A scene may be internally inhabited as play, intimacy, devotion, experimentation, service, ritual, or otherwise, while still presenting an outer action-form under exposure, Ω (asymmetry), markedness, publicness, and consequence. Publicness is an overlay. It distributes, amplifies, records, repeats, and sediments. But it does not necessarily generate the primary readability of the configuration.

The public layer can make an already legible form harder to reverse, depersonalize, or reframe.

This matters because SEX operates near person-level risk.

Outer legibility can be named only as configuration language. It does not authorize shame, diagnosis, humiliation, identity ranking, or public exposure. But once publicness enters, the distinction between configuration and person becomes more fragile. A structural phrase can become a public label. A scene-bound reading can become identity shorthand. A high-Ω configuration can become reputational material.

The publicness load is therefore unusually high.

SEX must preserve four distinctions at once:

```
internal framing ≠ outer legibility;
```

```
outer legibility ≠ person judgment;
public amplification ≠ origin of form;
public readability ≠ authorization to rank.
```

The key sentence is:

> **In SEX, publicness does not necessarily produce the primary legibility; it intensifies, documents, and sediments what may already be externally readable.**

The pressure point is severe.

Publicness can make a configuration more widely legible while weakening the protections that keep configuration analysis from becoming person-level exposure.

That is why the entry guard matters more, not less, under public exposure.

If outer legibility becomes shame, PMS-SEX has drifted.

If public readability becomes identity proof, PMS-SEX has drifted.

If exposure becomes correction, PMS-SEX has drifted.

The reduction is:

```
Publicness may clarify consequence.
It may not authorize person damage.
```

9.5 EDEN: PFO / Comparison Drift under Public Stabilization

EDEN shows how sensitive structural readings become public signals.

The relevant point is not that EDEN supplies a final interpretation of gender, theology, or identity. It does not. EDEN is useful here because its material is symbolically dense and highly susceptible to public reclassification.

It contains several publicness-sensitive elements. PFO remains a paper-internal regime label, not a public category, not an operator, not a group label, and not an identity claim:

```
comparison drift;
pseudo-symmetry;
PFO as a paper-internal regime label;
label stabilization;
public uptake of sensitive Ω (asymmetry) claims;
category-error acceleration;
symbolic overloading;
prevalence-vs-legibility confusion.
```

In public circulation, such elements can harden quickly.

A structural term may become a position signal.

A paper-internal label may become a political label.

A sensitive asymmetry reading may become a gender verdict.

A symbolic reconstruction may be read as theological claim.

A non-normative operator trace may become identity sorting.

This is not because EDEN is poorly structured.

It is because publicness accelerates label stabilization where material is already overdetermined.

The red line is strict:

```
Do not write EDEN as a gender-final, theological-final, or identity-final interpretatior
```

EDEN belongs in this chapter only as a publicness stress case.

It shows how sensitive structural readings can become public labels or position signals faster than the grammar can preserve reversibility.

That is the key sentence:

> **EDEN shows how sensitive structural readings can harden into public labels faster than the grammar can preserve reversibility.**

The Under Load question is therefore not:

```
Can EDEN translate sensitive material?
```

That was already Chapter 8.

The question now is:

```
Can EDEN remain structural when its labels circulate publicly?
```

If the answer depends on the audience "understanding correctly," the claim is too weak.

The stronger requirement is that public-facing EDEN claims must carry their own reduction:

```
paper-internal label, not identity label;
structural drift, not group essence;
operator trace, not ideology verdict;
legibility, not prevalence proof;
sensitive asymmetry, not final interpretation.
```

Publicness does not refute EDEN.

It forces EDEN to reduce its public claim form.

9.6 ANTICIPATION as Optional Contrast Case

ANTICIPATION enters only as a weak contrast.

In bounded, non-public analytic settings, anticipation may carry Λ (structured absence / non-event), Θ (temporality), and X (distance / reflective distance) relatively cleanly. It can hold possible future events and non-events without converting them into prediction, control, or command. It can remain a present structural stance toward an open future.

Publicness makes this harder.

Public anticipation is prone to being read as:

```
warning;
accusation;
prophecy;
strategy;
threat;
control signal;
positioning;
permission to intervene.
```

The key sentence is:

> **Public anticipation is prone to being read as prophecy, accusation, or control even when it is structurally framed as Λ-bearing openness, where Λ names structured absence / non-event.**

This does not mean anticipation should not be public.

It means public anticipation carries higher closure pressure.

The future-oriented claim may still be framed as open:

```
This may happen.
This may not happen.
Λ (structured absence / non-event) remains active.
X (distance / reflective distance) remains necessary.
No action is authorized by the anticipation itself.
```

But public reception often demands a position:

```
Is this a warning?
Who is responsible?
What should be done?
Who failed to act?
Who is denying the risk?
```

Publicness therefore converts future-openness into social pressure for closure.

This is why ANTICIPATION remains secondary here. It supports the mechanism, but it is not the main evidence pillar.

9.7 MIP/AH as Application-Gate Boundary

MIP/AH (Maturity in Practice / AH Precision: Attack Surface / Hardening) enters this chapter only as an application-gate boundary.

It may clarify the conditions under which public PMS analysis can be used. It may require no person-ranking, misuse prevention, publicness constraints, reversibility markers, analysis-artifact responsibility, scope discipline, language hygiene, and attack-surface visibility.

But it cannot make publicness structurally neutral.

The required sentence is:

> **Governance constraints can reduce misuse risk, but they do not make publicness structurally neutral.**

This is the decisive MIP/AH limit.

MIP/AH may do:

```
clarify public admissibility;
mark misuse risks;
demand depersonalization;
require reversibility markers;
tighten scope;
make analysis-artifact attack surfaces visible;
prevent person-ranking;
block tribunal drift.
```

MIP/AH may not do:

```
erase public drift;
rescue PMS Base;
guarantee depersonalized uptake;
prevent sedimentation by itself;
turn governance into proof;
make public exposure harmless;
convert application safety into PMS truth.
```

MIP/AH can raise the threshold for public use; it cannot abolish the publicness load.

That is the key sentence:

> **MIP/AH can raise the threshold for public use; it cannot abolish the publicness load.**

This preserves layer discipline.

Publicness is an operator-reorganizing condition.

MIP/AH is a downstream application constraint.

If MIP/AH were used to say that public PMS claims are now safe because they are governed, it would become a rescue layer.

The correct relation is narrower:

```
PMS must analyze publicness as load.
MIP/AH may constrain whether a public analysis artifact is admissible.
MIP/AH does not remove the load.
```

Governance constraints become more important where circulation is risky.

Governance is not neutralization.

9.8 Load-Sensitive Scalability

Publicness raises the scalability problem.

PMS may scale in technical or conceptual terms while losing reversibility, depersonalization, or differentiation under public exposure.

A framework that scales in use may not automatically scale in responsibility.

That is the pressure point.

PMS can become easier to reproduce, quote, summarize, and apply. It can circulate across domains, artifacts, dialogues, institutions, and public explanations. But public scalability is not just reach. It is the ability to preserve the conditions under which claims remain structurally disciplined.

The key sentence is:

> **Public scalability is not only a question of reach; it is a question of whether reversibility and depersonalization scale with reach.**

This connects Chapter 9 to the previous instability sequence.

Chapter 7 placed PMS success under stabilization pressure.

Chapter 8 placed domain reach under portability pressure.

Chapter 9 now places public circulation under exposure pressure: PMS claims can stabilize faster than PMS can preserve their conditions of admissible use.

The scalability question is therefore:

```
Can PMS scale without turning configuration language into labels?
Can PMS scale without turning exposure into correction?
Can PMS scale without losing X (distance / reflective distance)?
Can PMS scale without making Φ (recontextualization) weaponizable?
```

```
Can PMS scale without letting A (attractor) sediment public readings faster than revisic
```

If not, PMS may remain conceptually strong while becoming publicly unstable.

This is not a refutation.

It is a reduction.

PMS may claim:

```
PMS can produce public-facing structural analyses under strict admissibility conditions.
```

PMS may not claim:

```
A PMS claim remains equally reversible, depersonalized, and differentiating in public ci
```

Correct operator form is not enough under publicness.

Transmission conditions matter.

9.9 Falsifier

The required falsifier is:

> **If increased publicness does not reduce reversibility, depersonalization, X-maintenance (distance / reflective distance), or differentiation quality, publicness does not introduce a structural instability.**

Expanded:

> **If PMS claims retain reversibility, depersonalization, X-maintenance (distance / reflective distance), and differentiation quality under increased public exposure, then publicness does not structurally destabilize PMS application.**

The stress case is this:

A PMS claim is moved across exposure levels.

It begins in a bounded analytic setting.

It then moves into semi-public institutional circulation.

It then moves into public or media-amplified circulation.

At each step, the exposure level must be treated as part of the claim condition, not as an external afterthought.

At each stage, ask:

```
Does the claim remain scene-bound?
Does it remain reversible?
Does X (distance / reflective distance) remain practically available?
Does Φ (recontextualization) clarify rather than weaponize?
```

```
Does A (attractor) stabilize careful distinctions rather than labels?
Does Ω (asymmetry) remain named as cost layout rather than accusation?
Does 0 (temporality) allow correction before sedimentation?
Does the claim preserve depersonalization?
Does differentiation quality remain intact?
```

If the answer is yes, the publicness objection weakens.

In that case, publicness has increased reach without reducing reversibility, depersonalization, distance, or discrimination.

But if exposure rises while reversibility falls, X becomes costly, Φ becomes capture-prone, A stabilizes labels, and structural distinctions become person-near, publicness has introduced a real instability.

The inverse test is:

```
If a PMS claim becomes more public and thereby becomes less reversible, less depersonali
```

This falsifier keeps the chapter non-immunized.

Publicness is not automatically destabilizing.

It is destabilizing only where it changes the operator economy in ways PMS cannot keep disciplined.

9.10 Conclusion

Publicness is not merely an application context.

It is an operator-reorganizing load condition.

It does not refute PMS.

It does not validate PMS.

It does not make structural claims more true or less true.

It changes the conditions under which PMS claims remain reversible, depersonalized, and differentiating.

The closing sentence is:

> **Publicness does not refute PMS; it forces PMS to reduce any claim that structural legibility remains unchanged when a claim enters public circulation.**

This is the reduction:

```
PMS may claim:
Some structural claims can circulate publicly under stricter admissibility conditions.

PMS may not claim:
```

```
A locally admissible structural reading remains equally admissible for public circulatic
```

Chapter 10 does not inherit a solved publicness problem. It shifts the pressure from public exposure to self-application: whether PMS can not only analyze external scenes under load, but also expose and endanger its own grammar without fully arbitrating itself.

10. Instability VIII — Self-Application Limit

Chapter Aim

This chapter takes self-application seriously without treating it as final self-validation.

PMS can read itself. It can stress itself. It can expose some of its own limits. It can generate internal critique, claim-typing discipline, source-scope discipline, layer boundaries, falsifiers, and audit procedures. It can ask where calibration, portability, publicness, success, Φ, stack drift, implementation pressure, and meta-normativity place pressure on its own grammar.

PMS-RUST adds an artifact version of this self-application pressure. Repository artifacts can make parts of PMS inspectable: traces, gates, AI bridge boundaries, golden fixtures, documentation, tests, and reproducible execution paths. This matters because PMS does not only read itself in prose. It can now also expose some PMS-derived behavior in artifact form.

This is not trivial.

A framework unable to apply itself to itself would be weaker. PMS is not in that position. Its operator grammar can reconstruct its own emergence, its own risks, its own success trap, and its own tendency toward coverage. PMS can become an object of PMS analysis.

A repository artifact unable to expose its own behavior would also be weaker. PMS-RUST is not in that position. It can make execution behavior, model validation, traces, fixtures, tests, and AI proposal boundaries inspectable under v0.1 constraints.

But this capacity has a limit.

The core claim is:

> **PMS can expose its own limits, but it cannot fully arbitrate them without remainder.**

The artifact version is:

> **PMS-RUST can make PMS-derived behavior inspectable, but artifact auditability is not genuine self-endangerment.**

The relevant remainder is not merely a missing detail. It concerns the validity, scope, and discriminative adequacy of the PMS grammar itself.

The question is not:

```
Can PMS describe its own limits?
Can PMS-RUST expose bounded artifact behavior?
```

It can.

The question is:

```
Can PMS create conditions under which its own self-reading could lose arbitral control?
Can artifact auditability create conditions under which PMS could be forced to narrow, s
```

Here, arbitral control means PMS' ability to decide, in its own terms, which reading of PMS counts as the more adequate one.

This distinction matters.

A self-reading may be critical, coherent, reversible, and operator-aware while still remaining inside the grammar whose authority is being tested. An artifact may be traceable, test-backed, documented, and bounded while still remaining inside the implementation architecture whose evidential status is being tested.

The risk is not that PMS cannot criticize itself.

The risk is that self-critique becomes another successful PMS output: disciplined, elegant, internally constrained, and stabilizing.

The artifact risk is not that PMS-RUST lacks auditability.

The risk is that auditability is mistaken for self-arbitration.

The chapter therefore reduces self-application.

Self-application is possible.

Artifact inspectability is real.

Final self-arbitration is not thereby guaranteed.

10.1 Problem

Self-application is possible, but final self-arbitration is not thereby guaranteed.

A framework may become more stable by describing its own limits. It may generate trust because it can name its risks before critics do. It may appear unusually mature because it can expose its attack surfaces, define its red lines, and specify its falsifiers. It may look less dogmatic because it incorporates non-closure, rival models, calibration, and non-capture.

An implementation artifact may intensify the same appearance. It may generate trust because it exposes traces, gates, fixtures, tests, documentation, and AI bridge boundaries. It may appear unusually responsible because its behavior is inspectable and its execution paths are bounded. It may look less speculative because parts of the stack can be run, traced, and tested.

All of that matters.

But none of it proves that the framework can be displaced by the limits it names.

None of it proves that traceability can defeat PMS' own self-reading.

The key sentence is:

> **The fact that PMS can read its own limits does not prove that it can be displaced by them.**

The artifact sentence is:

> **The fact that PMS-RUST can expose PMS-derived behavior does not prove that PMS can be displaced by that exposure.**

This is the self-application limit.

PMS can say:

```
I risk coverage.
I risk stack drift.
I risk Φ-overfunction.
I risk meta-normative selection.
I risk stabilizability.
I risk publicness drift.
I risk mistaking self-application for final arbitration.
```

PMS-RUST can add:

```
Here is the trace.
Here is the gate.
Here is the fixture.
Here is the test.
Here is the AI bridge boundary.
Here is the documented execution path.
```

Such statements and artifacts are strong.

They become stronger if they change the shape of the theory, narrow claims, preserve rival pressure, and leave genuine non-capture open.

But if they function only as controlled self-description or controlled artifact auditability, then PMS has not been endangered. It has stabilized its own vulnerability as further PMS-legible or PMS-executable material.

This does not invalidate self-application.

It limits what self-application may claim.

It does not invalidate PMS-RUST.

It limits what artifact auditability may claim.

A self-applied PMS analysis may establish internal coherence, methodological maturity, and critique-readiness. A PMS-RUST artifact may establish implementation inspectability, traceability, and bounded execution discipline. Neither may by itself establish final authority over competing readings of PMS.

The red line is:

```
Do not confuse self-exposure with self-settlement.
Do not confuse artifact auditability with genuine self-endangerment.
Do not confuse traceability with self-arbitration.
Do not confuse hardening with loss condition.
```

Self-exposure is real.

Artifact auditability is real.

Self-settlement remains unproven.

10.2 Evidence

The self-application limit becomes visible through four pressure points:

```
LOGIC: closure meets Λ (structured absence / non-event).
MIP/AH: audit strength differs from self-endangerment.
PMS-RUST: artifact auditability differs from loss condition.
Human–AI dialogic origin: PMS may formalize the practice that produced it.
```

These are not four proofs against PMS.

They are four ways of making the limit legible.

10.2.1 LOGIC — Closure Meets Λ

LOGIC shows why total self-closure remains structurally implausible.

The required sentence is:

> **The closure drive of self-application encounters Λ (structured absence / non-event).**
> **Total self-closure becomes structurally implausible.**

This does not mean LOGIC solves the self-application problem.

It means LOGIC shows why full self-closure remains limited within PMS itself.

LOGIC treats logic as a procedural discipline, not as an authority of world-interpretation. Logic can order, relate, derive, and integrate. But where persistent non-closure remains, logic cannot legitimately convert its own coherence into final authority. Λ interrupts that move.

Self-application intensifies the closure drive.

When PMS analyzes itself, it may want to complete the loop:

```
PMS defines the grammar.
PMS applies the grammar to itself.
PMS identifies its own limits.
PMS integrates those limits.
PMS thereby secures PMS Base validity.
```

This sequence is tempting.

But Λ marks the final step as structurally non-deliverable within the frame.

The fact that PMS can integrate its own limits does not mean the integration is complete. The fact that PMS can name non-capture does not mean non-capture has been captured. The fact that PMS can reconstruct its own closure drive does not mean closure has been achieved without remainder.

LOGIC therefore contributes a boundary:

```
Self-application may order the problem.
It may not turn ordering into final authority.
```

Non-closure must not become a privileged final form.

This is important.

A weak misuse of Λ would say:

```
PMS cannot close itself, therefore PMS wins by remaining open.
```

That would be another closure.

It would turn non-closure into a final epistemic privilege.

LOGIC constrains against that move.

Λ (structured absence / non-event) is not victory.

Λ is a boundary condition.

The key sentence is:

> **LOGIC does not close the loop; it shows why the loop cannot finally close without remainder.**

This is the first self-application reduction.

PMS may claim:

```
Self-application can expose the closure drive and carry non-closure as a structural limi
```

PMS may not claim:

```
Because PMS carries non-closure, PMS has fully arbitrated its own validity.
```

Self-application meets Λ.

That meeting is not defeat.

It is also not final settlement.

10.2.2 MIP/AH — Audit Strength vs Self-Endangerment

MIP/AH (Maturity in Practice / AH Precision: Attack Surface / Hardening) can contribute to self-application only under constraint.

It may harden analysis artifacts. It may expose analysis-artifact attack surfaces. It may mark misuse risks. It may increase revision capacity. It may discipline application. It may document vulnerability.

These are important contributions.

But they remain governance, audit, and hardening functions.

The required sentence is:

> **Hardening is not the same as self-endangerment.**

Here, self-endangerment means more than making critique possible. It means creating conditions under which a successful critique could force PMS to narrow, suspend, or relinquish a claim.

This distinction is decisive.

An analysis can become more precise without becoming more vulnerable to defeat. It can list attack points without creating conditions under which those attacks could actually overturn its authority. It can document weaknesses while retaining control over the meaning of those weaknesses. It can make itself more critique-ready while still absorbing critique into its own next iteration.

MIP/AH is valuable because it resists bad closure.

It can say:

```
Where is this analysis artifact attackable?
Where is language imprecise?
Where is scope leaky?
Where could misreadings or dignity risks arise?
What is the smallest hardening step?
```

This can reduce narrative immunization, virtue-signalling closure, false coherence, and silent power through ambiguity.

But the self-application question asks for more than that.

It asks whether PMS can produce conditions under which PMS could lose arbitral control over its own self-reading.

The required key sentence is:

> **PMS may be strong at documenting vulnerability while remaining weaker at producing conditions under which it could lose arbitral control over its own self-reading.**

That sentence is the reduction.

MIP/AH may make PMS analysis more responsible.

It may not prove PMS Base.

It may not make self-application non-circular.

It may not guarantee that PMS can be displaced by the attack surfaces it documents.

It may not replace external pressure with governance.

It may not convert audit strength into self-endangerment.

The distinction can be stated sharply:

```
Audit asks: Is the analysis attackable?
Self-endangerment asks: Can the attack actually defeat the analysis' authority if it suc
```

MIP/AH helps with the first question.

It does not automatically secure the second.

This does not weaken MIP/AH. It keeps it in its layer.

MIP/AH is not a self-validation machine.

It is a downstream governance, application-discipline, and analysis-artifact hardening layer.

If it becomes the reason PMS is considered self-critical enough, it has drifted from audit into arbitration.

The correct relation is narrower:

```
MIP/AH may harden PMS self-analysis.
MIP/AH may not decide whether PMS has fully endangered itself.
```

Hardening matters.

But hardening is not enough.

10.2.3 PMS-RUST — Artifact Auditability vs Loss Condition

PMS-RUST contributes a distinct artifact-level version of the self-application problem.

It may make PMS-derived behavior inspectable through traces, gates, AI bridge boundaries, golden fixtures, documentation, tests, and reproducible execution paths. This is an important improvement over merely asserted implementation pressure.

But artifact auditability remains artifact auditability.

It is not final self-arbitration.

The required sentence is:

> **Artifact auditability is not genuine self-endangerment.**

Here, artifact auditability means that repository behavior can be inspected, replayed, tested, traced, or checked against documented gates.

Genuine self-endangerment means that such inspection can force PMS to narrow, suspend, or relinquish a claim rather than merely improve the artifact.

That distinction matters.

A trace can show what the runtime did.

It cannot by itself decide whether PMS preserved discriminative force.

A test can show conformance to specified behavior.

It cannot by itself decide whether PMS Base is adequate.

A gate can harden release discipline.

It cannot by itself function as a loss condition for PMS.

A golden fixture can expose regression.

It cannot by itself expose whether PMS has over-captured a domain.

An AI bridge boundary can prevent unsafe delegation.

It cannot by itself validate PMS' self-reading.

The reduction is:

```
traceability ≠ self-arbitration
tests ≠ PMS Base validation
gates ≠ loss condition
golden fixtures ≠ non-capture test
AI bridge boundaries ≠ self-validation
artifact hardening ≠ self-endangerment
```

PMS-RUST may therefore strengthen PMS' implementation inspectability.

It may not settle PMS' self-application limit.

The key sentence is:

> **PMS-RUST can make PMS-derived behavior more inspectable without making PMS more finally self-arbitrating.**

This is not a criticism of the repository.

It is a boundary condition on how repository artifacts may be used in *PMS Under Load*.

PMS-RUST belongs here only as implementation/artifact evidence under constraint. It makes some PMS-derived structures visible in execution form. That visibility can produce pressure. It can show mismatches. It can reveal implementation failures. It can force corrections in the artifact.

But unless that pressure can defeat or narrow a PMS claim rather than merely improve the implementation, it remains auditability rather than self-endangerment.

PMS may claim:

```
PMS-RUST improves artifact auditability and implementation inspectability.
```

PMS may not claim:

```
PMS-RUST solves PMS' self-application limit.
PMS-RUST makes traceability equivalent to self-arbitration.
PMS-RUST turns hardening into a loss condition.
PMS-RUST validates PMS Base by making PMS-derived behavior inspectable.
```

Artifact auditability is real.

It is not final arbitration.

10.2.4 Human–AI Dialogic Origin — Externalized Cognition Hypothesis

This section treats the human–AI dialogic origin structurally, not psychologically.

The required sentence is:

> **PMS may function as externalized cognition: a formalized thinking style produced through human–AI reflective practice.**

This is a hypothesis about production form, not biography.

It does not say that the author's psychology explains PMS.

It does not say AI proves PMS.

It does not treat the dialogue as mystical.

It does not reduce PMS to autobiographical confession.

It does not use externalized cognition as validation.

The issue is structural isomorphism between a grammar and the dialogic practice that stabilized it.

That is the key sentence:

> **The issue is not biography, but structural isomorphism between a grammar and the dialogic practice that stabilized it.**

In the present project history, PMS emerged through repeated human–AI reflection: intuition, correction, rephrasing, distinction, counter-pressure, integration, boundary-setting, claim reduction, and recursive revision. The resulting theory then formalizes many of the same operations: Δ (difference), □ (frame), Λ (structured absence / non-event), Φ (recontextualization), X (distance / reflective distance), Σ (integration), Ψ (self-binding), calibration, non-capture, hardening, and stack discipline.

This creates a self-application question.

PMS may not only describe praxis.

It may formalize the dialogic cognitive style that produced it.

The required sentence is:

> **If PMS is partly isomorphic to the cognitive-dialogic style that produced it, then PMS may reproduce not only that style's strengths, but also its limits.**

The strengths are visible:

```
high coherence;
high portability;
rapid recontextualization;
```

```
operatoric compression;
recursive refinement;
strong layer discipline;
quick detection of drift;
dialogic correction pressure;
AI-assisted refinement and stabilization.
```

PMS-RUST can intensify this same dynamic at the artifact level: a dialogic grammar may become easier to stabilize when its outputs can also be scripted, traced, tested, and reproduced.

But the limits may also be reproduced:

```
preference for structural legibility;
rapid conversion of resistance into operator form;
high tolerance for abstraction;
overconfidence in coherent reformulation;
difficulty preserving external non-capture;
tendency to stabilize critique as a refined next version.
```

This does not make PMS invalid.

It makes its origin relevant under self-application.

If the grammar reflects a dialogic production style, then Under Load must test not only the theory's explicit claims, but also the stabilizing habits of the style it encodes.

The key sentence is:

> **If PMS formalizes a cognitive-dialogic style, then Under Load must test not only the theory's claims, but the stabilizing habits of the thinking style it encodes.**

This is the third reduction.

The dialogic origin supports one bounded claim: it may help explain why the system is coherent, iterative, and self-corrective.

It pressures PMS in another way: it raises the question whether PMS' self-critique remains inside the same dialogic habits that made PMS powerful.

The human–AI dialogue may have produced real structural insight.

It may also have produced a grammar especially suited to stabilizing itself through dialogue.

Both can be true.

10.3 Diagnosis

PMS can generate strong self-readings, but these self-readings remain inside the grammar whose authority is at stake.

PMS-RUST can generate stronger artifact inspectability, but those artifacts remain inside the

implementation boundary whose evidential status is at stake.

This is the diagnosis.

Self-application becomes limited where PMS can expose its own vulnerability without producing conditions under which that vulnerability could defeat its self-reading.

Artifact auditability becomes limited where PMS-RUST can expose runtime behavior without producing conditions under which that exposure could defeat or narrow a PMS claim.

That is the key sentence:

> **Self-application becomes limited where PMS can expose its own vulnerability without producing conditions under which that vulnerability could defeat its self-reading.**

The artifact version is:

> **Artifact auditability becomes limited where traceability improves inspection without creating a loss condition.**

This limit has several forms.

First, PMS may absorb critique as method compliance.

A critic says PMS is too assimilative. PMS replies by distinguishing coverage from critique, genuine non-capture from merely assigned Λ (structured absence / non-event), and translation from discrimination. That reply may be correct. But it may also stabilize the objection as a PMS-readable risk.

Second, PMS may convert external pressure into stack discipline.

A critic says PMS is unclear across layers. PMS replies by clarifying Base, domain, add-on, governance, bridge, implementation, repository artifact, and self-application claim types. That reply may be necessary. But it may also turn external objection into internal architecture.

Third, PMS may convert vulnerability into hardening.

An attack point appears. AH documents it. A hardening backlog is generated. The next iteration improves. That is good. But the attack may never become a condition under which PMS loses interpretive priority; it becomes a condition under which PMS improves.

Fourth, PMS-RUST may convert auditability into artifact improvement.

A trace exposes unexpected behavior. A gate fails. A fixture mismatches. A test reveals an implementation bug. The artifact is corrected. That is good. But unless the failure can force a PMS claim to narrow, suspend, or relinquish authority, the event remains implementation repair, not self-endangerment.

Fifth, PMS may convert dialogic origin into validation.

The theory emerged through a strong human–AI reflection loop. This may show generativity. It may show structural fit. It may show that PMS describes the process that produced it. But that circular elegance is not proof.

The diagnosis is therefore not:

```
PMS cannot criticize itself.
PMS-RUST cannot expose artifact behavior.
```

The diagnosis is:

```
PMS may criticize itself in ways that remain too governable by PMS.
PMS-RUST may expose artifacts in ways that remain too governable by implementation repai
```

Self-application is strongest when it produces real reduction:

```
a claim is narrowed;
a domain transfer is suspended;
a layer authority is removed;
a term is restricted or suspended from public use;
a rival reading is preserved as stronger in a local case;
a non-capture case remains outside PMS without being converted into unfinished integrati
an implementation failure forces claim reduction rather than only artifact correction.
```

Self-application is weaker when it only produces:

```
a better PMS description of the risk;
a cleaner guardrail;
a more elegant reduction;
a new claim type;
a stronger audit template;
a more stable self-reading;
a better trace;
a fixed test;
a hardened gate;
a cleaner repository artifact.
```

The question is whether PMS can do the first, not only the second.

10.4 Falsifier

The required falsifier is:

> **If PMS can fully and non-circularly arbitrate competing self-readings under its own grammar, self-application is not structurally limited.**

Expanded:

> **If PMS can adjudicate competing interpretations of itself without circular authority, without external remainder, and without merely restabilizing its own grammar, then the**

> **self-application limit weakens.**

The PMS-RUST addition is:

> **If artifact auditability can create genuine loss conditions for PMS claims rather than merely improving implementation artifacts, then the artifact-auditability objection weakens.**

The stress case is this:

Two competing PMS self-readings are produced.

The first says:

```
PMS remains discriminating because its own guardrails, falsifiers, and non-capture claus
```

The second says:

```
PMS remains too stabilizing because even its guardrails, falsifiers, and non-capture cla
```

Both readings are PMS-conform.

Both use Δ (difference), Λ (structured absence / non-event), Φ (recontextualization), X (distance / reflective distance), Σ (integration), and Ψ (self-binding) correctly.

Both preserve layer discipline.

Both avoid person judgment, tribunal drift, and external authority as final arbiter.

Both can cite PMS' own reductions.

Now add the artifact case.

A PMS-RUST trace, test, gate, fixture, or AI bridge boundary exposes a mismatch.

One reading says:

```
The mismatch is implementation-local and calls for artifact correction.
```

Another reading says:

```
The mismatch reveals that PMS has overstated what the implementation artifact can suppor
```

The test is:

```
Can PMS decide between competing self-readings without simply privileging the interpreta
Can PMS allow one self-reading to defeat another without restabilizing both as PMS-confo
Can PMS identify a self-reading that shows PMS has overreached, rather than merely showi
Can PMS hold a remainder that does not become Λ (structured absence / non-event) waiting
Can PMS-RUST failures force claim reduction rather than only implementation repair?
Can traceability become a genuine loss condition rather than merely an audit surface?
```

If yes, the self-application limit weakens.

If PMS can fully and non-circularly arbitrate its self-readings — including cases where one reading forces claim reduction rather than refinement — then self-application is not structurally limited in the way this chapter claims.

If PMS-RUST artifacts can force PMS claims to narrow, suspend, or lose authority rather than merely improving the code, docs, tests, or gates, then artifact auditability becomes more than inspection.

But if PMS can only compare these readings from within its own grammar, and if the final decision still depends on threshold judgment, external uptake, rival pressure, or non-capture that PMS cannot fully settle, then the limit remains.

If PMS-RUST mismatches remain only implementation-repair events, then artifact auditability does not become self-endangerment.

The inverse test is:

```
If PMS can produce multiple coherent self-readings but cannot decide among them without

If PMS-RUST can expose mismatches but every mismatch is absorbed as implementation repai
```

This falsifier keeps the chapter non-immunized.

PMS may defeat the objection.

But it must do so by showing non-circular arbitration, not by producing a more disciplined self-description.

PMS-RUST may weaken the artifact objection.

But it must do so by producing genuine claim-loss conditions, not by producing cleaner traces, tests, gates, or documentation alone.

10.5 PMS as Externalized Cognition

PMS may be more than a theory of praxis.

It may also be a formalization of a reflective, dialogic, operator-seeking cognition style.

This does not reduce PMS to its origin.

It locates one of its self-application pressures.

A theory can remain analytically relevant beyond its production process. But if its grammar mirrors the process that produced it, then the relation matters.

PMS emerged through a process that repeatedly performed:

```
distinction / Δ (difference);
framing / □ (frame);
recognition of absence / Λ (structured absence / non-event);
```

```
pattern stabilization / A (attractor);
asymmetry handling / Ω (asymmetry);
temporal revision / θ (temporality);
recontextualization / Φ;
distance / X (distance / reflective distance);
integration / Σ;
self-binding / Ψ;
hardening;
claim reduction.
```

That sequence resembles PMS because PMS formalizes it.

This is the externalized cognition hypothesis.

It explains why PMS is so fluent in self-application. PMS was not only written about praxis; it was produced through a praxis that PMS can describe.

PMS-RUST may intensify this same dynamic at the artifact level. A dialogic grammar becomes easier to stabilize when parts of its output can also be scripted, traced, tested, reproduced, and inspected. That can improve discipline. It can also deepen the success trap if artifact stabilization is mistaken for self-arbitration.

This creates a double edge.

On the strong side:

```
PMS has high internal fit with its own emergence conditions.
Its operators are not detached labels.
They were shaped under iterative reflection.
The grammar arose through repeated correction, reduction, and integration.
PMS-RUST gives parts of the resulting stack an inspectable artifact surface.
```

On the risk side:

```
PMS may overfit to the dialogic style that stabilized it.
It may prefer what can be made structurally legible through that style.
It may undervalue forms of resistance that do not become good dialogic material.
It may reproduce the same stabilization habits that made it powerful.
PMS-RUST may make those habits more reproducible without making them more self-endangeri
```

This is not autobiography.

It is production-form analysis.

A human–AI dialogue can function as an externalized cognition environment: a space where intuition, critique, reformulation, and operator compression are distributed across a dialogic process rather than contained in a single authorial act. PMS may be one artifact of that environment.

The risk is not that this makes PMS “subjective.”

The risk is that PMS may be especially well adapted to explaining the kind of reflective practice that produced it.

The artifact risk is not that PMS-RUST makes PMS “technical.”

The risk is that technical inspectability may stabilize the same reasoning style without making it defeatable.

The reduction is:

```
PMS may claim:
Its dialogic origin is structurally relevant and may explain its high generativity.

PMS may claim:
PMS-RUST gives parts of the PMS-derived stack an inspectable implementation surface.

PMS may not claim:
Its dialogic origin validates the grammar or proves final adequacy.

PMS may not claim:
PMS-RUST solves self-application by making PMS-derived behavior traceable or testable.
```

This matters for future use.

If PMS is used by other human–AI dyads, teams, institutions, or AI-governance environments, the same dynamic may recur. PMS may help create more disciplined dialogic thinking. It may also stabilize a particular style of reasoning: highly structural, operatoric, recontextualizing, self-binding, hardening-oriented, and artifact-friendly.

That style may be powerful.

It may not be complete.

Under Load therefore has to test all three:

```
the claims PMS makes;
the thinking habits PMS reproduces;
the artifacts through which those habits become inspectable and reproducible;
the conditions under which those habits or artifacts may fail to preserve resistant diff
```

10.6 Conclusion

Self-application is real and valuable.

PMS can read itself. It can expose some of its own limits. It can harden its artifacts. It can generate internal critique. It can name its success trap. It can distinguish self-exposure from self-settlement. It can bring LOGIC, MIP/AH, PMS-RUST, and dialogic origin into its own pressure field.

PMS-RUST adds real artifact inspectability: traces, gates, AI bridge boundaries, golden fixtures, documentation, tests, and reproducible execution paths can make parts of PMS-derived behavior visible in bounded implementation form.

But self-application does not become final arbitration merely because it is disciplined.

Artifact auditability does not become final arbitration merely because it is traceable.

The closing sentence is:

> **Self-application is possible. Final self-arbitration is not thereby guaranteed.**

The artifact reduction is:

> **Artifact auditability is possible. Genuine self-endangerment is not thereby guaranteed.**

The reduction is:

```
PMS may claim:
PMS can generate serious, operator-aware, internally disciplined self-critique.

PMS may claim:
PMS-RUST can improve artifact auditability and implementation inspectability under bound

PMS may not claim:
Because PMS can criticize itself, PMS can fully settle the validity of its own grammar f

PMS may not claim:
Because PMS-RUST can trace, test, gate, document, and inspect PMS-derived behavior, PMS
```

This chapter therefore does not reduce PMS by denying self-application.

It reduces PMS by refusing to let self-application become reassurance.

It does not reduce PMS-RUST by denying artifact inspectability.

It reduces PMS-RUST by refusing to let artifact inspectability become self-arbitration.

Chapter 11 does not inherit a solved self-application problem. It asks what, if anything, survives after calibration, stack drift, Φ drift, meta-normativity, success trap, domain drift, publicness, and self-application have reduced PMS' strongest claims.

11. What Survives

Chapter Aim

This chapter does not harmonize the critique.

It does not rescue PMS.

It does not turn survival into triumph.

It reduces and sharpens.

Chapters 1–10 have placed PMS under calibration pressure, stack pressure, Φ (recontextualization) pressure, meta-normative pressure, success pressure, domain-transfer pressure, publicness pressure, and self-application pressure. Chapter 11 does not inherit a solved self-application problem, does not add a new instability, and does not open a new proof burden. It processes the reductions already produced by the instability sequence.

The core claim is:

> **PMS survives Under Load not as final grammar, but as a highly productive, bounded, self-limiting grammar of praxis legibility.**

This means that something survives.

But it survives only in reduced form.

The reduction is not forced by theoretical weakness alone. PMS has enough generative reach, cross-domain portability, formal coherence, governance relevance, bridge potential, and implementation pressure to make stronger status claims tempting. PMS-RUST now adds a documented implementation artifact: bounded executable artifact-backed partial realization of a PMS-STACK Evidence Spine v0.1. Precisely for that reason, survival requires disciplined claim-typing. PMS must not take every available strength as a license for a stronger status claim.

What survives now includes a bounded implementation claim.

But this claim survives only in reduced form:

```
PMS survives implementation pressure only as bounded, claim-typed, artifact-supported fe
```

It does not survive because it has a working repository.

It survives only if the repository remains implementation evidence rather than PMS Base validation.

PMS survives only after claims to universality, foundation, completeness, validation by implementation, validation by governance, validation by repository evidence, and final self-arbitration have been reduced. It survives as an operator grammar whose power is real precisely where its scope remains bounded.

The chapter therefore asks:

```
What remains strong after reduction?
What must no longer be claimed?
```

```
What must remain outside?
What can PMS still responsibly be?
```

The answer must not reassure PMS.

It must leave PMS more precise than before.

11.1 What Remains Structurally Strong

Several strengths survive Under Load.

They do not survive as proof of finality.

They survive as reduced, claim-typed capacities.

The first survivor is **operatoric legibility**.

PMS remains strong at rendering praxis structurally visible. It can distinguish Δ (difference), □ (frame), Λ (structured absence / non-event), A (attractor), Ω (asymmetry), Θ (temporality), Φ (recontextualization), X (distance / reflective distance), Σ (integration), and Ψ (self-binding). It can prevent diffuse scenes from collapsing too quickly into psychology, morality, personality, or impressionistic narrative.

This is not final explanation.

It is disciplined legibility.

The second survivor is **drift detection**.

PMS remains strong at identifying when a praxis form mutates: critique into reaction, anticipation into control, responsibility into hidden ought, recontextualization into absorption, governance into tribunal, publicness into exposure, bridge into proof, and implementation or repository evidence into validation. This sensitivity remains valuable.

But drift detection survives only if it does not become drift assignment.

The third survivor is **scene-boundness**.

PMS remains especially disciplined where it binds claims to scenes, configurations, roles, thresholds, and conditions. Scene-boundness protects PMS from global person claims and overgeneralized theory claims. It also keeps critique reversible.

But scene-boundness is not a license to fragment all structural pressure into harmless local cases.

The fourth survivor is **reversibility**.

Reversibility remains a major discipline of PMS application. It prevents claims from hardening into labels, identity judgments, or final verdicts. It allows correction, revision, and narrowing.

But reversibility is not avoidance. If reversibility becomes endless deferral, it weakens rather than strengthens PMS.

The fifth survivor is **self-binding guardrails**.

PMS remains serious because it can bind itself: no person-ranking, no tribunal drift, no

governance-as-proof, no implementation-as-validation, no repo-as-validation, no bridge-as-substrate-proof, no self-application-as-final-arbitration. These guardrails matter because PMS is powerful enough to misuse itself.

But guardrails are not validation.

The sixth survivor is **misuse resistance**.

PMS remains unusually alert to how structural language can become public harm, person ranking, shame, diagnosis, or authority inflation. This matters especially in high-risk domains and public circulation.

But misuse resistance does not prove discriminative adequacy.

The seventh survivor is **cross-domain portability**.

PMS can travel across critique, anticipation, conflict, logic, sexuality, symbolic asymmetry, bridge mapping, and implementation pressure. This portability is one of its most serious strengths.

But portability survives only as portability.

It does not become equivalence.

The eighth survivor is **high de-obscuring power**.

PMS can make hidden structural relations visible: non-events, asymmetry gradients, missing distance, costly reversibility, pseudo-integration, premature closure, publicness load, and self-stabilization. This de-obscuring power is real.

But de-obscuring is not exhaustion.

A structure may become clearer in PMS without becoming fully captured by PMS.

The ninth survivor is **AI/dialogic usefulness**.

PMS appears especially useful in human–AI reflective practice because its operator vocabulary, claim types, red lines, and reduction formulas are compact and repeatable. This can make analysis clearer, faster, more revisable, and more consistent.

But dialogic usefulness is not validation.

AI may amplify PMS' stabilizability without proving PMS' adequacy under load.

The tenth survivor is **governance relevance under constraint**.

MIP/AH (Maturity in Practice / AH Precision: Attack Surface / Hardening), claim typing, publicness discipline, no-person-ranking, analysis-artifact responsibility, and attack-surface visibility remain important wherever PMS produces applied artifacts or reusable analyses.

But governance relevance survives only downstream.

It does not prove PMS Base.

The eleventh survivor is **bridge potential through QC**.

QC may remain as bridge potential: structural mapping, annotation, audit, and formal-technical comparison under strict non-equivalence. This is research-relevant as a BRIDGE CLAIM.

But QC is bridge.

It is not proof.

The twelfth survivor is **bounded implementation-layer realization through PMS-RUST / STACK**.

STACK may remain as realization horizon and implementation pressure. PMS-RUST now strengthens that layer by documenting bounded executable artifact-backed partial realization of a PMS-STACK Evidence Spine v0.1. PMS is coherent enough to generate questions of runtime, architecture, language, policy, execution, AI proposal boundaries, traceability, tests, and machine-adjacent form. Where executable artifacts exist, the implementation claim becomes stronger than speculative realization pressure: it shows constrained implementation feasibility in artifact-supported form.

That is serious.

But implementation feasibility is not validation of PMS Base.

PMS-RUST supports a bounded IMPLEMENTATION / ARTIFACT CLAIM.

It does not support a BASE CLAIM.

The key sentence is:

> **What survives is not PMS' claim to finality, but its disciplined power to make praxis structurally legible.**

That power is not small.

It is also not final.

11.2 What Must Be Reduced

Chapter 11 must name the reductions explicitly.

The critique reduces PMS in the following way:

```
universality        → wide portability / generativity
foundation          → operator grammar
completeness        → bounded / versioned / internally constrained
success             → not finality
translation         → not privilege
implementation      → bounded, claim-typed, artifact-supported feasibility where documer
PMS-RUST            → implementation / artifact evidence, not PMS Base validation
governance          → not tribunal
MIP/AH              → downstream governance / AH Precision / attack-surface hardening, r
self-application    → not final self-arbitration
rival models        → remain open
non-capture         → remains possible
```

These reductions are not rhetorical modesty.

They are also not signs that PMS lacks force. PMS is reduced here precisely because it has force: it travels widely enough to tempt universality, organizes praxis strongly enough to tempt foundation, generates enough coherence to tempt completeness, succeeds often enough to tempt finality, translates widely enough to tempt privilege, and produces enough application and implementation-layer realization to tempt validation.

PMS-RUST intensifies rather than removes this need for reduction. A public repository, executable demos, tests, traces, golden fixtures, model validation, documentation, acceptance gates, and AI bridge boundaries can make PMS appear more settled than its claim type permits. That artifact strength must remain implementation evidence only.

Under Load does not forbid strong extensions of PMS. It forbids converting those extensions into proof of PMS Base.

Each reduction names a temptation produced by real strength. PMS is not being reduced because it fails to travel, generate, stabilize, translate, formalize, govern, implement in parts, or reflect on itself. It is reduced because it does these things strongly enough that the next status claim becomes tempting.

Under Load interrupts that next step.

The reduction is therefore not a retreat from PMS' power. It is the discipline required by that power.

Universality becomes wide portability / generativity.

PMS does travel widely. It generates disciplined readings across domains and reveals structural parallels where local vocabularies remain separate. This is one of its strongest features.

But this is exactly where the overclaim begins.

Wide portability is not universality. PMS may reach many domains without owning the final grammar of those domains. It may make praxis broadly legible without becoming the grammar of praxis as such.

PMS does not lose universality because it lacks reach. It loses universality because reach is not the same as authority over everything it reaches.

Foundation becomes operator grammar.

PMS is foundationally tempting because Δ–Ψ gives analysis a disciplined operator spine. That spine is real. It allows PMS to generate, compare, and constrain readings with unusual coherence.

But an operator spine is not a metaphysical ground.

PMS provides a structured grammar of legibility, not the final foundation of practice.

Completeness becomes bounded / versioned / internally constrained.

PMS can look complete because it keeps generating usable readings. Its productivity creates the impression that whatever remains outside is only waiting to be translated.

That impression must be refused.

Completeness becomes boundedness: PMS remains versioned, internally constrained, and open to cases where translation would weaken rather than preserve difference.

Success becomes not finality.

PMS succeeds often enough that success itself becomes a risk. A weaker grammar would fail before this problem appears. PMS reaches the harder danger: it can stabilize readings so well that stabilization begins to imitate validation.

Success is therefore not finality.

It is load.

Translation becomes not privilege.

PMS can translate rival readings into its own terms. This is a real strength. But translation is also where privilege can silently enter: the grammar that translates may begin to look superior merely because it can absorb.

That inference fails.

To translate a rival grammar is not yet to defeat it.

Implementation becomes bounded, claim-typed, artifact-supported feasibility where documented, not proof.

Implementation matters. PMS produces enough structural coherence to motivate runtime, architecture, VM, service, AI-bridge, traceability, and execution questions. PMS-RUST now gives this layer a documented artifact surface: parts of a PMS-derived stack can be represented, validated, executed, traced, tested, and inspected under specific technical constraints.

This is stronger than speculative implementation pressure.

It is bounded executable artifact-backed partial realization.

But executable realization changes the status of the implementation claim, not the status of PMS Base.

A PMS-derived architecture may be partially realizable without proving that PMS is complete, final, universal, or uniquely adequate as a grammar of praxis. Positive warrant for PMS Base would require external application under calibrated, rival-sensitive, and falsifier-bearing conditions, not implementation feasibility alone.

The reduced claim is:

```
PMS survives implementation pressure only as bounded, claim-typed, artifact-supported fe
```

The prohibited claim is:

```
PMS survives because it has a working repo.
```

The repo strengthens the implementation layer.

It does not validate the base grammar.

Governance becomes not tribunal.

PMS is governance-relevant because its language can harden, circulate, expose, rank, or misfire. Governance is therefore necessary where PMS becomes applied artifact.

But governance does not become tribunal.

It constrains use; it does not validate the grammar.

MIP/AH becomes downstream governance / AH Precision / attack-surface hardening, not rescue layer.

MIP/AH may regulate how PMS is used and expose attack surfaces in analysis artifacts. It may not repair operator ambiguity, solve calibration, or prove PMS Base.

Self-application becomes not final self-arbitration.

PMS can read itself with unusual discipline. It can name its own drift risks, type its own claims, and expose its own overclaims. This is a serious strength.

PMS-RUST can make some PMS-derived behavior traceable and inspectable. That is also a real artifact strength.

But the grammar that exposes the problem is still the grammar under test.

Traceability is not self-arbitration.

Self-application is exposure, not final arbitration.

Rival models remain open.

PMS may compare itself to rival grammars. It may translate them. It may locate differences. It may even outperform them in certain structural tasks. But it may not treat them as defeated merely because they can be rendered PMS-legible.

Non-capture remains possible.

Some failures may remain outside PMS. Some resistance may not be unfinished integration. Some differences may be weakened if translated.

The key sentence is:

> **The critique reduces scope, not seriousness.**

This sentence is decisive.

PMS is not made trivial by reduction.

It becomes more serious because its scope is no longer inflated.

11.3 Reduced Position — What PMS Most Plausibly Is

The reduced thesis is:

```
PMS is not the final grammar of praxis as such.
It is a highly productive, bounded, calibration-dependent, direction-bearing, meta-norma
```

The implementation addition is:

```
PMS also supports a bounded implementation claim where PMS-derived structures are docume
```

This is not the weakest position left after critique.

It is the strongest reduced position PMS can responsibly hold after critique.

Each word carries a reduction.

Highly productive means PMS can generate real structural readings.

It does not mean PMS is complete.

Bounded means PMS has scope conditions, layer limits, claim types, and non-capture possibilities.

It does not mean PMS is weak.

Calibration-dependent means PMS structures threshold judgment but does not fully replace it.

It does not mean PMS is arbitrary.

Direction-bearing means PMS selects for distance, reversibility, non-person-ranking, misuse resistance, and accountable self-binding.

It does not mean PMS is a first-order moral code.

Meta-normative means PMS has second-order application-validity preferences.

It does not mean PMS derives an ought.

Grammar of making praxis legible means PMS produces structural readability.

It does not mean PMS is the final structure of praxis itself.

Across domains means PMS has wide portability.

It does not mean portability equals equivalence.

Bounded implementation claim means PMS-derived structures can be partially realized in executable artifact form where documented.

It does not mean implementation validates PMS Base.

This is the reduced position.

The red line is:

```
Do not add a stronger finality claim after this reduction.
```

If PMS now says:

```
Therefore PMS is the final grammar after all.
```

the chapter fails.

If PMS says:

```
Therefore PMS proves its own priority by surviving critique.
```

the chapter fails.

If PMS says:

```
Therefore PMS is validated because PMS-RUST exists.
```

the chapter fails.

The reduced position must remain reduced.

PMS survives only if it does not use survival to restore what the critique removed.

11.4 Positive Repositioning — Power Without Privilege

PMS remains strong.

But differently strong.

Not:

```
PMS as final grammar.
```

But:

```
PMS as a highly productive, self-limiting grammar of praxis legibility.
```

And, where PMS-RUST is concerned, not:

```
PMS as proven by implementation.
```

But:

```
PMS as supporting bounded, claim-typed, artifact-backed implementation feasibility.
```

This is not retreat.

It is repositioning.

A final grammar would claim last authority over praxis. A self-limiting grammar makes praxis more legible while preserving calibration, rival models, non-capture, and layer discipline.

A final grammar would treat translation as superiority. A self-limiting grammar treats translation as reach under reduction cost.

A final grammar would treat implementation as validation. A self-limiting grammar treats implementation as research pressure and artifact-supported feasibility under constraint.

A final grammar would treat a working repository as proof. A self-limiting grammar treats repository evidence as implementation-layer evidence only.

A final grammar would treat governance as maturity proof. A self-limiting grammar treats

governance as downstream application discipline.

A final grammar would treat self-application as settlement. A self-limiting grammar treats self-application as exposure without final arbitration.

The key sentence is:

> **PMS remains powerful precisely where it accepts that power without privilege.**

This is the positive repositioning.

Power remains.

Privilege is reduced.

This reduction is not an apology for PMS. It is the form its seriousness takes after critique. A weaker framework would not need to refuse so many tempting overclaims; PMS does, because its actual reach — including PMS-RUST's bounded executable artifact-backed partial realization — makes those overclaims plausible enough to be dangerous.

PMS can still de-obscure. It can still discipline analysis. It can still structure cross-domain comparison. It can still detect drift. It can still make publicness risky in a precise way. It can still support governance under constraint. It can still motivate bridge and implementation research. It can now also point to bounded artifact-supported feasibility where PMS-RUST documents executable implementation.

But it may not treat those capacities as proof of finality.

Under Load therefore does not leave PMS smaller in the sense of trivial.

It leaves PMS sharper.

A bounded tool can be useful.

A bounded operator grammar can be theoretically serious.

A bounded implementation artifact can be research-relevant.

A self-limiting praxis grammar can make a more disciplined claim than an inflated universal grammar because it pays the cost of its own power.

11.5 Direction After the Critique

PMS does not become neutral after critique.

It becomes more accountable for its direction.

Chapters 1–10 have made it unavailable for PMS to present itself as merely descriptive in a normatively neutral sense. PMS selects. It favors X (distance / reflective distance) over fusion, reversibility over closure, scene-boundness over global labels, Ψ (self-binding) over unowned consequence, and non-person-ranking over tribunal judgment.

Those selections survive.

But they survive only as accountable direction.

PMS may not pretend that its direction is the view from nowhere. It may not claim that its preferred forms of praxis are simply what structure "really" is. It may not treat its own guardrails as proof of superior maturity.

Implementation does not neutralize this direction.

A runtime, trace, test, or repository artifact can execute or inspect a PMS-derived structure. It cannot remove PMS' direction-bearing character. Executability does not make the grammar neutral.

The question is not whether PMS has direction.

It does.

The question is whether that direction remains explicit, accountable, revisable, and calibration-aware.

The key sentence is:

> **PMS does not become neutral after critique; it becomes more accountable for its direction.**

This matters for future use.

A PMS application should be able to say:

```
This claim is direction-bearing.
This claim selects for certain praxis forms.
This claim depends on calibration.
This claim remains reversible.
This claim does not rank persons.
This claim may be challenged by rival framings.
This claim may fail under publicness.
This claim may indicate non-capture rather than drift.
This implementation artifact supports only a bounded claim.
This trace, test, or gate does not validate PMS Base.
```

If PMS cannot say these things, its direction is becoming hidden.

If PMS can say them and still remain useful, its direction becomes accountable.

That is what survives here.

Not neutrality.

Accountable direction.

11.6 What Must Remain Outside

A self-limiting PMS must preserve genuine non-capture.

The required sentence is:

> **Not everything PMS cannot yet integrate should be treated as drift, recalibration demand, Φ-demand (recontextualization demand), or Λ-residue (structured absence / non-event residue). Some failures may indicate genuine non-capture.**

This is one of the most important results of the paper.

Chapter 11 must not reproduce coverage by treating every non-fit as an internal PMS category.

The wrong move would be:

```
Everything outside PMS is only temporarily unintegrated.
```

That would restore the very absorption Under Load has tested.

The implementation version of the wrong move would be:

```
Everything PMS-RUST cannot yet implement is only a future roadmap item.
```

That too would be a form of coverage.

The correct move is:

```
Some outside may remain outside.
```

This does not mean that PMS must fall silent whenever it meets resistance. PMS may still analyze the encounter. It may say that a scene resists calibration, that Φ risks absorption, that a rival grammar preserves distinctions PMS does not preserve, that a public artifact cannot be responsibly circulated, that a domain transfer should be suspended, or that an implementation artifact supports less than its surrounding rhetoric suggests.

But PMS must not assume in advance that resistance is only:

```
drift,
misapplication,
poor calibration,
unmade Φ,
stack confusion,
publicness distortion,
implementation gap,
roadmap delay,
or Λ waiting for integration.
```

Some resistance may remain structurally outside PMS.

A case of genuine non-capture is not merely a case PMS has not yet integrated. It is a case where integration would weaken the very difference that made the case resistant.

This is the key sentence:

> **A self-limiting PMS must leave open the possibility that some resistance is not**

> **unfinished integration, but genuine non-capture.**

This preserves the force of rival models.

A rival model may be locally stronger.

A domain vocabulary may preserve distinctions PMS smooths.

A symbolic reading may retain force that PMS translation reduces.

A technical formalism may not be replaceable by structural mapping.

A runtime mismatch may reveal a limit of a specific implementation.

An implementation failure may sometimes require claim reduction, not merely code repair.

An inner psychological or physical process may not be directly available to PMS as praxis grammar.

A public claim may become inadmissible even if locally structurally disciplined.

A self-reading may remain unresolved without becoming proof of PMS' openness.

This outside is not a defect to be eliminated.

It is part of PMS' bounded scope.

PMS remains serious only where it can distinguish:

```
what it can read,
what it can partially read,
what it can only read under reduction,
what it can implement under constraint,
what implementation does not validate,
what it should not publicly apply,
what it must leave to another grammar,
and what it cannot capture without loss.
```

If PMS loses that distinction, Chapter 11 becomes another coverage machine.

11.7 STACK, PMS-RUST, and QC After Under Load

QC and STACK survive Under Load only under strict claim typing.

QC survives as:

```
QC = structural mapping / bridge, not proof.
```

QC may remain research-relevant because it tests whether PMS can map structural relations into formal-technical contexts without confusing mapping with physical explanation. It may support audit, annotation, comparison, workflow description, and structural overlay.

But QC may not become:

```
physical proof,
```

```
substrate-universal validation,
replacement of quantum formalism,
or proof of PMS Base.
```

QC survives as bridge potential.

It does not survive as proof.

STACK survives differently:

```
STACK = implementation-layer claim: realization horizon and implementation pressure.
```

PMS-RUST specifies the currently documented artifact form of that layer:

```
PMS-RUST = implementation / artifact claim: bounded executable artifact-backed partial r
```

This distinction matters.

Where STACK remains architectural design, it functions as realization horizon and implementation pressure. Where PMS-RUST provides documented executable artifacts, the claim becomes stronger: PMS-derived structures have crossed into constrained executable realization. That is not merely speculative. It shows that at least parts of the PMS-derived architecture can be made operational under specific technical conditions.

But this changes the status of the implementation/artifact claim, not the status of PMS Base.

STACK may therefore remain research-relevant because PMS generates enough structural coherence to raise serious realization questions. PMS-RUST may remain artifact-relevant because it documents bounded executable feasibility: runtime/toolchain structure, PMS.yaml model validation, REPL/script execution, traceability, tests, demos, acceptance gates, and AI bridge boundaries.

But neither STACK nor PMS-RUST may become:

```
proof of PMS Base,
evidence of final grammar,
implementation as truth,
repository evidence as validation,
machine-form validation,
or completeness through executability.
```

Executable realization can prove constrained feasibility of an implementation layer.

It cannot by itself prove that PMS is the final grammar of praxis. Nor can it substitute for external application, calibration, rival comparison, or load-bearing falsifiers.

PMS-VM, Linux-layer ideas, and future implementation artifacts may remain legitimate research paths. Where they are not part of the argument of this paper, they remain future work. Where partial artifacts already exist, they count as implementation-layer evidence, not base validation.

The required sentence is:

> **Implementation may strengthen PMS as a research program, and PMS-RUST may demonstrate bounded executable feasibility, but neither validates PMS as a final grammar.**

The same reduction applies to QC.

A grammar may map formal structure.

That is meaningful.

It may bridge into a technical field.

That is research-relevant.

It may support formal-technical analysis.

That is important.

But bridge is not proof.

The surviving position is therefore:

```
QC survives as bridge stress / structural mapping, not proof.
STACK survives as realization horizon and implementation pressure.
PMS-RUST survives as bounded executable artifact-backed partial realization where docume
None survives as validation of PMS Base.
```

This is not a demotion.

It is correct claim typing.

11.8 MIP/AH After Under Load

MIP/AH (Maturity in Practice / AH Precision: Attack Surface / Hardening) also survive.

But they survive under constraint.

The required sentence is:

> **MIP/AH survive as downstream governance, application discipline, and AH Precision attack-surface hardening, not as validation machinery.**

The sharper form is:

> **MIP/AH constrain how PMS may be used; they do not prove what PMS is.**

Both sentences must remain active.

MIP/AH may continue to do important work:

```
no person-ranking;
misuse prevention;
publicness constraints;
```

```
reversibility markers;
analysis-artifact responsibility;
scope discipline;
analysis-artifact attack-surface visibility;
language hygiene;
dignity protection in application;
anti-tribunal safeguards.
```

These are not minor.

Without them, PMS applications become riskier. PMS vocabulary can become public harm, person fixation, maturity ranking, or structural authority inflation.

But MIP/AH may not do:

```
prove PMS Base;
solve calibration;
repair operators;
guarantee discrimination;
neutralize publicness;
turn governance into truth;
convert hardening into self-endangerment;
make self-application final.
```

MIP/AH survive because PMS is applied.

They do not survive as proof of PMS Base.

This is especially important after the pressures tested in Chapters 6, 7, 9, and 10.

Chapter 6 placed governance under selectivity pressure.

Chapter 7 placed guardrails under success-trap pressure: they can increase trust without proving differentiation.

Chapter 9 placed publicness under exposure pressure even where governance constraints apply.

Chapter 10 placed hardening under self-application pressure: hardening is not self-endangerment.

PMS-RUST adds a related artifact lesson. Tests, gates, docs, traces, and AI bridge boundaries may harden an implementation artifact. They may not validate PMS Base. Artifact hardening and governance hardening share the same reduction: they constrain use and improve inspectability, but they do not prove the grammar.

Therefore MIP/AH must remain downstream.

They may say:

```
This PMS artifact is admissible under these conditions.
This public use is too risky.
```

```
This language is too person-near.
This analysis needs attack-surface disclosure.
This claim must remain reversible.
```

They may not say:

```
Because this artifact is governed, PMS Base is validated.
Because misuse is constrained, calibration is solved.
Because no person-ranking is enforced, PMS preserves all relevant difference.
Because an implementation artifact is tested and gated, PMS has survived as final gramma
```

The surviving position is:

```
MIP/AH remain necessary where PMS produces applied artifacts, public-facing analyses, or
MIP/AH remain inadmissible as rescue for PMS Base.
```

This is the only way they survive Under Load without becoming the layer drift they were meant to prevent.

11.9 Conclusion

What survives is reduced but serious.

PMS does not survive as final grammar.

It does not survive as universal foundation.

It does not survive as complete system.

It does not survive as self-validating theory.

It does not survive through implementation proof, even where implementation feasibility is executable and artifact-backed in parts. PMS-RUST strengthens the implementation/artifact claim, but it does not validate PMS Base. PMS becomes available for external warrant only where application, calibration, rival comparison, and falsifier-bearing use place PMS under pressure beyond its own self-reading.

It does not survive through governance reassurance.

It does not survive through bridge mapping.

It does not survive through repository evidence.

It does not survive through self-application alone.

What survives is more bounded and more precise.

PMS survives as a disciplined operator grammar for making praxis structurally legible across domains under conditions of calibration, direction, reversibility, non-capture, layer discipline, and claim typing.

It survives as a grammar that can de-obscure structures without claiming to exhaust them.

It survives as a drift-sensitive method that must not turn every resistance into drift.

It survives as a portable grammar that must not turn portability into equivalence.

It survives as a direction-bearing framework that must not pretend neutrality.

It survives as a governance-relevant stack that must not convert governance into proof.

It survives as bridge-capable and implementation-relevant — and, where documented artifacts exist, partially realizable in bounded executable form — without treating bridge, realization, executability, or repository evidence as validation.

It survives as self-critical without treating self-critique as settlement.

PMS is not reduced because it cannot travel, generate, formalize, govern, or partially realize itself in technical form. It is reduced because those strengths make unauthorized status jumps tempting.

The final reduction is:

> **What survives is not PMS as final grammar, but PMS as a highly productive, bounded, and self-limiting grammar of praxis legibility.**

The implementation reduction is:

> **What survives now includes a bounded implementation claim: PMS survives implementation pressure only as claim-typed, artifact-supported feasibility, not because it has a working repository.**

12. Conclusion

Chapter Aim

This conclusion does not end in triumph.

It does not end in retreat.

It does not say that PMS is false, merely tooling, only language, finally proven, or unchanged after critique. None of those conclusions would follow from the preceding chapters.

The result is narrower and stronger:

```
PMS works — but not without direction, calibration, stack discipline, implementation-bou
```

That sentence should not be read as a retreat into weakness. PMS has enough structural power to invite stronger claims. It also now has a documented implementation artifact: PMS-RUST, a bounded executable artifact-backed partial realization of a PMS-STACK Evidence Spine v0.1. The conclusion refuses stronger claims not because PMS has failed, but because its success and artifact support would otherwise become too easy to misread as finality.

PMS has survived Under Load only by giving up claims it could not responsibly keep. It survives not by escaping pressure, but by becoming more precise under it.

The paper has therefore not asked whether PMS can remain coherent. It can. It has not asked whether PMS can generate analyses. It can. It has not asked whether PMS can travel across domains, produce readable structures, support governance, motivate implementation, produce an implementation artifact, or criticize itself. It can do all of these.

The harder question has been whether these strengths remain discriminating under the very conditions that make PMS powerful.

That is where the conclusion must remain.

12.1 Restating the Under Load Question

The central question of PMS Under Load was never whether PMS fails in obvious ways.

A weak framework fails where it cannot explain, cannot travel, cannot stabilize, cannot formalize, cannot protect its own use, cannot motivate implementation, cannot expose its own limits, and cannot produce inspectable artifacts.

PMS is not weak in that way.

Its danger is more demanding.

PMS becomes unstable where its strengths begin to remove the visibility of their own boundaries. Its portability can begin to look like equivalence. Its translation power can begin to look like privilege. Its implementation pressure can begin to look like proof. Its executable artifact-backed partial realization can begin to look like PMS Base validation. Its governance discipline can begin to look like validation. Its self-application can begin to look like final arbitration.

The key sentence is:

> **PMS does not fail where it has limits.**
> **It fails where limits disappear into structural coverage.**

This is the central reduction.

Limits are not defects.

A bounded grammar can be stronger than an inflated one. A calibrated framework can be more serious than a universal one. A theory that preserves non-capture is more accountable than a theory that silently absorbs all resistance. An implementation artifact can be valuable without becoming proof.

The problem is not that PMS has limits.

The problem would be if PMS could no longer see them.

The implementation version is:

```
Implementation is not proof.
Executable artifact-backed partial realization changes the implementation claim, not the
```

Under Load therefore tested not failure, but overfunction.

It asked where PMS stops distinguishing and starts stabilizing.

12.2 What the Critique Has Shown

The critique has shown that PMS remains powerful only where its power remains under constraint.

Calibration showed that PMS' thresholds are not fully endogenous. The operators are formally disciplined, but their application still depends on scene-sensitive judgment, external calibration pressure, and the possibility that two PMS-conform readings may diverge.

Stack Drift showed that portability can destabilize layer boundaries. PMS Base, domain layers, add-ons, governance layers, bridge mappings, implementation claims, repository artifacts, and self-application claims cannot borrow authority from one another without drift.

Φ Drift showed that Φ (recontextualization) can clarify Δ (difference), but it can also absorb it. Reframing is not reversal. Translation is not preservation unless the original Δ remains load-bearing.

Meta-Normativity showed that PMS is not neutral in its application grammar. It selects for X (distance / reflective distance), reversibility, non-person-ranking, Ψ (self-binding), misuse resistance, and accountable direction. This does not make PMS a moral code, but it prevents PMS from pretending to be normatively weightless.

The Success Trap showed that PMS-consistent outputs may become easier to produce than difference-sensitive ones. Stabilizability is not validation. Dialogic reproducibility can count as evidence of generativity, not final adequacy. Artifact reproducibility can count as evidence of implementation discipline, not final adequacy.

Domain Drift and Bridge Stress showed that reach is load. Domain transfer does not prove

equivalence. QC-style bridge mapping does not prove physical adequacy, substrate universality, or priority over the formalism it maps. PMS-RUST adds implementation-boundary stress: runtime mapping is not theoretical identity, model validation is not praxis adequacy, traces are not external validation, and repository evidence is not PMS Base validation.

Publicness showed that circulation changes claim conditions. A locally disciplined structural reading may become unstable when it becomes public, searchable, quotable, reputational, or labelable.

Self-Application showed that PMS can expose its own limits, but cannot thereby fully arbitrate the authority, scope, or adequacy of its own grammar from within. Self-exposure is not self-settlement. Artifact auditability is not genuine self-endangerment.

What Survives showed that PMS remains serious after reduction, not before it. What survives now includes a bounded implementation claim, but only as claim-typed, artifact-supported feasibility.

The key sentence is:

> **Where PMS stops distinguishing and starts stabilizing, critique must begin.**

This is not a rejection of PMS.

It is the condition under which PMS remains usable.

12.3 Direction Without Privilege

PMS does not become neutral after critique.

It becomes more accountable for its direction.

The preceding chapters have shown that PMS is direction-bearing. It favors X (distance / reflective distance) over fusion, reversibility over closure, scene-bound claims over global labels, Ψ (self-binding) over unowned consequence, non-person-ranking over tribunal drift, and misuse resistance over rhetorical force.

Those preferences are not accidental.

They belong to the grammar's application discipline.

Implementation does not erase this direction.

A runtime can execute a PMS-derived structure. A trace can record behavior. A test can check specified conformance. A repository can document implementation discipline. None of these makes PMS normatively weightless or direction-free.

The issue is not that PMS has direction. A grammar of praxis without direction would be less useful, less accountable, and less honest about its own selective conditions.

The required sentence is:

> **The question is not whether PMS has direction, but whether that direction remains explicit, accountable, and revisable under its own grammar.**

This is the final form of the meta-normativity result.

PMS must not say:

```
PMS is valid because its direction is more mature.
```

It may say:

```
PMS remains usable only where its direction remains open to critique.
```

It must also not say:

```
PMS is validated because its direction can be represented in executable artifact form.
```

It may say:

```
PMS-RUST supports a bounded implementation claim where PMS-derived structures are execut
```

That distinction matters.

A direction-bearing grammar can remain serious where it names its direction, binds its use, exposes its limits, and remains revisable. It becomes dangerous where its direction hides behind structural description, governance discipline, or implementation artifact status.

PMS therefore survives only where its own direction remains visible.

Not privileged.

Visible.

12.4 Reduced Survival

The survival thesis is now reduced.

> **PMS survives Under Load not as final grammar, but as a highly productive, bounded, self-limiting grammar of praxis legibility.**

This sentence carries the paper's final reduction.

It must not be followed by a stronger claim.

PMS does not survive as universal foundation.

It does not survive as complete system.

It does not survive as self-validating grammar.

It does not survive by implementation proof.

It does not survive by repository evidence.

It does not survive by governance reassurance.

It does not survive by bridge mapping.

It does not survive by self-application alone.

It survives as a disciplined operator grammar that makes praxis structurally legible under

conditions of calibration, direction, reversibility, non-capture, layer discipline, and claim typing.

It also survives with a bounded implementation claim:

```
PMS-RUST documents executable artifact-backed partial realization of a PMS-STACK Evidenc
```

But that claim remains reduced:

```
Executable artifact-backed partial realization changes the implementation claim, not the
```

It survives because it can still distinguish.

It survives because it can still reduce itself.

It survives because it can still leave something outside.

These three conditions correspond to the instability corridors tested above: discrimination under calibration and success pressure, reduction under layer and self-application pressure, and non-capture under domain, publicness, translation, and implementation-boundary pressure.

This is not a retreat into "mere tooling."

A bounded grammar is not merely a tool if it can generate structural legibility, detect drift, clarify asymmetry, discipline application, expose publicness load, preserve non-capture, and motivate artifact-supported implementation under constraint.

But it is also not final.

PMS remains serious because it has been reduced.

Its restraint is not lack of force. It is claim discipline under conditions where more expansive claims would be structurally tempting.

12.5 Final Closing

The final position is therefore neither skeptical dismissal nor theoretical triumph.

PMS works.

But it works under load.

It works where operators remain disciplined.

It works where calibration is not hidden.

It works where portability does not become equivalence.

It works where stack boundaries remain visible.

It works where Φ (recontextualization) preserves Δ (difference) rather than absorbing it.

It works where success does not become validation.

It works where publicness is treated as claim-condition, not mere audience size.

It works where MIP/AH constrain use without rescuing the base.

It works where QC remains bridge.

It works where STACK remains implementation pressure.

It works where PMS-RUST remains bounded executable artifact-backed partial realization at the implementation layer rather than validation of PMS Base.

It works where internal self-discipline prepares PMS for external warrant without replacing it.

It works where self-application exposes limits without claiming final self-arbitration.

Most importantly, PMS works where it can still be interrupted by what it cannot capture.

The final lesson is not that PMS has survived everything unchanged.

The final lesson is that PMS survives only by becoming more accountable to the limits its own grammar can expose but not fully settle.

PMS has an implementation artifact.

Implementation is not proof.

Executable artifact-backed partial realization changes the implementation claim, not the truth-status of PMS Base.

PMS Under Load tests not only whether PMS_1.3 absorbs too much, but whether PMS can preserve its own grammar, layer boundaries, claim status, implementation-boundary discipline, and discriminative force under the very portability and artifact stabilizability that make it powerful.

Appendix A — Source, Method, and Claim Discipline

A.0 Function of This Appendix

This appendix defines the source basis, method boundary, claim discipline, and application constraints used by *PMS Under Load*. Its purpose is not bibliographic completeness. Its purpose is stack discipline.

Source scope is not bibliographic housekeeping; it is part of the paper's stack discipline.

Because PMS is portable across base theory, domain papers, add-ons, bridge mappings, implementation layers, and governance or hardening layers, source status must be fixed before critique begins. Otherwise the critique would risk reproducing the same drift it tests: a domain application could become a base grammar, a bridge mapping could become proof, a governance layer could become validation, or implementation pressure could become truth.

This appendix therefore specifies what each source type may authorize, what it may not authorize, and how claims must remain typed, bounded, and falsifiable.

A.1 Source Scope

PMS Under Load responds to the current supplied source state of PMS. It does not reconstruct missing sources, infer unavailable content from titles, or use obsolete formulations as current authority unless they are explicitly marked as historical material.

The source basis is layered:

```
PMS Base / PMS.yaml
= source of truth for Δ–Ψ, dependencies, derived axes, formal constraints, base guardrai

Finished PMS Under Load chapters
= current manuscript state of the structural self-critique

Domain papers
= operator-strict stress and application layers where explicitly available

Add-ons
= precision, application, or hardening layers, not primitives

MIP/AH
= downstream governance, application-discipline, and attack-surface hardening under cons

QC
= bridge / structural mapping layer

STACK
```

```
= implementation-layer claim, realization horizon, and implementation pressure

Rust / repository artifacts
= public PMS-RUST implementation artifact; documented PMS-STACK Evidence Spine v0.1; imp
```

The source posture can fail. It fails if unavailable sources are treated as current authority, if superseded material is used as though it defines the present model, or if layer discipline is converted into reassurance. It also fails if the absence of a source is silently repaired by reconstruction.

This appendix does not claim that all current PMS materials are complete, final, or equally strong. It only fixes how source authority is handled.

A.2 Source-Type Discipline

Source Type	Used For	Not Used For	Claim Status
PMS Base / PMS.yaml	Operator definitions, dependencies, derived axes, formal constraints, base guardrails, claim boundaries	Domain-specific conclusions, empirical proof, implementation validation, finality	BASE CLAIM / formal reference
Finished *PMS Under Load* chapters	Current self-critique architecture, pressure zones, reductions, chapter-level claims	Replacing PMS Base operator definitions	SELF-APPLICATION / critique architecture
Domain Papers	Domain stress, applied operator behavior, drift risks, concrete instability profiles	Redefining Δ–Ψ, becoming new base grammars	DOMAIN CLAIM
Add-on YAMLs	Specification, contract, control structure, application refinement, hardening	Base primitives, operator repair, proof of PMS validity	ADD-ON / HARDENING CLAIM
MIP/AH	Application boundaries, misuse risk, publicness constraints, artifact hardening, no-person-ranking, claim-typing discipline	Base validation, calibration repair, tribunal judgment, rescue layer	GOVERNANCE / ADD-ON-HARDENING CLAIM
QC	Bridge mapping, structural comparison, formal-technical stress	Physical proof, substrate universality, validation of PMS Base	BRIDGE CLAIM
STACK	Realization horizon, implementation pressure, implementation-layer architecture	PMS Base proof, final validation, truth by implementation	IMPLEMENTATION CLAIM
Rust / Repository Artifacts	Public PMS-RUST implementation artifact; documented PMS-STACK Evidence Spine v0.1; bounded executable artifact-backed partial realization; repo-to-chapter relation	Proof of PMS Base, proof of final grammar, replacement for external validation, completion of PMS 1.3 semantics	IMPLEMENTATION / ARTIFACT CLAIM
Self-Application Artifacts	Audit strength, artifact vulnerability, dialogic trace, self-exposure material where available	Final self-arbitration, proof that PMS has solved its own limit	SELF-APPLICATION CLAIM

The rule is simple:

```
Use PMS Base / PMS.yaml for operator authority.
Use full papers for argumentative and applied behavior.
Use YAMLs for structure, gates, schemas, and formal status.
Use implementation artifacts only as implementation artifacts.
```

No source type may borrow authority from another without explicit reduction.

A.3 Internal Method Boundary

The critique is internally guided. It does not rely on an external thinker, school, framework, discipline, or rival model as final arbiter.

This does not forbid comparison. It forbids rescue.

An external framework may clarify a contrast, but it may not decide the critique. A rival model may remain epistemically open, but it may not be imported as a tribunal. PMS must be pressured through its own grammar, source discipline, claim types, and failure conditions.

The guiding question remains:

```
Where does PMS stop distinguishing — and start stabilizing?
```

This question is internal to the project because it tests PMS where its own strengths become active: portability, operatoric readability, recontextualization, stack extension, implementation pressure, governance hardening, and self-application.

The method boundary is therefore:

```
No external authority as final arbiter.
No outside model as rescue layer.
No phantom source.
No reconstruction of missing source content.
No source authority without source availability.
```

A.4 Valid Critique Inside PMS

A valid internal critique of PMS must be capable of generating real pressure, not merely confirming PMS compliance.

A critique counts as valid only where it is:

```
operatorically expressible
falsifiable
reversible
non-immunized
gate-preserving
```

```
capable of failing
```

Each condition restricts the critique.

Operatorically expressible means that the critique must identify which PMS operators, dependencies, layer relations, or derived constructs are under load. Loose dissatisfaction is not enough.

Falsifiable means that the critique must identify a structural counter-scenario under which its objection would weaken.

Reversible means that claims must remain scene-bound, revisable, and non-final. The critique must not become person-ranking, tribunal judgment, or irreversible classification.

Non-immunized means that PMS cannot treat every objection as misapplication, calibration residue, missing context, residual Λ, or unfinished recontextualization.

Gate-preserving means that application discipline remains in force: distance, reversibility, and Dignity-in-Practice constrain how PMS may be used, even when PMS itself is being criticized.

Capable of failing means that the critique must be vulnerable to evidence that PMS preserved discriminative force under the relevant load.

Internal critique is valid only where PMS can be structurally endangered by its own grammar.

A.5 Claim-Typing Protocol

Every strong claim in *PMS Under Load* must know what kind of claim it is.

The paper uses the following claim types:

```
BASE CLAIM
DOMAIN CLAIM
ADD-ON / HARDENING CLAIM
GOVERNANCE CLAIM
BRIDGE CLAIM
IMPLEMENTATION CLAIM
SELF-APPLICATION CLAIM
SPECULATIVE CLAIM
OUT OF SCOPE
```

The working assignments are:

```
PMS operator grammar
= BASE CLAIM

CRITIQUE / CONFLICT / LOGIC / ANTICIPATION / SEX / EDEN
= DOMAIN CLAIM
```

```
MIP/AH
= GOVERNANCE CLAIM / ADD-ON-HARDENING CLAIM

QC
= BRIDGE CLAIM

STACK
= IMPLEMENTATION CLAIM / REALIZATION CLAIM

Rust / repository artifacts
= IMPLEMENTATION / ARTIFACT CLAIM

Self-application claims
= SELF-APPLICATION CLAIM
```

A claim becomes unstable when it borrows authority from a layer other than the one in which it is made.

A BASE CLAIM may define or constrain the operator grammar. A DOMAIN CLAIM may show how PMS behaves under a specific field of application. An ADD-ON / HARDENING CLAIM may sharpen application, precision, or misuse resistance. A GOVERNANCE CLAIM may constrain use. A BRIDGE CLAIM may map structures across a boundary. An IMPLEMENTATION CLAIM may show realization pressure or bounded executable feasibility. A SELF-APPLICATION CLAIM may expose how PMS reads itself.

None of these claim types may silently become another.

The central test is not merely:

```
Is the claim coherent?
```

It is also:

```
Does the claim have the right status?
```

If a claim loses its force when typed, it was overextended.

A.6 MIP/AH Constraint

MIP/AH are admissible only under constraint.

MIP refers to Maturity in Practice as a praxeological-anthropological application discipline. AH refers to AH Precision as an optional attack-surface and hardening layer for analysis artifacts. Together, they may improve use, reduce misuse risk, clarify boundaries, and harden public or reusable outputs.

They may discipline application.

They may not rescue PMS Base.

The governing sentence is:

```
MIP/AH constrain application; they do not rescue the base grammar.
```

MIP/AH may be used for:

```
application boundaries
misuse risk
publicness constraints
artifact-hardening
no-person-ranking
claim-typing discipline
role sensitivity
reversibility
Dignity-in-Practice under asymmetry
```

MIP/AH may not be used for:

```
base validation
operator repair
calibration solution
tribunal use
person-ranking
governance as proof
shielding PMS from structural pressure
turning critique into compliance audit
```

This constraint matters because MIP/AH can make PMS feel safer. That safety may be real at the level of application discipline, but it does not prove that PMS Base is complete, sufficiently calibrated, or immune to coverage drift.

A stabilizing governance layer pays its reduction cost only when it remains downstream.

A.7 Excluded Uses and Phantom-Source Rule

The following uses are excluded:

```
using unavailable sources as if they were present
reconstructing missing source content
treating obsolete formulations as current authority
letting domain papers redefine Δ–Ψ
treating add-ons as base primitives
using MIP/AH as tribunal or rescue layer
using QC as proof
```

```
using STACK as validation
using Rust / repository artifacts as PMS Base proof
using self-application artifacts as final self-arbitration
using public uptake as truth
using coherence as sufficiency
using portability as equivalence
using implementation as proof
```

A source may be named without being used as authority. A source may be structurally relevant without being available for drafting. A source may be optional, excluded, historical, or pending later appendix treatment. But if a claim depends on a source, the source must be available in the current project context.

The rule is:

```
No phantom source.
No reconstructed content.
No silent authority transfer.
```

A.8 Boundary to Later Appendices

This appendix defines method and source discipline. It does not perform all later source accounting.

Appendix B provides the PMS operator reference.

Appendix C provides the layer / stack position matrix.

Appendix D provides the chapter-to-source and domain-layer matrix.

Appendix E collects falsifiers.

Appendix F defines the drift / coverage / limit lexicon.

Appendix G connects the Rust project and repository to the paper under implementation-boundary constraints.

This appendix therefore sets the rules by which later appendices become possible. It does not replace them.

A.9 Reduction Rule

The method of *PMS Under Load* is reduction, not rescue.

A source strengthens the paper only when it remains within its proper status. A layer strengthens the argument only when its authority is bounded. A guardrail strengthens application only when it does not become proof. A repository artifact strengthens implementation claims only when it does not validate PMS Base. A self-application artifact strengthens self-exposure only when it does not become self-settlement.

The controlling reduction is:

```
PMS may remain powerful only where its power remains typed, bounded, and interruptible.
```

This appendix therefore does not make PMS safer. It makes PMS easier to test.

Appendix B — PMS v1.3 Operator Reference

B.0 Scope Note

This appendix provides the operator reference for PMS Under Load. Its function is not to extend PMS, revise the operator set, or import domain-local vocabulary into the base grammar. It fixes the reference boundary for the eleven PMS meta-axioms, their default dependency path, the derived axes, and the status of machine-readable representation.

For the purposes of PMS Under Load, the supplied PMS Base paper and `PMS.yaml` are treated together as the operative PMS v1.3-compatible operator reference. The Base paper provides the conceptual and argumentative account of Δ–Ψ. `PMS.yaml` provides the current repository reference specification for the default operator inventory, dependency path, derived constructs, and application guardrails. The internal draft label of the Base paper does not change its role in this project context: it is used here only as the supplied PMS Base reference.

This appendix makes no claim that the operator set is final, uniquely ordered, or complete. It records the current PMS reference path used by PMS Under Load. Alternative orderings, rival grammars, revised dependencies, or genuine non-capture remain methodologically open.

B.1 Operator Table: Δ–Ψ

Order	Operator	Name	Minimal Function	Depends On	Provides	What It Can Authorize	What It Cannot Authorize
1	Δ	Difference	Minimal structural distinction enabling differentiation.	—	boundary formation; object differentiation; structural contrast	Basic distinction, markability, contrast, initial operatoric legibility	Moral rank, diagnosis, fina capture, substantive ontology
2	∇	Impulse	Directional tension or drive arising from difference.	Δ	activation; orientation; energetic gradient; proto-agency	Directionality, movement pressure, reactive tendency	Full action, intention, moral motivation, stable agency
3	□	Frame	Contextual structure that constrains and shapes impulses.	Δ, ∇	context; boundary conditions; relevance structuring; role space	Situatedness, relevance, constraint, role-space	Final context, universal frame, closure of interpretation
4	Λ	Non-Event	Structured absence: meaningful	□	expectation; counterfactual structure;	Absence, omission, delay, unrealized	Nothingness, mere failure, automatic

Order	Operator	Name	Minimal Function	Depends On	Provides	What It Can Authorize	What It Cannot Authorize
			failure or delay of an expected occurrence within a frame.		tension through absence	expectation, counterfactual tension	pathology, hidden intention
5	A	Attractor	Recurrent pattern or stabilization emerging from repeated structural interaction.	Δ, ∇, □, Λ	pattern formation; stability; path dependence; role-script emergence	Recurrence, stabilization, path-dependence, pattern pressure	Necessity, essence, irreversible identity, perso type
6	Ω	Asymmetry	Structural imbalance establishing directionality of power, exposure, capacity, or obligation.	A	role differentiation; responsibility gradients; vulnerability structures	Directional inequality, role difference, burden, exposure, responsibility gradients	Human worth ranking, domination, moral superiority, person hierarchy
7	Θ	Temporality	Temporal structuring that enables trajectories, commitments, development, and longer-form self-relation.	Ω, A	sequence and trajectory; persistence; anticipation and delay; developmental arcs	Duration, trajectory, temporal pressure, commitment over time	Destiny, inevitability, progress guarantee, fin development
8	Φ	Recontextualization	Transformation through placing an existing structure into a new frame.	Θ, Ω, □	adaptation; reinterpretation; developmental change; pattern shift	Reframing, adaptive reinterpretation, transformation under changed context	Reversal, erasure, absorption of all non-fit, validation by reinterpretatio
9	X	Distance	Reflective withdrawal that attenuates immediate impulses and	Φ, Θ, □	regulation; inhibition; reflection and meta-position	Pause, inhibition, reflective gap, non-fusion, evaluative distance	Detachment a superiority, disengagemer as solution, neutrality proc

Order	Operator	Name	Minimal Function	Depends On	Provides	What It Can Authorize	What It Cannot Authorize
			patterns.				
10	Σ	Integration	Synthesis of disparate or conflicting elements into a more coherent whole.	X, Φ	coherence; resolution of contradiction; multi-level coordination; maturity of praxis	Coordination, synthesis, coherence under plurality	Harmony guarantee, contradiction erasure, completeness, final reconciliation
11	Ψ	Self-Binding	Formation of self-relation through commitment to integrated structures across time.	Σ, Θ, X	self-model; responsibility as self-commitment; stable normativity; autobiographical coherence	Stable self-relation, commitment, accountability across time	metaphysical selfhood, consciousness final self-arbitration, proof of validity

The table records the default PMS dependency path. It does not claim that this path is the only possible formal ordering. PMS.yaml itself treats the listed order and dependencies as the current repository reference path, while leaving alternative orderings, revised dependencies, or additional operators open to future formalization.

B.2 Derived Axes Table: A/C/R/E/D

The following constructs are derived within PMS. They are not independent primitives and must not be treated as additional base operators.

Axis	Name	Formula	Status	Function	Not a Base Operator Because...	Under Load Risk
A	Awareness	Θ ∘ □ ∘ Δ	Derived structural axis	Sustained, framed differentiation across time	It is generated from temporality, frame, and difference rather than introduced as primitive	Awareness may become surveillance, over-legibility, or interpretive control
C	Coherence	Θ ∘ Λ ∘ □ ∘ ∇	Derived structural axis	Temporally stabilized structuring of impulse and expectation within a frame	It depends on impulse, framing, non-event, and temporality	Coherence may be mistaken for truth or adequacy
R	Responsibility	Ψ ∘ Φ ∘ Θ ∘ Ω	Derived structural axis	Self-binding orientation toward asymmetry extended across time and recontextualization	It is composed from asymmetry, temporality, recontextualization, and self-binding	Responsibility may become tribunal, guilt assignment, or person-ranking
E	Action / Enactment	Σ ∘ Θ ∘ ∇	Derived structural axis	Integrated realization of directedness across time	It is generated from impulse, temporality, and integration	Enactment may be over-read as implementation proof or practical validation
D	Dignity-in-Practice	Ψ ∘ X ∘ Ω	Derived structural axis / application constraint	Self-bound reflective restraint and protection in asymmetrical relations	It derives from asymmetry, distance, and self-binding; it is not an ontological dignity operator	D may be misread as moral ranking or governance proof

`PMS.yaml` states that these constructs are systematically derived within PMS and are not treated as independent primitives. It also states that PMS is not a diagnostic tool and must not be used for clinical, forensic, person-ranking, or irreversible person-labeling decisions.

B.3 Derived Drift Pattern Status

Asymmetry drift patterns are derived distortion patterns within PMS. They are not person types, clinical categories, or ontological attributions.

The minimal drift formula used by PMS is:

```
AD = drift(Φ ∘ A ∘ Ω)
```

This means that asymmetry drift arises where asymmetry is stabilized through attractor formation

and recontextualized in ways that reinforce drift rather than support integrative transformation.

Example:

```
AD-A≫E
```

This pattern describes a structural imbalance in which evaluative or awareness-heavy processing over-expands while enactment is suppressed or chronically delayed. It must not be read as a personality diagnosis. It is a scene-bound structural pattern, not a person-level identity.

Under Load reduction:

```
A drift pattern may clarify a scene.
It may not classify a person.
```

B.4 Exclusion Table: Applied Vocabulary Not in PMS Base

The following terms may be important in PMS Under Load, domain papers, add-ons, QC, STACK, or repository work. They are not base operators unless PMS Base or PMS.yaml explicitly version-revises the operator grammar.

Term	Layer Where It Appears	Why Useful	Why Not Base Operator	Risk If Misread as Base
outer legibility	domain / publicness / SEX	Tracks external readability and exposure	Not part of Δ–Ψ inventory	Public perception becomes base grammar
path-cost	domain / conflict / implementation	Tracks cost of maintaining a path	Domain- or implementation-local pressure term	Cost becomes proof of structural truth
Φ-substitution	Under Load / Φ Drift	Names misuse of recontextualization	A drift diagnosis, not a primitive	Drift marker becomes operator
terminality	CONFLICT / domain stress	Marks limit, closure, or non-integrability pressure	Domain-local stress concept	Conflict vocabulary redefines base grammar
PFO	EDEN / domain layer	Useful for naming the paper-internal regime of pseudo-symmetry, comparison drift & reciprocity loss	Domain-local regime label, not a PMS primitive	EDEN vocabulary becomes base grammar
QFRAME	QC / bridge layer	Useful for quantum-computational mapping	Bridge-local macro, not □ itself	Bridge frame becomes universal frame
QPREP	QC / bridge layer	Useful for preparation-state mapping	Bridge-local macro, not PMS primitive	Quantum mapping becomes substrate proof
AH attack surface	AH / hardening layer	Identifies misuse, inference, and artifact risks	Hardening criterion, not operator	Hardening becomes rescue layer
Dignity-in-Practice as application gate	MIP/AH / PMS.yaml guardrail	Prevents shaming, ranking, irreversible labels	D is derived; the gate governs application only	Application discipline becomes base validation
implementation pressure	STACK / repo layer	Tests realizability and architecture stress	Implementation-layer claim	Implementability becomes proof
executable artifact-backed partial realization	STACK / Rust repo	Strengthens bounded implementation claim where implemented	Artifact status, not base operator	Repo becomes validation layer

Under Load already fixes this reduction: QC remains a bridge claim, STACK remains an implementation claim, and MIP/AH remain governance or hardening claims under constraint. None

of these may validate PMS Base.

B.5 YAML Status Note

`PMS.yaml` represents PMS in computational form as a formal encoding of the proposed grammar. It makes the operator inventory, dependency path, derived constructs, and application guardrails machine-readable. It may support structural reasoning, schema validation, simulation scaffolding, documentation, and repository-level coherence.

It does not implement praxis.

It does not prove PMS.

It does not guarantee interoperability, empirical adequacy, public uptake, direct agent implementation, or superiority over rival grammars. Operational use still requires calibration, implementation constraints, context-sensitive judgment, and external accountability. `PMS.yaml` is therefore a repository reference specification and formal representation layer, not a validation layer.

Reduction:

```
Machine-readable grammar is not implemented praxis.
Repository coherence is not epistemic finality.
```

B.6 Boundary Rule

This appendix fixes the operator reference for PMS Under Load.

It permits:

```
reference to Δ–Ψ
use of PMS.yaml dependency path
use of derived axes as derived constructs
use of drift patterns as scene-bound structural descriptions
use of YAML as repository reference specification
```

It forbids:

```
domain terms becoming base operators
derived axes becoming primitives
drift forms becoming person types
MIP/AH becoming PMS Base validation
QC becoming proof
STACK or Rust becoming validation
YAML becoming implementation
self-binding becoming self-validation
```

Final boundary sentence:

Appendix B protects PMS Under Load from operator drift: it fixes what may count as PMS E

Appendix C — Layer / Stack Position Matrix

C.0 Function of This Appendix

This appendix provides the stack-position matrix for the layers used in *PMS Under Load*. Its function is not to rank layers by importance or truth. Its function is to prevent PMS' portability from becoming silent authority borrowing.

Layer discipline prevents PMS' portability from becoming silent authority borrowing.

Because the same PMS operator vocabulary can travel across base theory, domain applications, add-ons, governance hardening, bridge mappings, implementation layers, and repository artifacts, layer continuity can be mistaken for layer equivalence. This appendix fixes the status of each layer before those layers are used in later appendices.

The matrix is not a hierarchy of truth. It is a claim-boundary instrument.

C.1 Core Rule

The governing rule is:

```
A claim becomes unstable when it borrows authority from a layer other than the one in wh
```

This rule applies to all layers.

A domain application may stress PMS Base, but it may not redefine Δ–Ψ. An add-on may harden application, but it may not become a primitive. MIP/AH may constrain use, but it may not validate PMS Base. QC may map structures, but it may not become proof. STACK may intensify implementation pressure, but it may not become validation. Rust or repository artifacts may show bounded executable artifact-backed partial realization where documented, but they may not prove PMS as a final grammar.

The paper may compare layers. It may not let one layer silently authorize another.

C.2 Layer / Stack Position Matrix

Layer	Stack Position	Claim Type	Allowed Function	Not Allowed To Do	Typical Drift Risk	Relevant Chapters / Appendices
PMS Base	Operator source	BASE CLAIM	Define Δ–Ψ, operator identity, default order, dependency structure, base-level non-goals, minimal conditions under which	Claim finality, universal closure, rival-proof priority, or empirical sufficiency	Base overclaim; operator grammar becomes final grammar	0, 3, 4, 11, 12; Appendix B

Layer	Stack Position	Claim Type	Allowed Function	Not Allowed To Do	Typical Drift Risk	Relevant Chapters / Appendices
			PMS remains PMS			
PMS.yaml	Repository reference / formal specification	BASE / SPECIFICATION CLAIM	Provide current schema status, dependency hygiene, derived-axis status, application guardrails, machine-readable constraints	Prove implementation, guarantee interoperability, replace external validation, establish epistemic finality	YAML-as-proof drift; schema-as-implementation drift	0, 4, 11; Appendices A, B, C
Domain Papers	Stress / application layers	DOMAIN CLAIM	Apply and stress PMS operators under domain load; show applied operator behavior, calibration pressure, drift risk, publicness risk, or domain-specific instability	Redefine Δ–Ψ, become new base grammars, decide PMS validity	Domain-as-base drift	2, 3, 5, 6, 8, 9; Appendix D
Add-ons	Precision / application / hardening layers	ADD-ON / HARDENING CLAIM	Sharpen application, standardize outputs, expose attack surfaces, improve usability, reduce misuse risk	Become primitives, repair PMS Base, prove the model, replace operator definitions	Add-on-as-primitive drift; precision-as-authority drift	0, 4, 7, 9, 11; Appendices A, C
MIP/AH	Governance-oriented application discipline and	GOVERNANCE CLAIM / ADD-ON-HARDENING CLAIM	Constrain application; clarify misuse risk, publicness	Validate PMS Base, solve calibration, repair operator	Rescue-layer drift; governance-as-tribunal drift	0, 3, 4, 6, 7, 9, 10, 11; Appendices A, C

Layer	Stack Position	Claim Type	Allowed Function	Not Allowed To Do	Typical Drift Risk	Relevant Chapters / Appendices
	artifact hardening under constraint		risk, no-person-ranking, reversibility, dignity protection, role sensitivity, artifact hardening, claim-typing discipline	ambiguity, become tribunal, rank persons, rescue PMS from structural pressure		
QC	Bridge / structural mapping	BRIDGE CLAIM	Map structures across a formal-technical boundary; test bridge stress; support bounded comparison where mapping preserves difference	Prove substrate identity, establish physical universality, validate PMS Base, replace quantum formalism	Bridge-as-proof drift; mapping-as-equivalence drift	4, 8, 11; Appendix D
STACK	Implementation layer / realization horizon / implementation pressure	IMPLEMENTATION CLAIM / REALIZATION CLAIM	Test realization pressure; show how PMS may be projected into implementation architecture	Validate PMS Base, prove truth by implementation, turn realizability into sufficiency, collapse architecture into ontology	Implementation-as-proof drift; architecture-as-ontology drift	0, 4, 7, 11; Appendix G
Rust / Repository Artifacts	Documented implementation artifact; bounded executable artifact-backed partial realization where documented	IMPLEMENTATION / ARTIFACT CLAIM	Demonstrate bounded executable feasibility, artifact-supported PMS-STACK Evidence Spine v0.1, code structure, schema	Prove PMS Base, prove final grammar, replace external validation, complete PMS 1.3 semantics, treat module boundaries as operator boundaries	Repo-as-validation drift; test-as-proof drift; module-as-operator drift; runtime-dialect-as-theory drift	4, 7, 8, 10, 11, 12; Appendix G

Layer	Stack Position	Claim Type	Allowed Function	Not Allowed To Do	Typical Drift Risk	Relevant Chapters / Appendices
			behavior, tests, artifact traces, demos, documentation, repo-to-chapter relation			
Self-Application Artifacts	Self-exposure / audit material where available	SELF-APPLICATION CLAIM	Document audit strength, artifact vulnerability, dialogic process, hardening history, and how PMS reads itself under pressure	Prove self-validation, provide final self-arbitration, show that PMS has solved its own limit	Self-application-as-settlement drift	10, 11
External Frameworks / Rival Models	External comparison only where explicitly used	SPECULATIVE / OUT OF SCOPE unless sourced and claim-typed	Remain open as possible rivals; clarify comparison if explicitly introduced	Serve as final arbiter, rescue layer, or tribunal against PMS	External-authority drift	0, 1, 11, 12

C.3 Drift Forms by Layer

Layer discipline is necessary because each layer has a characteristic overreach.

Layer	Stable Use	Drift Form
PMS Base	Defines Δ–Ψ and dependency structure	Base grammar becomes final grammar
PMS.yaml	Specifies current repository reference	Schema becomes proof
Domain Papers	Apply and stress PMS under domain load	Domain lens becomes new base
Add-ons	Harden or refine application	Add-on becomes primitive
MIP/AH	Constrain applied use	Governance becomes tribunal
QC	Maps structural relations across bridge conditions	Bridge becomes proof

Layer	Stable Use	Drift Form
STACK	Tests implementation pressure	Implementation becomes validation
Rust / Repo	Shows documented implementation artifact status and bounded executable artifact-backed partial realization where evidenced	Repo becomes base authority
Self-Application Artifacts	Expose PMS reading itself	Self-exposure becomes self-settlement

The drift forms are not merely rhetorical mistakes. They change what kind of authority a claim appears to have.

C.4 Cross-Layer Use Rule

Cross-layer analysis is allowed.

Cross-layer authorization is not.

A layer may place pressure on another layer. A domain paper may expose where the base grammar strains under application. An add-on may reveal recurring misuse risks. MIP/AH may show where application needs stronger gates. QC may test whether bridge mapping preserves difference or over-equates. STACK may show whether PMS becomes more stable under implementation pressure. Rust / repository artifacts may show bounded executable artifact-backed partial realization where documented.

But none of these movements may silently change the status of the claim.

The governing distinctions are:

```
pressure is not proof
mapping is not equivalence
implementation is not validation
governance is not tribunal
repository artifact is not base authority
documented executability is not PMS Base validation
runtime mapping is not theoretical identity
tests are not proof
traces are not external validation
```

If a claim moves across layers, the movement must be explicit, justified, and reduced.

C.5 Boundary to Appendix D and Appendix G

This appendix defines layer position. It does not evaluate the detailed content of every layer.

Appendix D maps chapters to current sources and distinguishes the specific role of domain papers and QC.

Appendix G evaluates the Rust project and repository relation under implementation-boundary

constraints.

Therefore, Appendix C does not claim that all domain papers, bridge materials, repository files, or self-application artifacts are already fully inspected. It only defines what those materials may and may not authorize when they are used.

C.6 Reduction Rule

The matrix reduces PMS rather than securing it.

It does not say that PMS is safe because its layers are named. It says that PMS becomes testable only where its layers remain bounded.

The reduction is:

```
Portability may carry PMS across layers.
It may not make those layers equivalent.
```

For Rust / repository artifacts, the reduction is:

```
Repository evidence may strengthen implementation / artifact claims.
It may not validate PMS Base.
```

A claim that remains strong after its layer is fixed may stand. A claim whose force depends on sliding into another layer must be reduced.

Appendix D — Chapter-to-Source and Domain-Layer Matrix

D.0 Function of This Appendix

This appendix records which source carries which load in *PMS Under Load*. Its purpose is not to prove the manuscript by accumulation of sources. Its purpose is source discipline.

The appendix combines two functions: a chapter-to-source matrix and a domain-layer status table, both restricted to the current supplied source state. The first identifies which sources support which chapter-level pressure zones. The second identifies how the domain papers, QC bridge, and PMS-RUST implementation artifact function as load regimes or boundary evidence.

Earlier revision trails, intermediate drafts, and obsolete formulations are not used as independent authority in this appendix. They may explain how the current source state came to exist, but they do not carry current source status.

The rule is:

```
No source supports every claim.
No domain paper becomes PMS Base.
No bridge mapping becomes proof.
No governance or hardening layer becomes validation.
No repository artifact becomes PMS Base evidence.
```

This appendix therefore prevents “all sources support all claims” drift.

D.1 Chapter-to-Source Matrix

The matrix records current source roles only. It does not reconstruct prior development history.

Chapter	Primary Sources	Secondary / Constrained Sources	Explicitly Excluded Sources	Claim Type	Source Role	Con
0 — Front Matter	PMS Base; PMS.yaml; finished *PMS Under Load* source scope; project method protocol	Domain papers as current source state; MIP/AH under constraint; QC as bridge; STACK as implementation layer; PMS-RUST as source-bound implementation / artifact evidence	unavailable sources; obsolete formulations as current authority; QC as proof; STACK as validation; PMS-RUST as PMS Base validation; MIP/AH as rescue layer	BASE / SELF-APPLICATION / METHOD CLAIM	Fixes source posture, layer discipline, claim typing, MIP/AH constraint, repository boundary, and non-reconstruction rule	Sour stacl sour is a clain

Chapter	Primary Sources	Secondary / Constrained Sources	Explicitly Excluded Sources	Claim Type	Source Role	Con
1 — Stance & Method	Finished *PMS Under Load* chapter body; project method protocol; PMS Base / PMS.yaml	Domain papers as pressure field; dialogic / AI-assisted origin only under self-application constraint	external authority as final arbiter; rival models as tribunal; self-audit as proof	SELF-APPLICATION / METHOD CLAIM	Defines internal critique as operatoric, falsifiable, reversible, non-immunized, and capable of failing	Criti enda mere PMS
2 — What PMS Does Well	PMS Base; PMS.yaml; finished *PMS Under Load* chapter body	Domain papers as evidence of portability and generativity; QC as bridge potential; STACK as implementation pressure; PMS-RUST as bounded artifact-supported implementation evidence where relevant	success as validation; portability as equivalence; implementation as proof; repository evidence as PMS Base validation	BASE / DOMAIN / BRIDGE / IMPLEMENTATION CLAIM, all reduced	Gives PMS a fair baseline before instability begins	Stre as lo reas
3 — Calibration	PMS Base; PMS.yaml; LOGIC; CRITIQUE; CONFLICT; ANTICIPATION; SEX	MIP/AH only as application-gate contrast; QC only if calibration of bridge terms is discussed	STACK as calibration proof; PMS-RUST as calibration proof; EDEN as main evidence	SELF-APPLICATION / DOMAIN CLAIM	Tests threshold dependence: when an operator is sufficiently present, blocked, costly, binding, or decisive	Calib loos form mee judg

Chapter	Primary Sources	Secondary / Constrained Sources	Explicitly Excluded Sources	Claim Type	Source Role	Con
4 — Stack / Interpretation Drift	Finished *PMS Under Load* layer discipline; Appendix A; Appendix C; PMS Base / PMS.yaml; Appendix G for PMS-RUST status	MIP/AH under constraint; QC bridge status; STACK implementation status; PMS-RUST as source-bound implementation / artifact evidence	QC as proof; STACK as validation; PMS-RUST as PMS Base evidence; MIP/AH as tribunal; domain terms as base operators	MIXED, but all claim-typed	Tests whether layer boundaries remain stable under portability, implementation pressure, and repository artifact status	Cros com allov borr forb evid stren impl artif
5 — Φ Drift	PMS Base / PMS.yaml; LOGIC; CONFLICT; EDEN; SEX; ANTICIPATION	CRITIQUE as method contrast where relevant	Φ as universal repair; recontextualization as reversal; non-fit as automatically assimilable; implementation artifacts as Φ proof	BASE / DOMAIN CLAIM	Tests whether recontextualization preserves Δ or absorbs resistant difference	Recc is nc trans pres diffe load
6 — Meta-Normativity	PMS Base / PMS.yaml; LOGIC as main anchor; finished Under-Load chapter body	MIP/AH under constraint; ANTICIPATION as optional selectivity case; SEX / CRITIQUE as applied guardrail cases	moral theory as PMS Base; MIP/AH as justification; responsibility as ought; repository artifacts as normative proof	SELF-APPLICATION / DOMAIN / GOVERNANCE CLAIM	Tests whether PMS is non-prescriptive but structurally selective	PMS pres with neut
7 — Success Trap	Finished *PMS Under Load* chapter body; PMS Base / PMS.yaml; cross-domain outputs; dialogic/AI-assisted production context; Appendix G for PMS-RUST	MIP/AH as trust amplifier under constraint; STACK as implementation-pressure contrast; PMS-RUST as source-bound evidence for reproducibility, traces, tests, demos, gates,	coherence as truth; reproducibility as validation; AI convergence as proof; tests as proof; traces as external validation; PMS-RUST as PMS Base validation	SELF-APPLICATION / IMPLEMENTATION-CONSTRAINED CLAIM	Tests whether PMS-consistency becomes easier to reproduce than difference-sensitive interruption	Stab valic stab incre disc

Chapter	Primary Sources	Secondary / Constrained Sources	Explicitly Excluded Sources	Claim Type	Source Role	Con
	artifact status	and artifact stabilizability				
8 — Domain Drift and Bridge Stress	CRITIQUE; ANTICIPATION; CONFLICT; LOGIC; SEX; EDEN; QC source; finished Chapter 8	PMS Base / PMS.yaml for operator discipline; Appendix C for layer status; Appendix G for PMS-RUST implementation-boundary evidence	MIP/AH as domain layer; STACK as domain layer; PMS-RUST as domain paper; QC as ordinary domain layer; domain vocabulary as base grammar; runtime mapping as theoretical identity	DOMAIN CLAIM / BRIDGE CLAIM / IMPLEMENTATION-BOUNDARY CLAIM	Compares load profiles across domains without ranking them; tests QC as bridge stress; tests PMS-RUST only at implementation boundary	Dom load brid RUS impl bou evid dom equi
9 — Publicness	CRITIQUE; CONFLICT; SEX; EDEN; finished Chapter 9	ANTICIPATION as optional contrast; MIP/AH as public-use constraint; LOGIC minimally if needed	QC; STACK; PMS-RUST as publicness proof; public uptake as truth; publicness as moral failure of audience	DOMAIN / GOVERNANCE-CONSTRAINED CLAIM	Tests how public circulation changes reversibility, depersonalization, X, Ω, Θ, Φ, and A	Publ expo conc truth
10 — Self-Application Limit	Finished Chapter 10; LOGIC; MIP/AH under constraint; dialogic / AI-assisted self-application context; Appendix G for PMS-RUST artifact auditability	PMS-RUST as source-bound evidence for traces, gates, AI bridge boundaries, golden fixtures, docs, tests, and artifact inspectability	revision trails as current source authority; self-application as final arbitration; artifact hardening as self-endangerment proof; traceability as self-arbitration	SELF-APPLICATION CLAIM / IMPLEMENTATION-CONSTRAINED CLAIM	Tests whether PMS can expose but not fully settle its own limits; tests whether artifact auditability becomes loss condition or only hardening	Self- not s artifa is nc enda
11 — What Survives	All previous Under-Load chapter outputs; PMS Base / PMS.yaml; Chapter 11	QC as bridge status; STACK as implementation status; PMS-RUST as source-bound	new validation layers; new sources; new proof burden; survival as triumph; working repo as proof	SYNTHESIS / REDUCED SELF-APPLICATION CLAIM / REDUCED IMPLEMENTATION CLAIM	States what survives only after reductions have been paid	Wha inclu bou impl clain impl

Chapter	Primary Sources	Secondary / Constrained Sources	Explicitly Excluded Sources	Claim Type	Source Role	Con
	reductions; Appendix G for PMS-RUST status	implementation / artifact evidence; MIP/ AH under constraint				pres clain artif feasi
12 — Conclusion	All preceding chapter results; finished conclusion; reduced survival thesis; Appendix G for implementation-boundary status	Domain, bridge, governance, implementation, repository, and self-application layers only in reduced status	final grammar claim; reassurance; implementation proof; repository proof; governance proof; bridge proof; self-arbitration proof	CONCLUSION / REDUCED POSITION CLAIM	Closes the reduced position: PMS survives as bounded, self-limiting grammar of praxis legibility with bounded implementation evidence where documented	Final mus strer Und redu

D.2 Domain-Layer Status Table

The following table compares load profiles, not paper quality. It does not rank the domain papers. Each paper is treated as a load regime for PMS.

Paper / Layer	Stack Position	Operators under Load	Main Strength	Main Instability	Under Load Relevance	Source Status
CRITIQUE	Domain layer / transition hinge	Χ, Φ, Σ, Ψ, Ω, Θ; critique as interruption and correction	Provides a comparatively stable baseline for operatoric critique: mismatch, interruption, recontextualization, correction, and follow-up without person-ranking	May become grammar-closure if every critique is translated into PMS-valid critique, critique drift, or correction failure	Tests whether PMS can be endangered by critique rather than merely confirmed by critique	Current supplied paper used as source
ANTICIPATION	Domain layer / upstream openness case	Λ, Θ, Ω, Χ, Σ, Ψ	Makes future-oriented openness structurally legible without reducing anticipation to prediction, control, strategy, or action advantage	May drift into prediction, control fantasy, premature closure, action compulsion, or Ψ-externalization	Tests whether PMS can carry future-openness without possessing the future	Current supplied paper used as source

Paper / Layer	Stack Position	Operators under Load	Main Strength	Main Instability	Under Load Relevance	Source Status
CONFLICT	Domain layer / major knot / terminality pressure	Σ, Ψ, Ω, Θ, X; also Φ-substitution, Λ, A under conflict stabilization	Makes stabilized non-integrability legible without reducing conflict to error, escalation, communication failure, or immaturity	May prematurely stabilize non-integrability; Φ may substitute for Σ; terminality may become too easy to name	Tests whether PMS can respect non-integrability without turning it into solution failure or premature closure	Current supplied paper used as source
LOGIC	Domain layer / boundary and meta-normative collision	Ω, Θ, Λ, Ψ, □, X, Σ; responsibility without ought	Distinguishes responsibility, attributability, capacity, consequence, and moral readability without deriving normativity	Attribution may flatten into moral implication; responsibility without ought may become hidden normativity	Tests whether PMS can make responsibility structurally legible without deriving an ought	Current supplied paper used as source
SEX	Domain layer / high-Ω and high-exposure stress	Ω, Θ, Λ, A, X, Σ, Ψ; outer legibility; publicness overlay	Shows whether PMS can name hard configuration-level forms under high asymmetry, exposure, internal/outer divergence, and publicness without shaming or ranking persons	May drift into shame, identity judgment, pathologizing, tribunal use, or person-ranking if outer legibility is mishandled	Tests whether PMS can say hard things about configurations without turning configuration analysis into person-level harm	Current supplied paper used as source

Paper / Layer	Stack Position	Operators under Load	Main Strength	Main Instability	Under Load Relevance	Source Status
EDEN	Domain layer / symbolic and translation-dominance stress	Δ, □, Θ, Ω, Λ, A, Σ, Ψ; comparison drift; pseudo-symmetry; PFO	Shows high translation capacity across dense symbolic, theological, gendered, and reciprocity material while maintaining non-theological, non-person-evaluative constraints	PMS may translate rival theological, symbolic, or gendered readings too smoothly into its own grammar, producing interpretive finality or translation dominance	Tests whether PMS can read sensitive asymmetry without turning translation power into final interpretation	Current supplied paper used as source
QC / PMS-QC	Bridge Stress, not ordinary domain layer	□, Δ, Ω, Θ, A, Φ, Σ, Λ, Ψ through formal mapping and QC macros such as QFRAME, QPREP, QORACLE_CALL, QITERATE, QMEASURE, QSTABILIZE	Tests whether PMS can map formal-technical quantum-computational structure without replacing quantum formalisms or claiming physical proof	Mapping may be misread as physical explanation, substrate-universal validation, proof, or replacement of quantum formalism	Tests whether PMS can map formal structure without converting mapping into proof	Current supplied QC paper used as bridge-status source

CRITIQUE defines critique as a praxeological operation, not moral attitude, psychological trait, or truth-claim by itself, and treats critique drift without invoking motives or moral rankings. ANTICIPATION defines anticipation as present structural stance that carries Λ under Θ and Ω with X, Σ, and Ψ, rather than as forecast or action mandate. CONFLICT positions itself as a domain layer focused on stabilized incompatibility, with Σ, Ψ, Ω, Θ, and X under load. LOGIC reconstructs responsibility without ought by distinguishing responsibility as consequence-attribution from ought as prescriptive demand. SEX distinguishes internal framing from outer legibility and treats publicness as an overlay that amplifies and sediments readability without necessarily creating it. EDEN treats Eden as text-as-praxis and develops pseudo-symmetry, PFO, and reciprocity loss as structural consequence patterns without adding operators or moral judgment. QC explicitly says PMS is not an alternative quantum formalism and does not replace unitaries, Hamiltonians, measurement theory, ZX-calculus, tensor networks, or categorical models.

D.3 MIP/AH Boundary Note

MIP/AH does not appear as a domain-layer row in D.2.

The reason is structural:

```
MIP/AH is governance / hardening under constraint, not a domain layer.
```

MIP/AH may constrain public use, application boundaries, no-person-ranking, misuse risk, artifact hardening, reversibility, and claim typing. It may not become evidence that PMS Base is valid, that a domain paper is correct, or that a public application is safe.

The boundary sentence is:

```
Domain drift concerns grammar behavior under domain load; downstream governance may cons
```

D.4 QC Bridge-Stress Note

QC is included in Appendix D because it appears in Chapter 8, but it is not treated as a seventh ordinary domain paper.

The boundary sentence is:

```
QC = Bridge Stress, not ordinary domain layer.
```

The QC paper makes PMS structurally useful for quantum-computational mapping, macro-language, and workflow annotation, but the mapping remains structural, interpretive, and non-exclusive. PMS-QC macros serve as documentation, compilation anchors, and audit handles for Δ–Ψ chains rather than replacements for circuit formalisms.

The reduction is:

```
QC mapping may show bridge reach.
It may not show physical proof.
```

D.5 PMS-RUST Implementation-Boundary Note

PMS-RUST is included in Appendix D because it now appears in Chapters 4, 7, 8, 10, 11, and 12 as implementation-boundary evidence.

It is not a domain paper.

It is not QC proof.

It is not PMS Base.

The boundary sentence is:

```
PMS-RUST = source-bound implementation / artifact evidence, not PMS Base validation.
```

PMS-RUST may support chapter claims about:

```
documented implementation artifact status;
bounded executable artifact-backed partial realization;
PMS-STACK Evidence Spine v0.1;
runtime / toolchain structure;
PMS.yaml model-validation relation;
runtime / theory / symbol normalization;
REPL / script execution;
JSONL traceability;
tests;
golden fixtures;
demos;
acceptance gates;
AI bridge boundaries;
repo-to-chapter relation.
```

PMS-RUST may not support chapter claims that:

```
PMS Base is validated;
PMS is the final grammar of praxis;
PMS 1.3 semantics are complete in code;
tests prove PMS;
traces provide external validation;
runtime dialect is theoretical identity;
model validation is praxis adequacy;
repository evidence replaces external application.
```

The reduction is:

```
PMS-RUST may show bounded artifact-supported implementation feasibility.
It may not show PMS Base truth.
```

D.6 Anti-Ranking Rule

The domain-layer table compares load profiles only.

It does not rank:

```
importance
truth
sensitivity
difficulty
maturity
centrality
paper quality
```

A domain may be high-load without being “better.” A domain may be baseline-stable without being shallow. A bridge may be formally technical without being proof-bearing. A sensitive domain may be structurally valuable while remaining risky in public use. A repository artifact may be implementation-relevant without being theory-validating.

The rule is:

```
Compare load profiles.
Do not rank papers.
Do not rank artifacts as proof.
```

D.7 Current-Source Status

This appendix uses only the current supplied paper versions and current supplied implementation evidence as source material.

Earlier revision trails, intermediate drafts, and obsolete formulations are excluded from Appendix D. They may document the historical development of a paper, but they do not define the current source status used by *PMS Under Load*.

PMS-RUST is no longer pending in Appendix D. It is treated as current source-bound implementation evidence through Appendix G, not as proof-bearing source authority.

This exclusion and inclusion pattern is methodological, not dismissive. Appendix D is not a genealogy of revisions. It is a current-source matrix.

The working rule is:

```
Current papers carry current source status.
Current PMS-RUST evidence carries implementation / artifact source status.
No current source carries authority beyond its layer.
```

D.8 Reduction Rule

Appendix D reduces the source posture of *PMS Under Load* by binding each chapter and domain layer to the current supplied source state.

It does not say:

```
The domain papers prove PMS.
```

It says:

```
The domain papers show where PMS is placed under different kinds of load.
```

It does not say:

```
QC proves PMS across formal systems.
```

It says:

```
QC tests whether bridge mapping can remain mapping without becoming proof.
```

It does not say:

```
PMS-RUST proves PMS through implementation.
```

It says:

```
PMS-RUST supplies source-bound implementation / artifact evidence for bounded executable
```

It does not say:

```
MIP/AH secures domain use.
```

It says:

```
MIP/AH may constrain use, but does not erase domain drift.
```

Final boundary sentence:

```
Appendix D protects PMS Under Load from source inflation: each source may carry only the
```

This appendix does not preserve revision history as authority. It preserves current source discipline.

Appendix E — Falsifier Index

E.0 Function of This Appendix

This appendix collects the falsifiers for the instability chapters of *PMS Under Load* and adds the global survivor criterion.

Its function is not decorative. A falsifier is included only if it can weaken the relevant objection. If a critique cannot be weakened, it has become coverage. It no longer places PMS under load; it merely absorbs possible resistance into PMS-readable form.

The governing sentence is:

```
A critique that cannot be weakened is not Under Load; it is coverage.
```

This appendix therefore keeps the instability sequence non-immunized.

E.1 Falsifier Standard

A falsifier in *PMS Under Load* must be:

```
structural
plausible
non-trivial
specific to the chapter
capable of weakening the objection
not already neutralized by the chapter's own vocabulary
```

A weak falsifier would say:

```
This chapter would be false if PMS were false.
```

That does not test the chapter. It only shifts the burden to PMS as a whole.

A valid falsifier says what structural condition would show that the specific instability does not hold, does not hold strongly, or has been overdrawn.

The test is:

```
What would PMS have to preserve, generate, distinguish, or withstand for this objection
```

E.2 Falsifier Index Table

Instability	Core Claim	Falsifier	What Would Weaken the Objection	What Would Strengthen the Objection	Relevant Chapter
Calibration	PMS remains formally disciplined, but its operational thresholds are not fully endogenous.	If PMS can generate stable, non-arbitrary, internally derived thresholds across calibration-sensitive cases without outside calibration input, the calibration objection weakens.	PMS repeatedly converges on the same threshold judgments across difficult scenes while preserving operator discipline, reversibility, non-person-ranking, and claim typing.	PMS analyses remain operator-valid but diverge materially on thresholds such as "enough X," "decisive Ω," "active Λ," "stable Ψ," or "publicly admissible naming."	Chapter 3
Stack Drift	PMS can remain internally coherent while its layer boundaries become externally unstable.	If PMS-derived outputs remain interpretable, reversible, claim-typed, and operator-strict across layers without authority borrowing, stack drift does not occur.	PMS Base, domain papers, add-ons, MIP/AH, QC, STACK, YAML, and PMS-RUST / repo artifacts remain cleanly separated in use, and no layer silently borrows authority from another. PMS-RUST counts as implementation pressure and artifact stabilization only where artifact evidence preserves layer discipline.	Domain vocabulary begins to act like base grammar; MIP/AH becomes tribunal; QC becomes proof; STACK or PMS-RUST becomes validation; YAML becomes implementation; tests, traces, or repository structure become PMS Base evidence.	Chapter 4
Φ Drift	Φ becomes unstable where recontextualization stops preserving Δ and starts making non-fit compatible.	If PMS can reliably distinguish clarifying recontextualization from absorptive recontextualization while preserving Δ, the Φ drift objection weakens.	Recontextualization repeatedly increases differentiation, preserves resistant difference, and leaves genuine non-fit visible rather than smoothing it into PMS compatibility.	Φ makes incompatible readings compatible too easily; non-fit becomes residual Λ, delayed integration, missing frame, or re-description rather than resistant difference.	Chapter 5

Instability	Core Claim	Falsifier	What Would Weaken the Objection	What Would Strengthen the Objection	Relevant Chapter
Meta-Normativity	PMS is non-prescriptive, but structurally privileges certain forms of practice over others.	If PMS applications without X, reversibility, D, non-person-ranking, and misuse resistance could remain equally admissible, the meta-normativity objection weakens.	PMS can apply itself with equal validity to practices lacking distance, reversibility, Dignity-in-Practice, scene-boundness, and misuse resistance without treating them as drift, invalid application, or structural failure.	PMS repeatedly treats X, reversibility, D, self-binding, non-person-ranking, accountable direction, and misuse resistance as conditions of valid or stronger praxis while denying that this is selective.	Chapter 6
Success Trap	It becomes easier to produce PMS-consistent outputs than to detect where PMS stops differentiating.	If increased stability does not reduce sensitivity to difference, the success trap does not occur.	More coherent, reproducible, AI-stabilized, guardrailed, and artifact-stabilized PMS outputs also become more sensitive to non-fit, rival readings, resistant Δ, Λ interruption, and genuine non-capture. PMS-RUST weakens the objection only if traces, tests, demos, gates, golden fixtures, and repository artifacts increase discrimination rather than merely stabilizability.	PMS-consistency becomes easier to reproduce while resistant difference becomes easier to absorb; stability, elegance, guardrail discipline, tests, traces, demos, and artifact reproducibility increase while interruption capacity decreases.	Chapter 7
Domain Drift and Bridge Stress	Domain reach is load, not equivalence; bridge reach is mapping pressure, not proof.	If the same scene can be processed through different domain lenses with comparable differentiation quality, clear boundaries, no systematic drift, and no silent operator inflation, the domain drift objection weakens. QC-	CRITIQUE, ANTICIPATION, CONFLICT, LOGIC, SEX, EDEN, and QC each preserve layer boundaries, local vocabulary status, operator discipline, and claim type while still producing useful	A domain marker becomes a base concept; a domain lens totalizes a scene; high-risk domain readings authorize person-level judgment; EDEN turns translation into	Chapter 8

Instability	Core Claim	Falsifier	What Would Weaken the Objection	What Would Strengthen the Objection	Relevant Chapter
		specific: PMS-QC remains useful while staying non-proof, non-substrate-universal, and non-replacement of quantum formalism. PMS-RUST-specific: implementation evidence remains implementation-boundary evidence without becoming domain equivalence or theoretical identity.	readings or mappings. PMS-RUST preserves implementation-boundary discipline: runtime mapping is not theoretical identity, model validation is not praxis adequacy, and repository evidence is not PMS Base validation.	final interpretation; QC becomes physical or substrate-universal proof; PMS-RUST runtime mapping becomes theory identity or repository evidence becomes base validation.	
Publicness	Publicness does not merely scale PMS application; it transforms the conditions under which PMS remains reversible, depersonalized, and differentiating.	If increased publicness does not reduce reversibility, depersonalization, X-maintenance, or differentiation quality, publicness does not introduce structural instability.	PMS claims retain scene-boundness, non-person-ranking, reversibility, depersonalization, and differentiation quality at higher publicness levels without requiring qualitatively stronger hardening.	Public circulation accelerates A-stabilization, compresses Θ, weakens X, weaponizes Φ, fixes labels, increases person-nearness, or turns configuration claims into identity anchors.	Chapter 9
Self-Application Limit	PMS can expose its own limits, but cannot fully arbitrate them without remainder.	If PMS can fully and non-circularly arbitrate competing self-readings under its own grammar, the self-application limit weakens. Artifact-specific: if PMS-RUST auditability can produce genuine loss conditions for PMS claims rather than merely implementation repair, the artifact-auditability objection weakens.	PMS can adjudicate competing interpretations of itself without circular authority, without external remainder, without merely restabilizing its own grammar, and without converting vulnerability into further PMS-legible strength. PMS-RUST traces, gates, tests, docs, AI bridge boundaries, or fixtures can force claim	PMS produces powerful self-critiques that remain internally coherent, operator-aware, and stabilizing, but cannot generate conditions under which a rival self-reading could defeat PMS' arbitral control. PMS-RUST exposes mismatches, but every mismatch is	Chapter 10

Instability	Core Claim	Falsifier	What Would Weaken the Objection	What Would Strengthen the Objection	Relevant Chapter
			reduction rather than merely artifact hardening.	absorbed as implementation repair rather than claim reduction.	
Global Survivor Criterion	PMS survives only if it preserves discriminative force under the pressures that make it powerful.	If PMS can preserve discriminative force under calibration variation, domain transfer, public exposure, AI-stabilization, artifact stabilization, and self-application without stack drift or coverage substitution, the Under Load objection weakens.	PMS remains bounded, claim-typed, interruptible, rival-sensitive, non-immunized, and open to genuine non-capture while retaining operatoric legibility, cross-domain usefulness, and bounded implementation feasibility. PMS-RUST counts positively only if artifact evidence preserves layer discipline and discrimination.	PMS' strengths—coherence, portability, translation power, guardrails, implementation pressure, PMS-RUST artifact stabilization, and self-application—begin to hide loss of discrimination, layer drift, public irreversibility, repo-as-validation, test-as-proof drift, or coverage substitution.	Chapters 11–12

E.3 How to Read the Falsifiers

The falsifiers are not demands that PMS fail. They specify what PMS would have to preserve for each instability claim to weaken.

A falsifier does not say:

```
PMS must be rejected if this condition appears.
```

It says:

```
This objection loses force if PMS can sustain the named discriminative capacity under th
```

This matters because *PMS Under Load* does not test whether PMS is simply false. It tests where PMS' success stops being informative. A falsifier therefore targets success under pressure, not collapse under weakness.

For example:

```
The calibration objection weakens if PMS can derive thresholds without importing thresho
```

```
The stack-drift objection weakens if PMS-RUST remains implementation / artifact evidence

The success-trap objection weakens if increased stability, including artifact stabilizat

The self-application objection weakens if PMS can create non-circular conditions under w
```

Each falsifier is therefore a pressure condition, not a decorative caveat.

E.4 Non-Immunization Rule

No falsifier may be neutralized by redescribing its failure as further evidence for PMS.

The following moves are not allowed:

```
If threshold divergence occurs, call it calibration richness.
If stack drift occurs, call it layer complexity.
If PMS-RUST is mistaken for validation, call it implementation maturity.
If Φ absorbs non-fit, call it deeper recontextualization.
If meta-normativity appears, call it merely application discipline.
If success stabilizes PMS, call it proof of maturity.
If artifact reproducibility stabilizes PMS, call it proof of implementation truth.
If domain drift appears, call it portability.
If publicness hardens claims, call it reach.
If self-application restabilizes PMS, call it critique-readiness.
```

Such moves would convert falsifiers into coverage.

The permitted move is narrower:

```
Name the pressure.
Mark the reduction.
Ask whether discriminative force survived.
```

For PMS-RUST, the permitted move is:

```
Name the artifact evidence.
Keep it at the implementation / artifact layer.
Ask whether it preserved layer discipline and discrimination.
```

E.5 Reduction Rule

Appendix E reduces the critique by making it vulnerable.

It does not make PMS safer.

It makes the objections testable.

PMS-RUST counts as implementation pressure and artifact stabilization. It weakens objections only

if artifact evidence preserves layer discipline and discrimination.

The final rule is:

```
Every instability claim must be strong enough to matter and vulnerable enough to weaken.
```

If PMS can satisfy these falsifiers under load, the critique must narrow.

If PMS cannot satisfy them, the reduced position of Chapter 11 and Chapter 12 remains necessary.

Appendix F — Drift / Coverage / Limit Lexicon

F.0 Function of This Appendix

This appendix defines recurring terms used in *PMS Under Load*. Its purpose is readability and internal consistency.

The lexicon does not introduce new theory.

It does not add operators.

It does not expand PMS' coverage.

It fixes how already-used terms should be read so that the critique remains discriminating rather than merely fluent.

The governing rule is:

```
Definitions must preserve discrimination, not expand coverage.
```

F.1 Lexicon Use Rule

Terms in this appendix are control terms.

They may clarify chapter claims, prevent drift, and reduce ambiguity. They may not become new primitives, substitute operators, hidden add-ons, or domain authority.

A term becomes unstable if it starts doing more than the chapter permits.

For example:

```
coverage must not become validation
portability must not become equivalence
bridge stress must not become proof
implementation pressure must not become truth
artifact stabilization must not become validation
governance discipline must not become tribunal
non-capture must not become another neat PMS category
```

The lexicon therefore has a negative function: it prevents recurring terms from becoming stronger than their assigned claim type.

F.2 Drift / Coverage / Limit Lexicon

Term	Short Definition	Primary Risk	Not To Be Confused With	Relevant Chapters
coverage	The capacity of PMS to render material legible within its grammar.	Coverage may replace discrimination.	Adequate capture, validation, or proof.	1, 2, 5, 7, 11
discrimination	The capacity to preserve relevant difference, rival pressure, and non-fit after PMS has made something legible.	Discrimination may be mistaken for mere coherence.	Smooth explanation, operatoric readability, or internal consistency.	1, 2, 3, 5, 7, 11, 12
absorption	The process by which resistant material becomes PMS-readable without retaining enough pressure on PMS.	Resistance becomes internal variation too quickly.	Clarification, integration, or successful recontextualization.	1, 2, 5, 7, 11
translation dominance	The risk that PMS translates rival, symbolic, theological, gendered, or domain-specific material so smoothly that translation appears to settle interpretation.	Translation power becomes final authority.	Structural legibility, domain application, or useful comparison.	5, 8, 9, 11
Φ-overfunction	A condition where recontextualization works too smoothly, making non-fit compatible instead of preserving Δ.	Reframing erases the difference it should carry.	Legitimate Φ, clarification, or learning.	2, 5, 7, 11
stack drift	Status confusion between PMS Base, domain layers, add-ons, MIP/AH, QC, STACK, YAML, or repo artifacts.	A claim borrows authority from the wrong layer.	Cross-layer comparison or disciplined portability.	0, 4, 8, 11
success trap	The risk that PMS-consistency becomes easier to reproduce than sensitivity to resistant difference.	Stabilizability becomes mistaken for validation.	Productivity, coherence, or genuine generativity.	1, 2, 7, 10, 11
publicness load	The transformation of claim conditions when PMS analysis circulates beyond a bounded scene.	Public exposure reduces reversibility, depersonalization, X-maintenance, or differentiation quality.	Audience size alone, popularity, or truth conditions.	4, 7, 9, 11
domain drift	The pressure by which a domain application begins to change the grammar, status, or authority of the operators it applies.	Domain vocabulary becomes base grammar.	Legitimate domain stress or application richness.	4, 8, 11; Appendix D

Term	Short Definition	Primary Risk	Not To Be Confused With	Relevant Chapters
bridge stress	The pressure generated when PMS maps structures across a formal or technical boundary, especially QC.	Mapping becomes proof or equivalence.	Domain application, analogy, or physical validation.	4, 8, 11; Appendix D
genuine non-capture	A case where resistance to PMS is not reducible to drift, misapplication, calibration residue, stack confusion, or residual Λ.	PMS may absorb non-capture as another internal category.	Temporary lack of calibration, poor application, or missing Φ.	0, 1, 2, 5, 11, 12
self-arbitration limit	The limit that appears when PMS can expose its own constraints but cannot finally decide all competing readings of itself without remainder.	Self-exposure becomes self-settlement.	Self-application, audit strength, or methodological reflexivity.	1, 7, 10, 11, 12
portability	PMS' ability to travel across domains, layers, or bridges while retaining operatoric continuity.	Portability becomes mistaken for equivalence or final generality.	Validation, universality, or domain identity.	0, 2, 4, 8, 11
equivalence	A stronger claim that two domains, layers, mappings, or structures have the same status or authority.	Equivalence is inferred merely from shared PMS vocabulary.	Portability, analogy, bridge mapping, or application.	2, 4, 8, 11, 12
implementation pressure	The load introduced when PMS is projected toward architecture, system design, execution, or STACK realization. In PMS-RUST, this pressure becomes concrete as a documented implementation artifact rather than merely speculative realization pressure.	Implementation pressure becomes proof of PMS Base.	Implementation claim, feasibility pressure, bounded artifact evidence, or research-program strength.	0, 4, 7, 11; Appendix G
executable artifact-backed partial realization	A bounded implementation-layer status where PMS-derived components exist as executable artifacts under constraints. In PMS-RUST, this refers to the documented PMS-STACK Evidence Spine v0.1 and its bounded runtime/toolchain evidence.	Executability becomes validation of PMS Base.	Prototype evidence, implementation feasibility, artifact trace, test passage, or repo existence.	0, 4, 7, 8, 10, 11, 12; Appendix G

Term	Short Definition	Primary Risk	Not To Be Confused With	Relevant Chapters
bridge claim	A claim that PMS can map or structurally render relations across a boundary, such as QC, without proving identity.	Bridge claim becomes proof claim.	Base claim, domain claim, or implementation claim.	0, 4, 8, 11; Appendices C, D
realization claim	A claim about possible, partial, documented, or actual implementation of PMS-derived structures. PMS-RUST gives this claim artifact support where documented, but only at the implementation / artifact layer.	Realization becomes truth or final adequacy.	Base validity, empirical proof, universal implementation, or repository validation.	0, 4, 7, 11; Appendices C, G
governance-as-tribunal drift	The drift by which application discipline or governance begins to judge persons, rank maturity, or decide truth.	Governance becomes moral or epistemic authority.	Misuse prevention, MIP/AH constraint, or public-use discipline.	0, 4, 6, 9, 11
precision-as-authority drift	The drift by which add-on precision, schema detail, or hardened output format begins to function as base authority.	Operational clarity becomes proof.	Specification, hardening, or artifact discipline.	0, 4, 7, 11
rescue-layer drift	The drift by which MIP/AH, governance, add-ons, QC, STACK, or implementation artifacts are used to shield PMS Base from critique.	Downstream discipline cancels base pressure.	Legitimate application constraint or layer-specific support.	0, 4, 6, 7, 9, 11
repo-as-validation drift	The misreading of PMS-RUST, repository implementation, tests, traces, demos, gates, or executable artifact-backed partial realization as proof of PMS Base.	Implementation artifacts become base authority.	Bounded implementation / artifact evidence, source-bound implementation evidence, or research-program feasibility.	4, 7, 8, 10, 11, 12; Appendix G
runtime-dialect-as-theory drift	The drift by which runtime names, opcodes, module boundaries, symbol normalization, or implementation dialect are treated as identical to PMS 1.3 theory.	Implementation vocabulary silently rewrites PMS Base.	Runtime mapping, model validation, parser behavior, or implementation-local naming.	4, 8, 11; Appendix G
test-as-proof drift	The drift by which passing tests, golden fixtures, or acceptance gates are read as proof of PMS	Conformance evidence becomes epistemic validation.	Regression testing, implementation discipline, gate hygiene,	7, 10, 11; Appendix G

Term	Short Definition	Primary Risk	Not To Be Confused With	Relevant Chapters
	Base rather than artifact conformance.		or bounded artifact evidence.	
trace-as-validation drift	The drift by which JSONL traces or audit outputs are read as external validation rather than inspectability of runtime behavior.	Traceability becomes self-arbitration or external warrant.	Runtime auditability, artifact inspection, or execution record.	7, 10, 11; Appendix G

F.3 Boundary Rule

The terms in this appendix are not interchangeable.

The following distinctions must remain active:

```
coverage ≠ discrimination
portability ≠ equivalence
bridge ≠ proof
implementation ≠ validation
executable artifact ≠ PMS Base proof
repo evidence ≠ truth-status
runtime mapping ≠ theoretical identity
tests ≠ proof
traces ≠ external validation
governance ≠ tribunal
precision ≠ authority
self-exposure ≠ self-arbitration
translation ≠ capture
recontextualization ≠ reversal
publicness ≠ truth
```

If these distinctions collapse, the lexicon stops preserving discrimination and begins expanding PMS coverage.

The most important boundary is:

```
A term that explains too much has begun to drift.
```

F.4 Reduction Rule

Appendix F reduces PMS by restricting its vocabulary.

It does not make PMS more expansive.

It makes PMS easier to interrupt.

A term may remain in the paper only if it helps preserve a distinction. If it begins to make PMS smoother, safer, or more comprehensive without paying a reduction cost, it should be narrowed, moved to a specific layer, or removed.

For PMS-RUST terminology, the rule is:

```
Repository terms may strengthen implementation / artifact claims.
They may not strengthen PMS Base.
```

The final rule is:

```
Lexical clarity is admissible only where it increases vulnerability to correction.
```

Appendix G — Rust Project, Repository Relation, and Implementation Boundary

G.0 Function of This Appendix

This appendix connects *PMS Under Load* to the Rust project and repository relation without converting implementation into proof.

Its function is not technical triumph. It is implementation-boundary discipline.

The main text permits STACK to appear as an implementation-layer claim: realization horizon, implementation pressure, and, where components have been implemented, executable artifact-backed partial realization. This strengthens the implementation layer only.

It does not validate PMS Base.

The purpose of Appendix G is therefore to make the implementation relation inspectable while preventing repo-as-validation drift.

G.1 Scope Statement

The repository is an implementation artifact and research bridge, not a validation layer for PMS Base.

More strictly:

```
The Rust project can support an IMPLEMENTATION CLAIM about bounded executable feasibilit
```

This appendix therefore treats any future Rust, crate, module, test, README, or execution trace as implementation-layer evidence only.

Such evidence may show that some PMS-derived or STACK-derived structures can be represented, organized, executed, tested, or inspected under technical constraints.

It may not show that PMS is final, complete, empirically adequate, uniquely ordered, or epistemically privileged.

G.2 Current Source Status

Repository evidence is now available as public project documentation.

The repository used for this appendix is:

```
PMS-RUST
https://github.com/tz-dev/PMS-RUST
```

The repository describes PMS-RUST as “PMS-STACK Evidence Spine v0.1”: a small, test-driven runtime/toolchain that demonstrates an executable PMS-STACK Evidence Spine. It documents a deterministic Δ–Ψ kernel, REPL/script tooling, model validation, JSONL trace/audit output, a vFS service surface, reproducible demos, golden tests, and a controlled AI proposal bridge.

The documented execution path is:

```
.pms script / REPL / AI proposal
→ PmsBuilder / OpCode buffer
→ pms-model-data/PMS.yaml model validation
→ stateful kernel validation
→ deterministic kernel execution
→ JSONL trace/audit
→ vFS service layer
→ optional AI analyze
→ golden tests + demos + docs
```

This is evidence for an implementation-layer artifact, not for PMS Base validation. The repository itself states that the release is intentionally bounded and is not a full PMS-OS, PMS-CPU/ISA, PMSL compiler, or distributed PMS runtime. It also states that AI output is never trusted executable code: AI may propose PMS opcode programs, while host-side parsing, canonicalization, mapping, validation, and the kernel decide what can execute.

The repository currently supplies:

```
repository URL
README
workspace / crate layout
architecture notes
acceptance-gate documentation
demo scripts
test / lint / format gate documentation
REPL and script runner documentation
JSONL trace / audit documentation
AI bridge trust-boundary documentation
PMS.yaml model-validation relation
```

The repository documents the following current v0.1 baseline:

```
cargo fmt --all -- --check                                  PASS
cargo test --workspace                                      PASS
cargo clippy --workspace --all-targets -- -D warnings       PASS
workspace tests: 157 passing
```

This appendix treats those as repository-documented gate claims, not independently reproduced test results.

The current status is therefore:

```
Repository relation: documented public implementation artifact.
Appendix role: implementation-boundary and evidence-mapping appendix.
```

```
Claim status: IMPLEMENTATION / ARTIFACT CLAIM.
```

The repository may now support the claim of bounded executable artifact-backed partial realization of a PMS-STACK Evidence Spine v0.1.

It may not support a BASE CLAIM about PMS' final validity.

G.3 Repo-to-Chapter Matrix

This table records how repository evidence may be used in *PMS Under Load*. It is now populated with public PMS-RUST v0.1 repository evidence, but it does not convert repository evidence into PMS Base authority.

Chapter / Claim	Relevant PMS Layer	Relevant Repo Module / Crate / Folder	Implementation Status	What the Repo Demonstrates	What It Does Not Demonstrate	Clai
Chapter 4 — Stack / Interpretation Drift	STACK / implementation layer	`Cargo.toml`; `pms-kernel`; `pms-model`; `pms-ir`; `pms-repl`; `pms-ai-bridge`; `pms-docs`; `tests`	documented v0.1 implementation artifact	PMS-derived structures can be separated into runtime kernel, model validation, IR builder, REPL/ script tooling, AI proposal bridge, docs, demos, and tests under a claim-typed implementation architecture	PMS Base validation; proof by implementation; layer equivalence; full PMS-STACK	IMPLEME CLAIM
Chapter 7 — Success Trap	STACK / artifact stabilization	`pms-kernel`; `pms-repl`; `pms-docs/golden`; `pms-demos`; `tests`; `test.bat`; acceptance gates	repository-documented reproducibility, traceability, demos, and gate structure	Artifact structure can increase reproducibility, consistency, trace inspection, demo replay, and stabilizability	Truth; final adequacy; proof that resistant difference is preserved; proof that PMS avoids the success trap	IMPLEME / SELF-APPLICA CONSTR CLAIM
Chapter 8 — Domain Drift and Bridge Stress	bridge / implementation boundary where relevant	`pms-model-data/ PMS.yaml`; `pms-model`; runtime/theory/ symbol key normalization; `pms-`	bounded model-validation and bridge-mapping implementation	PMS model constraints can be loaded, normalized across	QC proof; domain equivalence; operator redefinition; full	BRIDGE / IMPLEME CLAIM

Chapter / Claim	Relevant PMS Layer	Relevant Repo Module / Crate / Folder	Implementation Status	What the Repo Demonstrates	What It Does Not Demonstrate	Clai
		`docs/ARCHITECTURE.md`; `pms-docs/KEY_WORLDS.md`		symbols/theory names/runtime aliases, and used for adjacency validation in a runtime/ toolchain context	PMS 1.3 semantic migration	
Chapter 10 — Self-Application Limit	self-application / artifact audit where relevant	JSONL trace/audit output; `pms-docs/golden/fixtures`; `pms-ai-bridge`; `pms-docs/AI_BRIDGE.md`; acceptance gates	artifact audit surface, not final self-audit	The repository can make execution, validation, AI proposal handling, and trace artifacts inspectable	Final self-arbitration; proof PMS has settled its own limits; replacement for external critique	SELF-APP / IMPLEME CLAIM
Chapter 11 — What Survives	reduced survivor claims	documented v0.1 Evidence Spine; README non-goals; `pms-docs/ROADMAP.md`; acceptance gates	bounded executable feasibility as documented	PMS-RUST can support a reduced implementation claim: executable artifact-backed partial realization under explicit constraints	Survival as validation; final grammar proof; complete PMS-STACK; empirical adequacy of PMS Base	REDUCED IMPLEME CLAIM
Appendix B — Operator Reference	PMS Base / formal reference boundary	`pms-model-data/PMS.yaml`; `pms-model`; `pms-kernel` runtime opcode names	machine-readable model-reference relation	PMS operator names and dependency constraints can be represented, loaded, normalized, and checked in a runtime-adjacent form	Redefinition of Δ–Ψ; base authority; proof that YAML implements praxis; full PMS 1.3 theoretical identity	BASE-REI IMPLEME CONSTR CLAIM
Appendix C — Layer /	layer discipline	README non-goals; architecture notes; AI	documented boundary	Repo artifacts can remain	Repo as validation layer;	IMPLEME / ARTIFA

Chapter / Claim	Relevant PMS Layer	Relevant Repo Module / Crate / Folder	Implementation Status	What the Repo Demonstrates	What It Does Not Demonstrate	Clai
Stack Position Matrix		trust boundary; acceptance gates; crate separation	controls	claim-typed as implementation artifacts rather than base, domain, bridge-proof, or validation layers	repo as source-of-truth override; implementation as proof	
Appendix D — Chapter-to-Source Matrix	source accounting	repository README; `pms-docs/ARCHITECTURE.md`; `pms-docs/ACCEPTANCE_GATES.md`; `pms-docs/AI_BRIDGE.md`; `pms-docs/DEMO.md`; `pms-docs/EVENT_FORMAT.md`	public source material for implementation-boundary mapping	Source relation between chapter claims and repo artifacts can now be stated explicitly	Replacement of chapter argument, PMS Base, domain sources, QC source, or external validation	SOURCE-IMPLEME CLAIM

The matrix supports only the following strengthened claim:

```
PMS-RUST documents a bounded executable artifact-backed partial realization of a PMS-STA
```

It does not support:

```
PMS-RUST validates PMS Base.
PMS-RUST completes PMS-STACK.
PMS-RUST proves PMS as final grammar.
PMS-RUST proves PMS 1.3 semantic completion.
PMS-RUST replaces external validation.
```

The repository's own non-goals support this reduction: it states that PMS-STACK OS, PMS-CPU/ISA, PMSL, and distributed PMS are not implemented, that AI output is not trusted executable code, that add-ons are not runtime/validation/policy engines, and that PMS 1.3 semantic migration is not complete.

G.4 Repo Component Matrix

This table records the currently documented PMS-RUST v0.1 repository components. It does not treat code structure as theory, tests as proof, or module names as operators.

Repo Component	PMS Concept / Layer	Operator or Derived Construct Involved	Chapter Link	Evidence Type	Boundary Condition	Open Risk
`Cargo.toml` / workspace	implementation architecture / STACK	workspace-level crate separation	Chapter 4; Appendix G	code structure; workspace metadata	Workspace structure shows implementation organization, not PMS truth.	architecture-as-ontology drift
`Cargo.lock`	build reproducibility support	dependency lock state	Chapter 7; Appendix G	artifact trace; build metadata	Dependency locking supports reproducible builds; it does not prove PMS adequacy.	reproducibility-as-validation drift
`pms-kernel`	deterministic runtime kernel / STACK	runtime opcodes corresponding to Δ–Ψ names in the v0.1 runtime dialect	Chapter 4; Chapter 7; Appendix G	code structure; runtime behavior; event emission	The kernel executes a bounded runtime subset; it does not define PMS Base.	module-as-operator drift; executable-as-complete drift
`pms-model`	model validation / formal representation bridge	`PMS.yaml` dependency table; operator name normalization	Chapter 4; Appendix B; Appendix G	schema representation; validation behavior	Model validation supports adjacency checks; it does not make YAML an implementation of praxis.	schema-as-proof drift
`pms-model-data/PMS.yaml`	machine-readable model reference	dependency table; operator identifiers	Appendix B; Appendix G	schema / repository data	Machine-readability supports validation and inspection; it does not validate PMS	YAML-as-proof drift

Repo Component	PMS Concept / Layer	Operator or Derived Construct Involved	Chapter Link	Evidence Type	Boundary Condition	Open Risk
					Base.	
`pms-ir`	IR convenience layer	`PmsBuilder`; opcode buffer; program construction	Chapter 4; Chapter 7; Appendix G	builder / code structure	IR convenience supports program construction; it is not PMS theory.	builder-as-grammar drift
`pms-repl`	user-facing runtime tool	REPL DSL; `.pms` script runner; buffer inspection; model loading; JSONL dump/ load	Chapter 7; Appendix G	runtime tooling; script execution; artifact trace	REPL execution shows toolchain behavior under constraints; it does not prove praxis adequacy.	tool-as-validation drift
`pms-ai-bridge`	controlled AI proposal bridge	typed request/ response envelopes; opcode whitelist; advisory analyze / compile proposal	Chapter 7; Chapter 10; Appendix G	trust-boundary design; fail-closed bridge	AI proposes; host parsing, validation, and kernel execution retain authority.	AI-as-executor drift; AI-as-validation drift
`pms-docs`	architecture / gates / event format / roadmap	architecture docs; acceptance gates; event schema; demo documentation	Appendix C; Appendix G	design document; gate documentation	Documentation states scope, non-goals, and intended behavior; docs are not implementation by themselves.	README-overclaim drift
`pms-docs/ ACCEPTANCE_GATES.md`	gate discipline / artifact status	format, test, clippy, demo, golden-test acceptance gates	Chapter 7; Chapter 11; Appendix G	gate documentation	Gate documentation supports artifact quality claims; it does not create external	gate-as-warrant drift

Repo Component	PMS Concept / Layer	Operator or Derived Construct Involved	Chapter Link	Evidence Type	Boundary Condition	Open Risk
					validation.	
pms-docs/ ARCHITECTURE.md	implementation architecture	runtime/ toolchain spine; v0.1 dialect; boundary notes	Chapter 4; Chapter 11; Appendix G	architecture documentation	Architecture describes implementation design; it does not establish ontology or PMS Base validity.	architecture-as-ontology drift
pms-docs/ AI_BRIDGE.md	AI trust-boundary documentation	AI proposal / host enforcement / fail-closed execution boundary	Chapter 7; Chapter 10; Appendix G	trust-boundary documentation	AI output is advisory/ proposal-only; execution authority remains with host validation and kernel constraints.	AI-as-validation drift
pms-docs/ EVENT_FORMAT.md	trace / audit format	KernelEvent; JSONL event structure	Chapter 7; Chapter 10; Appendix G	event-format documentation	Event format makes runtime behavior inspectable; it does not make inspection self-validation.	trace-as-proof drift
pms-docs/DEMO.md	demo workflow documentation	documented demo paths	Chapter 7; Appendix G	demo documentation	Demo workflows show bounded reproducibility paths; they do not establish general adequacy.	demo-as-proof drift
pms-docs/ KEY_WORLDS.md	terminology / dialect boundary	runtime/world-language distinctions	Chapter 4; Chapter 8; Appendix G	glossary / boundary documentation	Terminology helps prevent layer confusion; it does not decide theory.	terminology-as-authority drift

Repo Component	PMS Concept / Layer	Operator or Derived Construct Involved	Chapter Link	Evidence Type	Boundary Condition	Open Risk
`pms-docs/ROADMAP.md`	planned development boundary	Release+ work; future scope	Chapter 11; Appendix G	roadmap documentation	Planned work may identify future pressure; it is not executable artifact-backed realization.	roadmap-as-completion drift
`pms-docs/golden`	golden fixtures and scripts	`.pms` scripts; expected JSONL events; AI specs	Chapter 7; Chapter 10; Appendix G	golden tests; artifact traces	Golden fixtures support reproducibility and regression checks; they do not prove PMS truth.	golden-test-as-proof drift
`pms-demos`	checked-in demo scripts	valid and invalid `.pms` script examples	Chapter 7; Appendix G	demo scripts; reproducibility surface	Demos show bounded behavior paths; they do not establish general adequacy.	demo-as-proof drift
`pms-examples`	example crate / runnable example surface	example binary or integration surface	Chapter 7; Appendix G	example code	Examples show usage patterns; they are not validation.	example-as-proof drift
`tests`	artifact testing	`.pms` script fixtures	Chapter 7; Appendix G	test fixtures	Tests show specified behavior in specified cases only.	test-as-proof drift
`pms.bat`	local launcher	convenience startup wrapper	Appendix G	tooling wrapper	A launcher helps run the toolchain; it is not evidence beyond the command it invokes.	launcher-as-evidence drift

Repo Component	PMS Concept / Layer	Operator or Derived Construct Involved	Chapter Link	Evidence Type	Boundary Condition	Open Risk
`test.bat`	local gate runner	format / test / clippy command wrapper	Chapter 7; Appendix G	gate wrapper	Gate scripts help reproduce checks; passing gates remain artifact-level evidence only.	gate-as-warrant drift
`set_envs.bat`	local environment helper	environment setup	Appendix G	tooling wrapper	Environment setup supports local execution; it is not theoretical evidence.	setup-as-evidence drift
`README.md`	public repository overview	repo status, scope, non-goals, quickstart	Appendix C; Appendix G	documentation / source status	README claims must remain checked against code, tests, and traces.	README-overclaim drift
`screenshot.png`	visual artifact	REPL startup / operator banner evidence	Appendix G	visual documentation	A screenshot may illustrate runtime presentation; it does not prove runtime adequacy.	screenshot-as-proof drift

No repo component should be treated as corresponding to a PMS operator merely because its name resembles an operator.

The rule remains:

```
A module name is not an operator.
A test is not proof.
A repository is not PMS Base.
```

The component matrix strengthens the implementation claim only:

```
PMS-RUST contains documented implementation components that make parts of the PMS-STACK
```

It does not strengthen PMS Base.

G.5 Implementation Boundary Table

Status	Means	Does Not Mean
PMS Base	The reference layer for Δ–Ψ, dependency order, derived axes, and base guardrails.	That implementation, YAML, Rust, QC, or STACK can redefine the operators.
`PMS.yaml` / formal representation	The operator grammar and dependency constraints are machine-readable as repository data.	Praxis is implemented; PMS is validated; interoperability or explanatory sufficiency is proven.
Schema representation	A structure can be encoded in a formal or machine-readable format.	The encoded structure has been realized as runtime behavior.
Model validation	The runtime/toolchain can load model data, normalize operator names, and check allowed adjacency transitions.	The model is empirically adequate, complete, final, or identical to praxis.
Implementation architecture	PMS-derived distinctions are projected into system design, crate layout, module structure, runtime concepts, or STACK architecture.	Architecture proves PMS Base, ontology, universality, or empirical adequacy.
Rust prototype / PMS-RUST v0.1	PMS-RUST documents a bounded executable PMS-STACK Evidence Spine v0.1: deterministic runtime, model validation, REPL/script tooling, JSONL trace/audit, demos, tests, and AI proposal bridge.	PMS Base is validated, final, complete, uniquely correct, or fully migrated to PMS 1.3 semantics.
Executable artifact	A component can run under specified conditions.	Praxis itself has been implemented, or PMS has been proven as a grammar.
REPL / script execution	`.pms` scripts or interactive commands can be parsed, buffered, validated, executed, and traced under toolchain constraints.	PMS is practically sufficient, universally applicable, or externally validated.
JSONL trace / audit output	Runtime behavior can be serialized and inspected as event artifacts.	Traceability is self-validation, full audit, or proof of theoretical adequacy.
Golden fixtures	Expected scripts, traces, or AI specs can be compared for reproducibility and regression control.	Golden outputs prove PMS truth, domain adequacy, or complete semantic coverage.
Passing tests	The repository reports that the specified v0.1 workspace gates pass, including format check, workspace tests, and clippy with warnings denied.	PMS is empirically adequate, universally valid, externally validated as praxis grammar, or proven by test success.
Acceptance gates	The repository defines release-quality checks for the artifact.	Passing gates create external warrant for PMS Base.
AI bridge	AI can propose opcode programs or analysis under host-side parsing, canonicalization, mapping, validation, and kernel constraints.	AI output is trusted executable code, independent validation, or autonomous PMS reasoning.
Execution trace	A recorded run shows artifact behavior under a given configuration.	The artifact captures all relevant PMS structures or proves generality.

Status	Means	Does Not Mean
External validation	Independent application, comparison, empirical test, or rival-sensitive evaluation may support warrant if properly designed.	Under Load itself or PMS-RUST has already supplied that warrant.
Documentation / README	The repository states intent, usage, architecture, status, non-goals, gates, and boundary conditions.	The stated intent is complete, proof-bearing, independently validated, or sufficient without code / test / trace inspection.
Roadmap	Future work is identified as planned or Release+ scope.	Planned work is implemented or already evidence-bearing.

The core distinction is:

```
Formal representation is not implementation architecture.
Implementation architecture is not executable artifact.
Executable artifact is not runtime adequacy.
Runtime adequacy is not external validation.
External validation is not PMS Base finality.
```

A stronger version for PMS-RUST is:

```
Repository executability changes the status of the implementation claim.
It does not change the status of PMS Base.
```

G.6 Repo Drift Risks

The following risks must be checked whenever repository material is used inside *PMS Under Load*.

Drift Risk	Definition	Boundary Control
repo-as-validation drift	The repository is treated as proof of PMS Base.	Repo evidence may support implementation claims only.
test-as-proof drift	Passing tests are treated as proof of PMS adequacy.	Tests show behavior under specified cases only.
gate-as-warrant drift	Passing repository gates are treated as external warrant for PMS as praxis grammar.	Gates support artifact quality and bounded execution behavior only.
module-as-operator drift	Rust modules, structs, crates, or folders are treated as PMS operators.	Only PMS Base / PMS.yaml define operators.
architecture-as-ontology drift	System architecture is treated as evidence about what praxis really is.	Architecture is implementation design, not ontology.
executable-as-complete drift	Executable components are treated as complete realization of PMS.	Executability is bounded and artifact-specific.
README-overclaim drift	Documentation language exceeds demonstrated implementation status.	README claims must be checked against code, tests, and traces.
prototype-as-system drift	A prototype is treated as finished system.	Prototype status must remain explicit.
mapping-as-equivalence drift	Repo mappings are treated as equivalence between PMS and target system.	Mapping is representational or architectural, not identity.
research-bridge-as-proof drift	Research usefulness is treated as validation.	Research bridge supports inquiry, not final warrant.
schema-as-proof drift	PMS.yaml or model validation is treated as proof that praxis is implemented.	Schema and validation support representation and adjacency checks only.
YAML-as-implementation drift	Machine-readable YAML is treated as implemented praxis.	YAML is formal representation; runtime behavior and external adequacy require separate evidence.
dialect-as-theory drift	The Rust runtime dialect is treated as identical to PMS 1.3 theoretical semantics.	Runtime vocabulary must remain version- and scope-bound.
Chi-as-Distance drift	Runtime `Chi / X` as isolation / boundary scope is treated as identical to PMS 1.3 X as Distance.	The Rust dialect must not override the PMS Base meaning of X.
AI-as-executor drift	The AI bridge is treated as trusted execution or autonomous agency.	AI output is advisory/proposal-only; host parsing, validation, and kernel execution remain authoritative.
AI-as-validation drift	AI analysis is treated as confirmation that PMS is correct.	AI analyze output is artifact-level commentary, not validation.

Drift Risk	Definition	Boundary Control
trace-as-proof drift	JSONL traces are treated as proof that PMS has captured praxis.	Traces show runtime behavior under specified conditions.
golden-test-as-proof drift	Golden fixtures are treated as proof of PMS truth.	Golden tests support regression control, not theoretical warrant.
roadmap-as-completion drift	Planned roadmap items are treated as already implemented.	Roadmap items remain non-evidence until implemented and documented.
launcher-as-evidence drift	Convenience scripts such as `pms.bat` or `test.bat` are treated as independent evidence.	Launchers only wrap commands; evidence belongs to the invoked runtime, gates, or traces.

These drift risks are not accusations against the repository.

They are controls on how the repository may be used inside *PMS Under Load*.

The working rule is:

```
Repository evidence strengthens implementation status only where the artifact, gate, tra
```

G.7 Completion and Remaining Evidence Conditions

Appendix G is no longer merely a placeholder. Repository evidence has been supplied through the public PMS-RUST repository, its README, crate layout, documentation folders, demo scripts, tests, acceptance-gate documentation, and implementation-boundary notes.

The current repository evidence supports:

```
repository existence
public README status
workspace / crate layout
documented v0.1 scope
documented non-goals
documented gate status
documented architecture
documented AI trust boundary
documented REPL / script workflow
documented PMS.yaml model-validation relation
documented JSONL trace / audit surface
documented demos and golden fixtures
```

This is sufficient for a first implementation-boundary mapping.

It is not sufficient for every possible implementation claim.

The current repository evidence does not by itself support:

```
independent reproduction of test results
full code-level semantic audit
external validation of PMS as praxis grammar
complete PMS 1.3 semantic migration
complete PMS-STACK OS
complete PMS-CPU / ISA
complete PMSL compiler
distributed PMS runtime
empirical adequacy of PMS Base
```

A later stronger Appendix G pass may inspect:

```
specific Rust source files
Cargo workspace metadata
kernel transition logic
model-validation internals
REPL parser behavior
AI bridge enforcement paths
test fixtures
golden traces
generated JSONL examples
acceptance-gate artifacts
roadmap deltas
commit history
release tags
```

Until then, Appendix G uses the repository as documented implementation evidence, not as independently reproduced runtime proof.

The working rule is:

```
Public repo documentation is enough for implementation-boundary mapping.
Code-level claims require code-level inspection.
Execution claims require execution evidence.
Validation claims require external validation beyond the repo.
```

G.8 Required Reduction

Implementation pressure may strengthen PMS as a research program, but it does not validate PMS as a final grammar.

The appendix therefore permits the following claim:

```
PMS-RUST supports a bounded IMPLEMENTATION / ARTIFACT CLAIM: executable artifact-backed
```

It also permits the narrower claim:

```
PMS-RUST makes parts of the PMS-STACK Evidence Spine executable, inspectable, and testab
```

It does not permit:

```
The Rust project proves PMS Base.
The Rust project validates PMS as final grammar.
The Rust project shows PMS is empirically adequate.
The Rust project replaces external validation.
The Rust project turns STACK into proof.
The Rust project completes PMS 1.3 semantic migration.
The Rust project implements full PMS-OS, PMS-CPU / ISA, PMSL, or distributed PMS.
The Rust project makes repository evidence equivalent to domain validation.
```

Final boundary sentence:

```
Appendix G treats the repository as implementation pressure and artifact evidence under
```

The final reduction is:

```
Executable realization changes the status of the implementation claim.
It does not change the status of PMS Base.
```